

基于点质量模型的民用客机巡航爬升轨迹与燃油优化

摘要

针对民用客机巡航阶段燃油消耗集中、飞行高度与速度相互制约且高空风场不可忽略的特点,本文围绕高度、速度、质量、阻力、油耗和风场之间的耦合关系,建立“经验策略基线–固定路径油耗积分–无风最优控制–有风综合比较”的递进模型框架。模型以点质量动力学、升降平衡和燃油率方程为基础,并在不同子问题中分别固定终止质量、共同航程或终端机械状态,使策略比较、油耗复算和优化建模具有一致的物理口径。

对于问题一,建立小航迹角准稳态点质量模型,将水平运动、质量衰减、升力平衡、阻力极曲线和燃油消耗率在时间域中联立,以参考升力系数 $C_{L,ref}$ 为闭合条件,分别推导等速和等马赫两类巡航爬升策略。终止质量 62000 kg 下两策略总油耗均为 10450 kg;等速策略终点高度 10637.065 m、时间 761.917 s,等马赫策略终点高度 10437.818 m、时间 811.032 s、航程 211331.296 m。固定终止质量口径下油耗无区分度,比较重心转向高度和时间剖面差异;等马赫策略航程更大源于飞行时间延长,终点高度较低则由声速随高度下降与升力平衡共同决定。

对于问题二,在问题一等速路径上建立燃油路径积分模型,将标准 ISA 大气中温度、压力、密度和声速的高度连续分布逐层传递到升力系数、阻力、推力需求和燃油率。为避免固定终止质量掩盖大气修正的细微影响,取两策略共同可行航程 189781.310 m 作为比较基准。标准 ISA 总油耗 10427.256 kg, +10 K 静力温差修正后 10450.000 kg,相对增加 0.218%。时间域与路径域复算一致,静力平衡残差约 10^{-10} ; $|\Delta T| \leq 10$ K 时油耗变化不超过 0.3%,当前航程尺度下常温偏差属二阶修正。

对于问题三,将经验策略推广为固定航程、固定终端高度和速度、终端质量自由的最优控制问题,以最大化终端质量等价刻画最小油耗,避免“固定终止质量再最小化油耗”导致目标退化。求解分两阶段:先用航程域直接配点法构造连续可行初值,再以降维控制打靶法进行燃油优化。241 节点网格上无风优化得到总油耗 10342.814 kg、终端质量 62107.186 kg、终端时间 805.679 s,较可行初值降低 107.186 kg。终端速度重积分误差 1.71×10^{-5} m/s、燃油恒等式残差 8.58×10^{-5} kg,数值精度满足复核要求。

对于问题四,在问题三无风最优基础上引入随高度变化的风场,建立有风航程域最优控制模型,采用降维控制打靶法进行局部优化。固定共同航程 189781.310 m 下有风优化总油耗 9863.840 kg,较等速基线节省 5.403%;固定燃油 10450 kg 口径下航程下界 201168.189 m,较共同航程增大 6.0%。速度偏离惩罚系数 β 降至标称值 0.8 倍时燃油减少 14.116 kg,增至 1.2 倍时增加 6.230 kg,最优轨迹对 β 敏感度较低。

本文模型从经验策略逐步推进到最优控制,四个问题层层递进,各层数值结果均经交叉验证,可为巡航飞行最优设计提供方法参考。

关键词: 准稳态点质量模型; 燃油路径积分; 最优控制; 降维控制打靶法; 灵敏度分析

一、问题重述

1.1 问题背景

巡航阶段是民用客机燃油消耗最集中的飞行阶段之一，也是航空公司降低运营成本和排放强度的重要环节。巡航爬升过程中，飞行高度改变大气密度和声速，飞行速度影响升阻特性与空中时间，燃油消耗引起飞机质量持续下降，高空风场又改变空速到地速的转换关系。上述因素共同决定单位航程油耗，因此本题的核心不是单独优化高度或速度，而是在给定飞机与大气条件下刻画二者的联合变化规律。

题目给出某型客机的质量、机翼面积、阻力极曲线、发动机油耗参数、标准大气近似和高度相关风场等参数，并指定初始高度 9500 m、初始空速 240 m/s 与终止剩余质量 62000 kg。据此需建立可用于轨迹计算、油耗积分和策略比较的巡航模型，为传统经验策略与优化策略的定量评价提供统一基础。

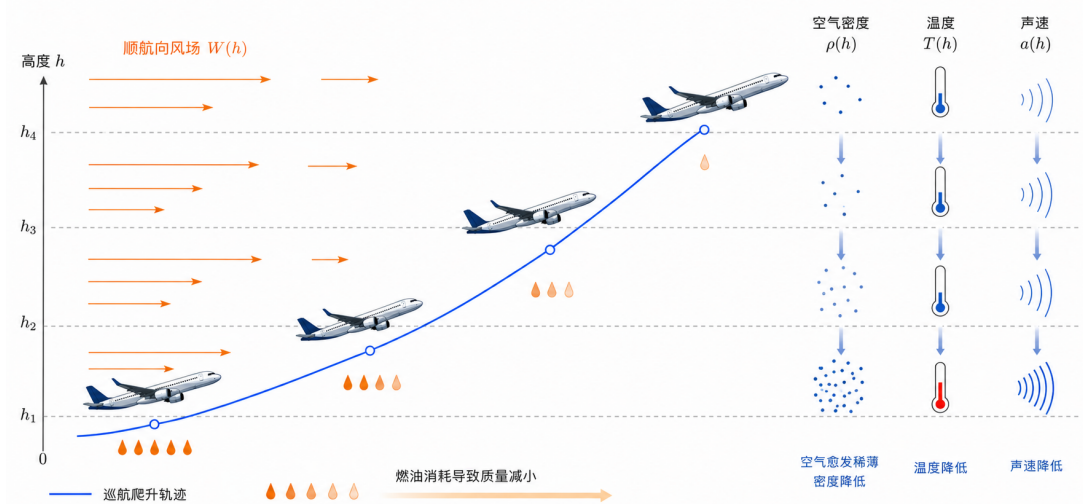


图1 民用客机巡航爬升物理场景示意图

1.2 问题提出

围绕该型客机巡航过程中的高度-速度联合决策，现要求我们解决以下问题：

问题一：在给定飞机气动和油耗参数下，建立描述高度、速度、航程和质量随时间变化的巡航动力学模型；以等速和等马赫数两类经验策略为对象，计算其高度、质量、飞行时间和燃油消耗，并比较两种策略的爬升特征。

问题二：将问题一得到的飞行过程转化为沿航程的燃油积分问题，重点考察大气温度、密度和声速随高度变化对油耗率的影响；在标准大气和温度偏差条件下分别计算总油耗，判断温度修正是否需要纳入燃油评估。

问题三：把高度和速度由预设经验策略推广为待优化轨迹，在动力学约束、飞行包线和终端要求下建立最小油耗模型；给出最优性条件和数值求解思路，并为无风和有风

数值求解提供统一模型结构。

问题四：整合前三问模型形成综合计算框架，在问题三无风最终优化轨迹基础上引入有风航程域优化，对经验策略和有风局部优化策略进行同口径比较，输出燃油节省、航程变化和关键参数敏感性结论；同时给出若干面向工程应用的模型扩展方向。

二、问题分析

巡航爬升本质上是能量管理和气动约束共同作用下的轨迹优化问题。相关飞行动力学教材通常以点质量模型、升阻极曲线和飞行包线刻画巡航阶段的主要物理关系 [1,2]；在数值求解层面，直接配点和打靶法常用于把连续最优控制问题转化为带约束的非线性规划 [3,4]。因此，本文将四个问题组织为由经验策略到最优控制、由无风到有风的递进链条，并用独立重积分和灵敏度分析限定各结论的适用范围。

2.1 问题一：经验策略闭合与动力学积分

问题一的输入是初始状态、终止质量和两类经验速度策略，输出应包括高度、航程、时间和燃油消耗。其建模难点在于：等速或等马赫只限定速度变化，并不能单独决定巡航爬升高度。本文以小航迹角准稳态模型连接升力平衡、能量方程和质量衰减，再用参考升力系数保持不变补足高度闭合条件。这样，等速与等马赫策略可在同一气动假设下积分求解，比较结果也不依赖额外人为控制律。

2.2 问题二：固定路径油耗与大气修正

问题二继承问题一的等速几何路径，但评价对象从“达到给定终止质量”转为“同一航程下消耗多少燃油”。若继续固定终止质量，各大气场景总油耗都会等于 $m_0 - m_f$ ，温差影响被比较口径抵消。本文固定路径和真空速，在标准 ISA 与常温偏差大气下重新计算密度、声速、升力系数、阻力和推力需求，并用共同可行航程统一比较区间。温差模型同步修正压力和密度，使静力平衡成立，从而把温度偏差的作用限定为同一几何路径上的气动与油耗变化。

2.3 问题三：固定端点的高度-速度最优控制

问题三要求飞机在固定航程和给定起止高度与速度的条件下，自主选择高度、速度、推力和航迹角随时间的变化规律，使总燃油消耗最小。此时高度和速度不再是预设经验值，而是待优化未知曲线，因此需要建立以小航迹角点质量模型为状态方程、以推力和航迹角为控制变量的连续最优控制模型；目标函数等价于最大化终端质量，状态方程包含水平位置、高度、空速和质量四个微分方程，并受飞行包线和起止边界约束。数值上先用航程域直接配点法将连续最优控制转化为带约束非线性规划以构造可行初值，再用降维控制打靶法求解最终燃油优化。

2.4 问题四：综合比较与灵敏度分析

问题四以前三问的计算结果为基础，在问题三无风最优轨迹上引入有风航程域最优控制，定量比较等速经验策略与有风最优策略在固定共同航程下的燃油消耗差异。同时，在固定燃油预算口径下估计可达航程下界，并对速度偏离惩罚系数 β 进行逐场景重新求解式灵敏度分析，讨论模型扩展方向。

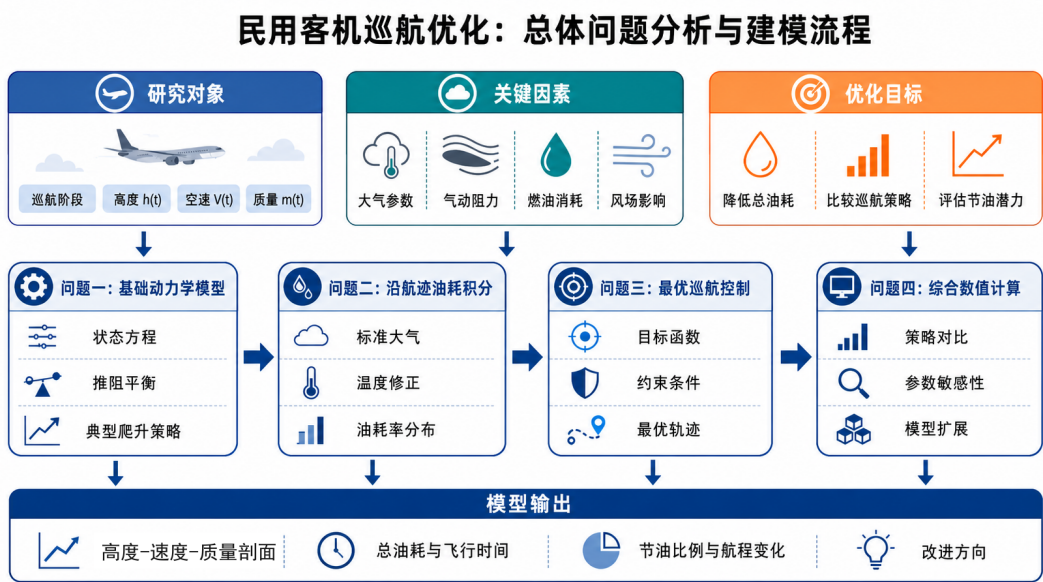


图 2 总体问题分析与建模流程图
[9]

三、 模型假设

1. **小航迹角准稳态**。巡航爬升过程中航迹角较小，升力近似平衡重力，地速近似为空速与风速之和。
2. **参考升力系数闭合**。题目仅给出策略名称而未给推力包线，故采用参考升力系数保持不变的闭合条件使两类策略可比。
3. **风场仅随高度变化**。风速正值表示顺航向风，风场只随高度变化，不考虑时间和水平位置变化。
4. **抛物线阻力极曲线**。阻力系数由零升阻力系数与升致阻力因子的抛物线关系确定，不引入跨音速波阻项。
5. **题设燃油消耗系数**。主结果采用题设推力燃油消耗系数，另以低一个数量级的值作工程合理性对照，不静默替换题设值。

四、符号说明

表 1 本文的主要符号说明

符号	含义	单位
t	飞行时间	s
$x(t)$	相对地面水平距离	m
$h(t)$	飞行高度	m
$V(t)$	真空速	m/s
$m(t)$	飞机质量	kg
$F(t)$	发动机推力	N
$D(t)$	气动阻力	N
$L(t)$	升力	N
S	机翼面积	m ²
C_L	升力系数	—
C_D	阻力系数	—
C_{D0}	零升阻力系数	—
$C_{L,\text{ref}}$	参考升力系数（作为策略闭合基准）	—
k	升致阻力因子	—
$T_{\text{atm}}(h)$	大气温度, $T_{\text{atm}}(h) = T_{\text{sl}} - L_T h$	K
$\rho(h)$	高度 h 处空气密度	kg/m ³
$a(h)$	高度 h 处声速	m/s
M	马赫数	—
$W(h)$	顺航向风速	m/s
c_F	推力燃油消耗系数	kg/(N · s)
β	速度偏离惩罚系数	s ² /m ²
V_{opt}	最佳效率速度	m/s
$\Phi(V)$	速度偏离惩罚函数, $\Phi = 1 + \beta(V - V_{\text{opt}})^2$	—
q_f	瞬时燃油消耗率	kg/s
J	总燃油消耗量	kg
m_0, m_f	初始质量、终止质量	kg
X_{ref}	共同航程（固定路径积分参考航程）	m

五、问题一模型的建立与求解

5.1 数据预处理

计算前先统一题设参数和单位。飞机初始质量为 $m_0 = 72450 \text{ kg}$ ，终止质量为 $m_f = 62000 \text{ kg}$ ，初始高度和真空速分别为 $h_0 = 9500 \text{ m}$ 、 $V_0 = 240 \text{ m/s}$ 。气动参数取机翼面积 $S = 112.3 \text{ m}^2$ 、零升阻力系数 $C_{D0} = 0.022$ 、升致阻力因子 $k = 0.045$ ；燃油模型取推力燃油消耗系数 $c_F = 2.8 \times 10^{-4} \text{ kg}/(\text{N} \cdot \text{s})$ 、速度偏离惩罚系数 $\beta = 0.00003 \text{ s}^2/\text{m}^2$ 和最佳效率速度 $V_{\text{opt}} = 235 \text{ m/s}$ 。

大气密度采用指数近似 $\rho(h) = \rho_0 \exp(-h/H_\rho)$ ，其中 $\rho_0 = 1.225 \text{ kg}/\text{m}^3$ 、 $H_\rho = 7300 \text{ m}$ ；声速采用标准对流层近似 $a(h) = \sqrt{\gamma R T_{\text{atm}}(h)}$ ，其中 $T_{\text{atm}}(h) = T_0 - L_T h$ 。风场按题设确认公式 $W(h) = 20 + 3 \times 10^{-5}(h - 10000)^2 \text{ m/s}$ 计算，正值表示顺航向风。上述预处理保证质量、长度、速度和油耗系数在后续方程中使用同一单位体系。

5.2 模型建立

小航迹角准稳态巡航爬升满足以下基本方程 [1,2]。

水平地面距离变化由空速和风速共同决定：

$$\frac{dx}{dt} = V + W(h). \quad (1)$$

升力近似平衡重力：

$$L = \frac{1}{2} \rho(h) V^2 S C_L \approx mg, \quad (2)$$

因此升力系数为

$$C_L = \frac{2mg}{\rho(h) V^2 S}. \quad (3)$$

阻力系数采用抛物线阻力极曲线 [2]：

$$C_D = C_{D0} + k C_L^2, \quad (4)$$

阻力为

$$D = \frac{1}{2} \rho(h) V^2 S C_D. \quad (5)$$

小航迹角下，纵向能量方程可写为

$$m \frac{dV}{dt} = F - D - mg \sin \gamma \approx F - D - mg \frac{1}{V} \frac{dh}{dt}, \quad (6)$$

即

$$F = D + m \frac{dV}{dt} + \frac{mg}{V} \frac{dh}{dt}. \quad (7)$$

燃油消耗率采用题设的速度偏离惩罚形式：

$$\frac{dm}{dt} = -c_F F [1 + \beta(V - V_{\text{opt}})^2]. \quad (8)$$

其中括号内为无量纲惩罚项，当 V 偏离最佳效率速度 V_{opt} 时，单位推力燃油消耗率增加。

(2)–(8) 构成巡航爬升的基本动力学模型。模型中，高度 $h(t)$ 和空速 $V(t)$ 决定气动状态与推力需求，水平距离 $x(t)$ 和飞机质量 $m(t)$ 则记录航程推进和燃油消耗；后续两类经验策略的差别体现在 $h(t)$ 与 $V(t)$ 的闭合方式上。

5.3 等速与等马赫的飞行方式

题目要求比较等速巡航爬升与等马赫数巡航爬升，但这两种策略本身只约束速度或马赫数，尚不能唯一给出高度演化。为使两种策略在同一气动基准下比较，本文采用由初始状态确定的统一闭合条件

$$C_L = C_{L,\text{ref}} = \frac{2m_0g}{\rho(h_0)V_0^2S}. \quad (9)$$

由题设初始状态计算得

$$C_{L,\text{ref}} = 0.658914. \quad (10)$$

该值接近抛物线阻力极曲线下最大升阻比升力系数

$$C_{L,(L/D)_{\text{max}}} = \sqrt{\frac{C_{D0}}{k}} = 0.699206. \quad (11)$$

因此，固定参考升力系数可解释为飞机通过迎角调整维持接近高升阻比的巡航爬升状态。该闭合条件不是唯一控制律，但能在题面未给推力包线和迎角控制细节时给出可复算的策略基线。下面分别写出等速和等马赫两种策略下的闭合关系，再用图形对两者的物理差异作直观说明。

策略 A：等真空速巡航爬升。 令

$$V(t) = V_0 = 240 \text{ m/s}. \quad (12)$$

由 $C_L = C_{L,\text{ref}}$ 与升力平衡 (2) 可得

$$\frac{m(t)}{m_0} = \frac{\rho(h(t))}{\rho(h_0)}. \quad (13)$$

代入指数密度模型 $\rho(h) = \rho_0 \exp(-h/H_\rho)$ ，得到高度与质量关系：

$$h(t) = h_0 - H_\rho \ln \frac{m(t)}{m_0}. \quad (14)$$

因此

$$\frac{dh}{dm} = -\frac{H_\rho}{m}, \quad \frac{dV}{dm} = 0. \quad (15)$$

策略 B：等马赫数巡航爬升。 令

$$M(t) = M_0 = \frac{V_0}{a(h_0)}, \quad V(t) = M_0 a(h(t)). \quad (16)$$

由升力闭合条件 (2)–(9) 可得

$$\frac{m(t)}{m_0} = \frac{\rho(h(t))a^2(h(t))}{\rho(h_0)a^2(h_0)}. \quad (17)$$

对其求微分，得到

$$\frac{dh}{dm} = \frac{1}{m \left[\frac{d \ln \rho}{dh} + 2 \frac{d \ln a}{dh} \right]}, \quad (18)$$

并且

$$\frac{dV}{dm} = V \frac{d \ln a}{dh} \frac{dh}{dm}. \quad (19)$$

由上述两组关系可知，两种策略都由升力平衡闭合高度，但等速策略只通过密度降低来适应质量下降，等马赫策略还叠加了声速随高度下降引起的空速变化。图 3 将这种“同一质量衰减、不同高度闭合”的差异直接画出。

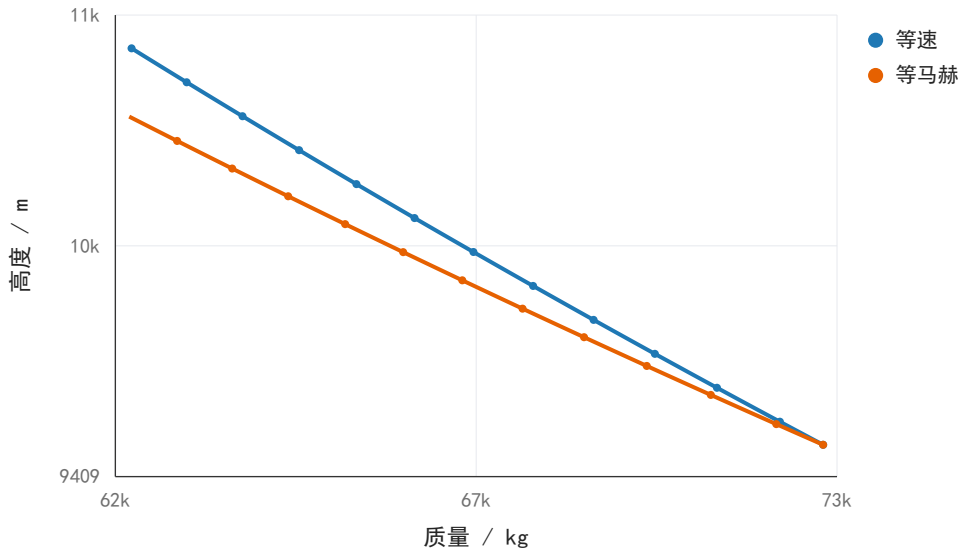


图 3 问题一两类经验策略的升力平衡闭合机理

由图 3 可见，质量从 72450 kg 降至 62000 kg 时，两类策略均随质量降低而爬升；等速策略马赫数随高度升高而增大，等马赫策略则通过降低真空速抵消部分密度下降，因此终点高度较低。

5.4 一维质量方程与数值求解

两类策略均可表示为 $h = h(m)$ 、 $V = V(m)$ 。记

$$A(m) = m \frac{dV}{dm} + \frac{mg}{V} \frac{dh}{dm}, \quad \Phi(V) = 1 + \beta(V - V_{\text{opt}})^2. \quad (20)$$

由能量方程 (7) 和质量方程 (8) 联立，有

$$F = D + A(m) \frac{dm}{dt}, \quad (21)$$

$$\frac{dm}{dt} = -c_F \Phi(V) \left[D + A(m) \frac{dm}{dt} \right]. \quad (22)$$

解得

$$\frac{dm}{dt} = -\frac{c_F \Phi(V) D}{1 + c_F \Phi(V) A(m)}. \quad (23)$$

该式将巡航爬升过程化为关于质量的一维常微分方程。本文从 $m_0 = 72450 \text{ kg}$ 积分至 $m_f = 62000 \text{ kg}$ ，并同步记录 $h(t)$ 、 $V(t)$ 、 $x(t)$ 、 $F(t)$ 及升力平衡、能量方程残差，以便后续检验。

数值求解采用隐式多步积分方法，相对容差 10^{-8} ，绝对容差 10^{-9} ，最大步长 5 s ，用于处理质量方程中推力项和燃油项耦合带来的刚性，提高积分过程的稳定性。

5.5 结果分析

5.5.1 两类策略的对比

表 2 给出两种策略的核心指标。由于题目规定终止质量为 62000 kg ，初始质量为 72450 kg ，所以两种策略在该比较口径下的总燃油消耗均为

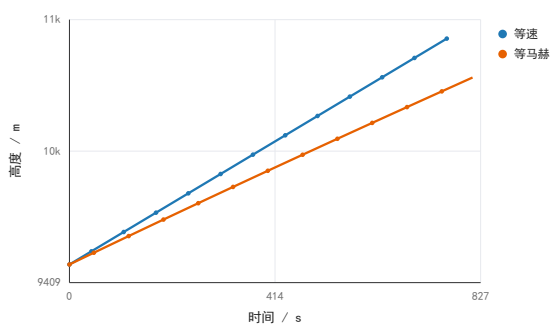
$$m_0 - m_f = 10450 \text{ kg}. \quad (24)$$

因此，问题一在固定终止质量口径下的总油耗没有策略区分度，比较重点应放在飞行时间、航程、最终高度和爬升率剖面上。表 2 先给出精确指标，随后分别用高度图和爬升率图解释这些数值差异从何而来。

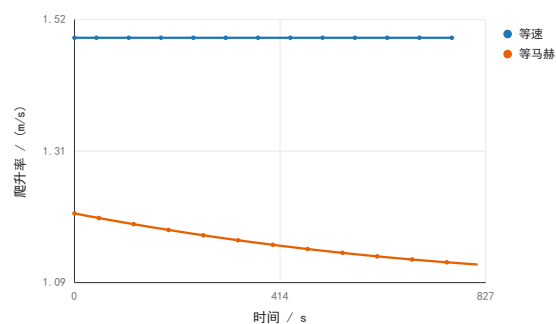
由表 2 可知，等速策略在较短时间内达到更高终止高度，其平均爬升率为 1.492374 m/s ；等马赫策略总飞行时间更长，最终高度较低，但因飞行时间更长且顺风累计作用更充分，地面航程达到 211331.296 m ，高于等速策略的 200668.442 m 。上述差异可从高度剖面和爬升率剖面同时观察，如图 4 所示。

表 2 两种巡航爬升策略的核心指标对比

指标	等速策略	等马赫策略	单位
最终高度	10637.065	10437.818	m
总飞行时间	761.917	811.032	s
地面航程	200668.442	211331.296	m
总燃油消耗	10450.000	10450.000	kg
平均爬升率	1.492374	1.156327	m/s
最大爬升率	1.492374	1.204912	m/s



(a) 高度-时间剖面



(b) 爬升率-时间剖面

图 4 问题一两类经验策略的轨迹剖面对比

图 4 表明,等速策略的高度曲线整体高于等马赫策略,并在 761.917 s 时达到 10637.065 m;其爬升率也在全程保持较高水平。等马赫策略虽用时 811.032 s,但爬升率较低且随时间变化,终点高度仅为 10437.818 m。

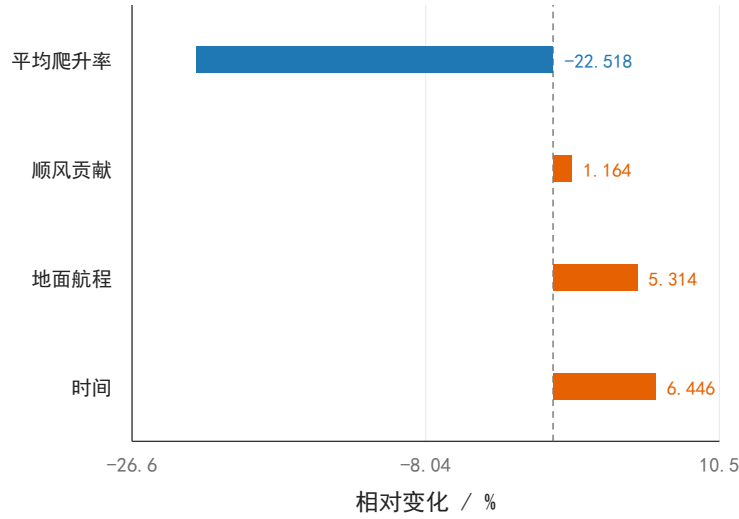


图 5 问题一等马赫策略相对等速策略的核心指标比例

由图 5 可见，等马赫策略飞行时间约为等速策略的 1.064 倍，地面航程约为 1.053 倍，而平均爬升率仅约为 0.775 倍；因此其航程增加主要来自时间延长和顺风累计，而不是更快爬升。

5.5.2 高度和质量剖面解释

等速策略下， $V = V_0$ 固定。随着飞机质量降低，为维持同一初始升力系数，空气密度也需按比例降低，因此飞机必须爬升到更高高度。指数密度模型使高度与质量满足对数关系 (14)，所以在终止质量固定时最终高度由质量比直接决定。

等马赫策略下， $V = M_0 a(h)$ 。在对流层近似中，声速随高度升高而下降，因此等马赫飞行意味着空速随高度升高而降低。升力平衡中的分母 $\rho(h)V^2$ 同时受密度下降和速度下降影响。由于 V^2 的降低削弱了动压，飞机不需要达到等速策略那么高的高度即可满足固定 C_L 的升力平衡关系。因此在本文模型下，等马赫策略最终高度低于等速策略。

这一机制也解释了爬升率差异。题面提示关注等马赫策略可能产生的爬升率优势，但在本文采用的闭合条件、风场和题设参数下，数值证据显示等速策略的平均爬升率更大。因此，本文将结论限定在当前模型和参数范围内：等马赫策略并未表现出更大的平均爬升率，其优势主要体现在更长飞行时间带来的航程增加。

5.5.3 验证与诊断

为检验数值解的可靠性，本文对初始条件、质量单调性、升力平衡、能量方程、步长敏感性、无风/有风对比和极端状态物理合理性进行了检查。验证结果见表 3。

由表 3 可知，两种策略均满足质量单调下降、升力平衡和能量方程约束，残差远低于 10^{-6} 的设定阈值；步长减半或加倍后，最终时间、最终航程和平均爬升率的变化均保

表 3 问题一数值解验证结果

检查项	等速策略	等马赫策略
初始条件误差	0	0
质量单调性	通过	通过
最大升力平衡相对残差	0	0
最大能量方程相对残差	9.72×10^{-17}	9.95×10^{-17}
步长敏感性：最终时间	3.18×10^{-12}	2.61×10^{-12}
步长敏感性：最终航程	6.69×10^{-10}	7.57×10^{-10}
步长敏感性：平均爬升率	6.22×10^{-15}	3.77×10^{-15}

持在极小量级，说明该一维质量积分模型在当前参数下对积分步长不敏感。

5.5.4 灵敏度分析

本文进一步考察推力燃油消耗系数和速度偏离惩罚系数对结果的影响。

将 c_F 降低一个数量级至 2.8×10^{-5} 时，等速策略飞行时间增至 8295.569 s，等马赫策略飞行时间增至 8624.642 s；但两策略最终高度仍分别保持为 10637.065 m 和 10437.818 m。这说明在当前闭合条件下，最终高度主要由升力平衡和终止质量决定，而时间尺度对燃油消耗系数高度敏感。

对 β 作 $\pm 10\%$ 、 $\pm 20\%$ 扰动时，两种策略最终高度同样保持不变，但飞行时间、航程和平均爬升率发生变化。表 4 给出了 β 扰动下的结果。

表 4 β 扰动下的策略指标变化

扰动	等速策略		等马赫策略	
	总时间/s	平均爬升率/($\text{m} \cdot \text{s}^{-1}$)	总时间/s	平均爬升率/($\text{m} \cdot \text{s}^{-1}$)
-20%	773.762	1.469527	817.141	1.147682
-10%	767.798	1.480943	814.073	1.152007
标称值	761.917	1.492374	811.032	1.156327
+10%	756.117	1.503821	808.017	1.160641
+20%	750.397	1.515283	805.029	1.164949

由表 4 可知， β 增大时，速度偏离惩罚加重，燃油消耗率增大，导致在相同终止质量下飞行时间缩短；反之 β 减小时飞行时间延长。该结果表明速度惩罚系数主要通过改变燃油消耗率影响时间和航程，而不改变由 $C_L = C_{L,\text{ref}}$ 与终止质量决定的高度终点。

六、问题二模型的建立与求解

6.1 沿固定航线计算油耗

问题二的关键是把大气修正的影响从固定终止质量口径中分离出来。本文固定问题一等速策略得到的几何路径 $h_{\text{ref}}(x)$ 与真空速路径

$$V_{\text{ref}}(x) = 240 \text{ m/s}, \quad (25)$$

在每种大气场景中重新计算升力系数、阻力、推力需求和油耗率。为保证所有温差场景在同一航程下均满足 $m(t_f) \geq 62000 \text{ kg}$ ，先分别计算各场景沿参考路径达到 62000 kg 的航程，再取其中最短者作为共同可行航程：

$$X_{\text{ref}} = 189781.310 \text{ m}. \quad (26)$$

沿固定路径，地速为

$$V_g(x) = V_{\text{ref}} + W(h_{\text{ref}}(x)). \quad (27)$$

燃油流率仍采用

$$q_f(x) = c_F F_{\text{req}}(x) [1 + \beta(V_{\text{ref}} - V_{\text{opt}})^2]. \quad (28)$$

由于 V_{ref} 固定，速度偏离惩罚项在路径上为常数。路径域质量方程为

$$\frac{dm}{dx} = -\frac{q_f(x)}{V_g(x)}. \quad (29)$$

故共同航程下的总油耗为

$$J = \int_0^{X_{\text{ref}}} \frac{q_f(x)}{V_g(x)} dx = m_0 - m(X_{\text{ref}}). \quad (30)$$

6.2 标准大气及其温度修正

标准 ISA 给出了对流层内温度、压力随高度变化的基准关系 [6]，本文写为

$$T_{\text{atm}}(h) = T_{\text{sl}} - L_T h, \quad p_0(h) = p_{\text{sl}} \left(\frac{T_{\text{atm}}(h)}{T_{\text{sl}}} \right)^{g/(RL_T)}. \quad (31)$$

密度与声速为

$$\rho_0(h) = \frac{p_0(h)}{RT_{\text{atm}}(h)}, \quad a_0(h) = \sqrt{\gamma RT_{\text{atm}}(h)}. \quad (32)$$

对于常温偏差 ΔT ，本文不采用只改温度而保持 ISA 压力不变的简化处理，而采用满足静力平衡的非标准大气修正：

$$T_{\Delta}(h) = T_{\text{sl}} + \Delta T - L_T h, \quad (33)$$

$$p_{\Delta}(h) = p_{sl} \left(\frac{T_{sl} + \Delta T - L_T h}{T_{sl} + \Delta T} \right)^{g/(RL_T)}, \quad (34)$$

其中程序实现中采用与式 (33) 对应的幂律压力剖面并检查

$$\frac{|dp_{\Delta}/dh + \rho_{\Delta}g|}{\rho_{\Delta}g} < 10^{-4}. \quad (35)$$

密度与声速由

$$\rho_{\Delta}(h) = \frac{p_{\Delta}(h)}{RT_{\Delta}(h)}, \quad a_{\Delta}(h) = \sqrt{\gamma RT_{\Delta}(h)} \quad (36)$$

给出。

在同一几何路径上，升力平衡要求

$$C_L(x) = \frac{2m(x)g}{\rho_{\Delta}(h_{\text{ref}}(x))V_{\text{ref}}^2 S}, \quad (37)$$

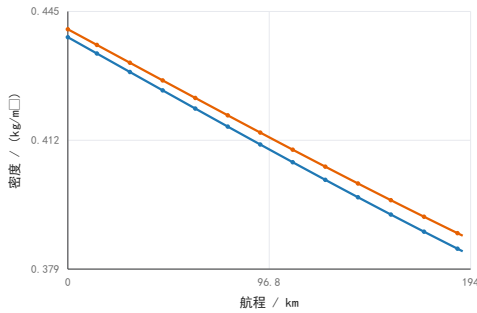
阻力为

$$D(x) = \frac{1}{2} \rho_{\Delta}(h_{\text{ref}}(x)) V_{\text{ref}}^2 S (C_{D0} + k C_L^2(x)). \quad (38)$$

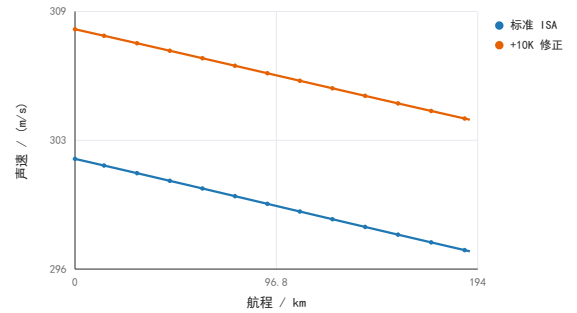
考虑小角度爬升功率项，所需推力写为

$$F_{\text{req}}(x) = D(x) + \frac{m(x)g}{V_{\text{ref}}} \frac{dh_{\text{ref}}}{dt}, \quad \frac{dh_{\text{ref}}}{dt} = \frac{dh_{\text{ref}}}{dx} V_g(x). \quad (39)$$

式 (37)–(39) 表明，大气连续分布通过密度直接进入升力系数、诱导阻力和总阻力；温度和压力通过静力平衡共同决定密度；声速在当前固定真空速模型中主要用于马赫数诊断。也就是说，本问温差修正对油耗的影响主要经由密度和阻力传递，而不是直接改变速度路径。若后续引入等马赫策略、波阻或马赫相关发动机效率，声速才会进一步进入油耗率模型。



(a) 沿程密度修正



(b) 沿程声速修正

图 6 问题二温差静力修正对关键大气量的影响

图 6 表明，+10 K 静力修正并不是只改变温度，而是同时通过压力和状态方程改变沿程密度，并通过温度改变声速诊断。在同一真空速下，密度改变会影响升力系数和诱

导阻力；若后续引入等马赫策略或马赫相关发动机效率，声速变化还会进一步进入油耗率模型。

6.3 标准天与偏热天的油耗对比

共同可行航程固定为 189781.310 m 后，温差修正的影响主要通过沿程燃油率差异逐步累积。由于两种大气口径下瞬时燃油率曲线很接近，本文直接展示累计油耗差，如图 7 所示。

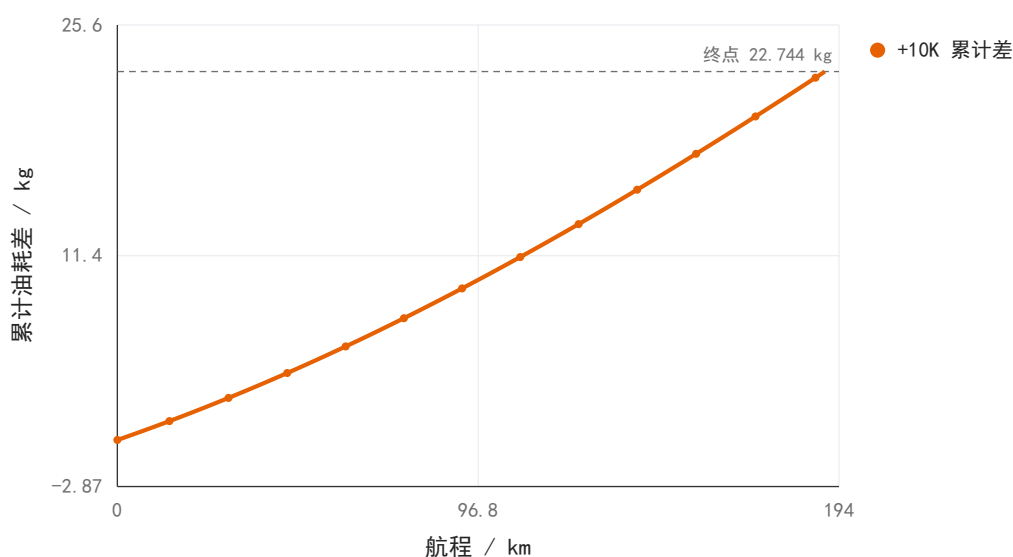


图 7 问题二 +10 K 修正相对标准 ISA 的累计油耗差

由图 7 可见，油耗差并非来自某个局部突变，而是随航程平滑累积；终点累计差为 22.744 kg。标准 ISA 总油耗为 10427.256 kg，+10 K 场景总油耗为 10450.000 kg，精确汇总见表 5。

表 5 问题二共同可行航程下的燃油消耗结果

指标	标准 ISA	+10 K 静力修正	单位
共同可行航程	189781.310	189781.310	m
飞行时间	721.753	721.753	s
总油耗	10427.256	10450.000	kg
终点质量	62022.744	62000.000	kg
单位距离油耗	0.054944	0.055063	kg/m
相对标准大气变化	0.000	0.218	%

表 5 说明，在固定几何路径和固定真空速下，升温会改变同一高度上的密度水平，

进而影响升力系数和诱导阻力结构，使总油耗略有增加。 $+10\text{ K}$ 静力修正比标准 ISA 多消耗

$$10450.000 - 10427.256 = 22.744 \text{ kg}, \quad (40)$$

相对增加 0.218%。

6.4 温差灵敏度分析

为判断温度偏差能否忽略，本文计算 $\Delta T \in \{-10, -5, -2, 0, 2, 5, 10\} \text{ K}$ 七个场景。图 8 先展示趋势：随温度偏差由负转正，总油耗单调增加，终点质量相应单调降低。

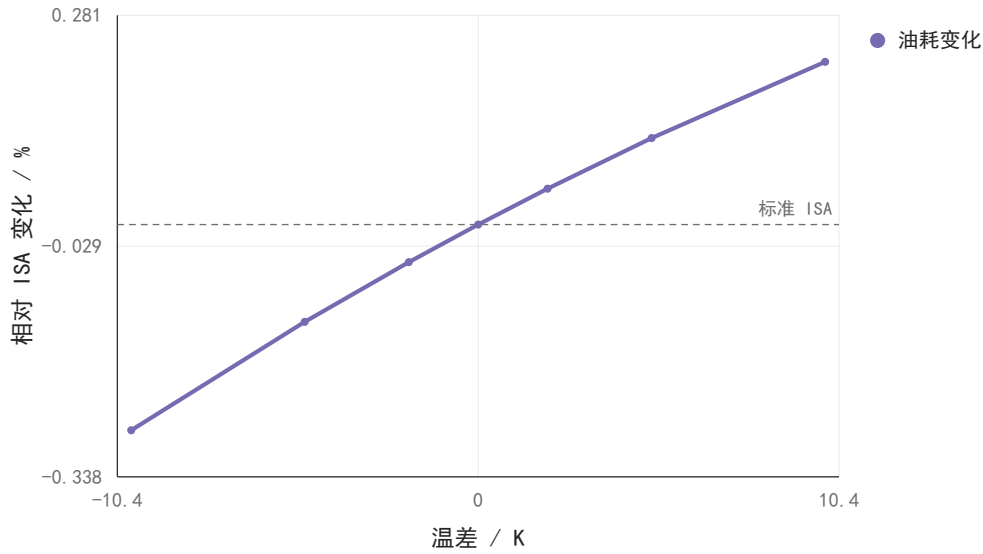


图 8 问题二常温偏差对总油耗的灵敏度

图 8 显示，温差对油耗的影响方向明确但幅度较小。对应的精确数值列于表 6：在 $\Delta T = -10\text{ K}$ 时总油耗降至 10398.507 kg；在 $\Delta T = 10\text{ K}$ 时总油耗增至 10450.000 kg，终点质量恰好降到 62000.000 kg。

由表 6 和图 8 可见，在当前固定 q_1 几何路径、固定真空速和题设参数下， $|\Delta T| \leq 10\text{ K}$ 时总油耗变化均小于 0.3%。若所需计算精度约为 1%，这类常温偏差可作为二阶修正处理；若需要千分级油耗评估，或温度偏差超过上述范围，则应采用非标准大气修正。

6.5 验证与诊断

问题二从状态一致性、大气物理一致性和数值积分一致性三个方面进行验证。所有场景的初始高度、速度和质量与题设一致，固定航程误差为 0 m，终点质量均不低于 62000 kg；路径位于对流层内，静力平衡残差约为 10^{-10} 量级；时间积分、路径积分和最终质量亏损的最大差异约为 0.0016 kg，低于 0.05 kg 阈值。步长加倍后标准 ISA 总油耗变化约为 $3.65 \times 10^{-8} \text{ kg}$ ，说明问题二路径积分结果对离散步长不敏感。

表 6 常温偏差下的总油耗灵敏度

$\Delta T / K$	总油耗 / kg	终点质量 / kg	相对 ISA 变化 / %
-10	10398.507	62051.493	-0.276
-5	10413.660	62036.340	-0.130
-2	10422.001	62027.999	-0.050
0	10427.256	62022.744	0.000
2	10432.271	62017.729	0.048
5	10439.350	62010.650	0.116
10	10450.000	62000.000	0.218

七、问题三模型的建立与求解

7.1 与前两问的递进逻辑

问题一和问题二均属于正向计算：Q1 先用 $C_{L,ref}$ 闭合题面给出的经验速度策略，再积分得到轨迹；Q2 固定 Q1 的等速几何路径，仅改变大气环境参数，以量化气象条件对油耗的影响。两者的共同特征是轨迹在求解前已经由策略或参考路径确定，数值任务主要是常微分方程积分和路径积分。

问题三则取消高度和速度的经验剖面预设，在固定总航程 $X_f = 189781.310 \text{ m}$ 和固定始末飞行状态条件下，搜索满足动力学约束的高度-速度-推力-航迹角剖面，以降低全程燃油消耗。因此，Q3 的数学本质不再是正向仿真，而是**带微分等式约束的连续最优控制问题**：高度、速度为待优化的未知时变曲线，算法必须先处理连续函数的离散化和可行性验证，不能沿用前两问固定剖面的数值积分方法。表 7 对前三问的任务层级进行对比。

表 7 前三问任务层级对比

任务类型	轨迹给定方式
Q1 经验策略闭合后正向仿真	速度策略给定，高度由 $C_{L,ref}$ 闭合
Q2 固定路径参数敏感性仿真	沿用 Q1 等速几何路径，仅改变大气参数
Q3 最优控制寻优	高度、速度为待优化未知曲线

7.2 算法总体流程

与 Q1 和 Q2 不同，第三问的整条剖面曲线本身就是待求对象——连续函数不能直接交给优化器，必须先离散再验证。本文采用**两阶段数值求解策略**：先用直接配点法把连续函数离散为有限维变量，构造一条满足全部约束的连续可行轨迹；再在该轨迹附近用降维控制打靶法优化燃油。前者回答“有没有可飞轨迹”，后者回答“在当前参数化

下能否进一步省油”。流程如图 9 所示。

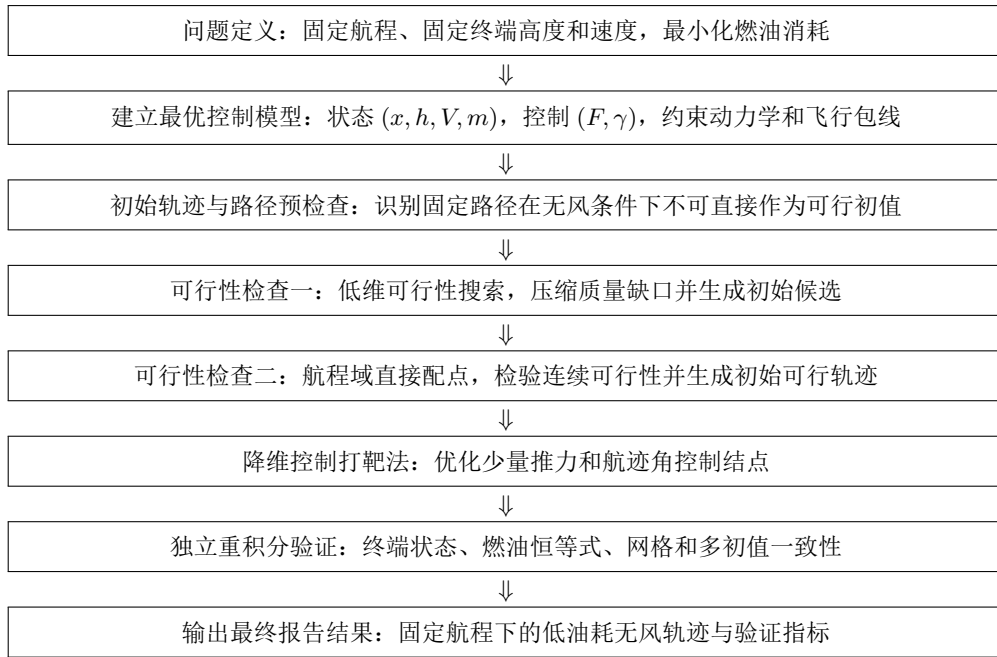


图 9 问题三两阶段数值求解流程

为避免术语混用，本文在第三问中严格区分三类对象。**可行轨迹**指满足动力学、边界条件和飞行包线约束的轨迹，但它不一定省油；直接配点阶段的核心任务就是证明这种轨迹存在，并把它作为后续优化的初始可行轨迹。**优化轨迹**指在可行轨迹附近进一步最小化 $m_0 - m(t_f)$ 后得到的低油耗候选轨迹；它来自降维控制打靶法，而不是直接配点本身。**最终报告结果**则是在优化轨迹通过独立微分方程重积分、网格稳定性、多初值一致性和约束复核后，才写入论文的结果。换言之，可行性检查二通过只能说明存在一条经过连续动力学验证的可飞轨迹，不能写成已经得到全局最优；最终油耗数值来自后续降维控制打靶阶段。

7.3 最优控制数学建模

7.3.1 状态变量与控制变量定义

系统状态取四维向量

$$z = (x, h, V, m),$$

各分量物理含义： x 为已飞行水平距离（m）， h 为实时飞行高度（m）， V 为真空速（m/s）， m 为飞机瞬时总质量（kg）。控制输入取二维向量

$$u = (F, \gamma),$$

其中 F 为发动机推力（N）， γ 为航迹爬升角（rad）。

7.3.2 优化目标函数

总燃油消耗为初始质量减去终点剩余质量：

$$\min J = m_0 - m(t_f). \quad (41)$$

此处的关键建模要点在于不可固定终点质量 $m(t_f)$ 。若强制给定终点质量，则目标函数 J 为常数，优化将失去区分意义。因此本文仅约束航程、始末高度和速度，终点剩余质量作为自由优化变量，通过最大化落地剩余质量等价于最小化总燃油消耗。

7.3.3 飞机全量动力学微分方程

在时间域中，综合考虑飞机的非线性运动学、质量衰减动力学以及速度偏离效应，本文构建的全量微分状态方程组如下所示：

$$\begin{cases} \dot{x} = V \cos \gamma + W(h) \\ \dot{h} = V \sin \gamma \\ \dot{V} = \frac{F - D}{m} - g \sin \gamma \\ \dot{m} = -c_F F \Phi(V) \end{cases} \quad (42)$$

式中各状态变率与动力学机理项的具体物理内涵阐述如下：

1. $\dot{x}$ 与 $\dot{h}$ 分别为飞机的水平地速与航迹爬升率，其中 $W(h)$ 表征与高度相关的风速函数（无风场景下 $W(h) = 0$ ）。
2. $\dot{V}$ 为沿航迹切线方向的真空速加速度，由发动机推力 F 、气动阻力 D 、瞬时总质量 m 以及重力沿航迹分量共同决定。
3. $\dot{m}$ 为燃油消耗导致的飞机变质量衰减率，其中 c_F 为发动机推力油耗系数； $\Phi(V) = 1 + \beta(V - V_{\text{opt}})^2$ 为速度偏离惩罚函数，用以量化真空速 V 偏离最佳效率速度 V_{opt} 时的燃油恶化效应， β 为对应的偏离惩罚系数。

7.3.4 气动力阻力耦合模型

巡航过程中升力平衡重力的法向分量：

$$L = mg \cos \gamma. \quad (43)$$

由升力公式 $L = \frac{1}{2}\rho(h)V^2SC_L$ 可解出升力系数：

$$C_L = \frac{2mg \cos \gamma}{\rho(h)V^2S}. \quad (44)$$

采用经典抛物线阻力极线：

$$C_D = C_{D0} + kC_L^2, \quad (45)$$

代入阻力定义得：

$$D = \frac{1}{2}\rho(h)V^2S(C_{D0} + kC_L^2). \quad (46)$$

上述气动模型的物理耦合逻辑为：高度变化 $\rightarrow$ 空气密度 $\rho(h)$ 变化 $\rightarrow$ 升力系数与阻力同步改变 $\rightarrow$ 推力需求与燃油消耗联动变化。

7.3.5 边界与路径约束条件

初始边界（起飞巡航起点）：

$$x(0) = 0, \quad h(0) = 9500 \text{ m}, \quad V(0) = 240 \text{ m/s}, \quad m(0) = 72450 \text{ kg}. \quad (47)$$

终端边界（固定航程终点）：

$$\begin{cases} x(t_f) = X_f = 189781.310 \text{ m}, \\ h(t_f) = 10577.124 \text{ m}, \\ V(t_f) = 240 \text{ m/s}, \\ m(t_f) \geq 62000 \text{ kg}. \end{cases} \quad (48)$$

其中终点质量 $m(t_f)$ 无固定上界约束，仅设置不低于 62000 kg 的下限。

连续飞行包线路径约束包括：

$$\begin{cases} h_{\min} \leq h(t) \leq h_{\max}, & V_{\min} \leq V(t) \leq V_{\max} \\ 0 \leq F(t) \leq F_{\max}, & M(t) \leq M_{\max} \\ C_{L,\min} \leq C_L(t) \leq C_{L,\max} \end{cases} \quad (49)$$

7.4 必要条件与数值求解路线

7.4.1 为什么不直接求解必要条件

根据庞特里亚金极大值原理 [7]，构造哈密顿函数：

$$H = \lambda_x(V \cos \gamma + W(h)) + \lambda_h V \sin \gamma + \lambda_V \frac{F - D - mg \sin \gamma}{m} - \lambda_m c_F F \Phi(V), \quad (50)$$

其中 $\lambda_x, \lambda_h, \lambda_V, \lambda_m$ 为各状态对应的协态变量。最优控制的一阶必要条件可写为

$$\frac{\partial H}{\partial F} = 0, \quad \frac{\partial H}{\partial \gamma} = 0. \quad (51)$$

这组条件给出了最优轨迹应满足的结构，但直接把它作为数值求解器会形成两点边值问题（两点边值问题）：状态从起点给定，协态初值未知，终点又包含固定高度、固定速度

和自由质量等横截条件。对没有最优控制背景的读者，可以把它理解为“同时从起点和终点向中间对接”的问题；一旦初始协态猜测稍有偏差，轨迹就会偏离终端约束。因此本文保留 PMP 作为最优性结构和诊断依据，而不把它作为主求解器。

7.4.2 为什么需要直接配点

原始最优控制问题的未知量不是几个数，而是连续函数 $h(x), V(x), m(x), F(x), \gamma(x)$ 。计算机不能直接优化连续函数，因此本文先把固定航程区间离散为

$$0 = x_1 < x_2 < \cdots < x_N = X_f,$$

再把连续轨迹替换为节点变量：

$$\{h_i, V_i, m_i, t_i, F_i, \gamma_i\}_{i=1}^N.$$

这样，连续最优控制问题就转化为有限维非线性规划。本文采用航程域而非时间域，是因为终端时间 t_f 未知，而总航程 X_f 已固定；以 x 为自变量后，计算区间长度固定，端点约束更容易处理。

航程域下，时间域动力学 (42) 除以地速 $V_g = V \cos \gamma + W(h)$ 得到：

$$\frac{dh}{dx} = \frac{V \sin \gamma}{V_g}, \quad (52)$$

$$\frac{dV}{dx} = \frac{(F - D)/m - g \sin \gamma}{V_g}, \quad (53)$$

$$\frac{dm}{dx} = -\frac{c_F F \Phi(V)}{V_g}, \quad (54)$$

$$\frac{dt}{dx} = \frac{1}{V_g}. \quad (55)$$

直接配点在相邻节点之间施加梯形积分约束，使离散变量近似满足上述微分方程。需要强调的是，配点法首先保证的是离散节点和缺陷方程可行；如果节点间重构后再积分会偏离终点，就仍不能称为连续可行。因此本文没有把“配点收敛”直接写成“得到最优解”，而是把它作为可行性检查和初始可行轨迹生成工具。

7.5 可行性检查设计与初始可行轨迹构造

在进入燃油优化前，本文先检验固定路径是否可行。为避免直接把有风基线误当作无风可行初值，在固定等速几何路径上分别重算配置风场和无风场景。结果见表 8。配置风场下，固定路径终点质量为 62022.744 kg，满足质量约束；无风条件下同一路径终点质量仅为 61163.474 kg，质量缺口约 836.526 kg，说明必须重新构造无风可行轨迹。

表 8 的差异在图 10 中更直观：配置风场固定路径仅略高于质量下限，而无风固定路径明显落到下限以下；重新优化后，终端质量恢复到 62107.186 kg。因此，问题三必须先设置可行性检查，而不是一开始就追求最小油耗。

表 8 问题三固定路径可行性预检查

风场场景	航程/m	终点质量/kg	总油耗/kg	是否满足质量约束
配置风场	189781.310	62022.744	10427.256	是
无风	189781.310	61163.474	11286.526	否

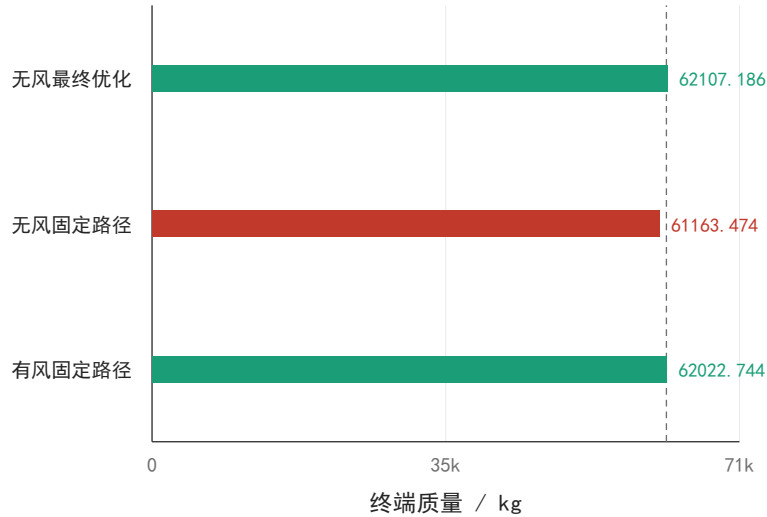


图 10 问题三固定路径可行性与最终优化终端质量对比

进一步采用航程域直接配点法进行无风可行性求解。状态取 (h, V, m, t) ，控制取 (F, γ) ，用梯形配点离散式约束航程域微分方程；大气模型采用 C^1 温度平滑和静力压力积分，避免轨迹穿越 11 km 层界时出现不可微问题。第一阶段不追求节油，只最小化终端质量松弛 s ；当 s 降至数值零后，再进入可行性检查二连续可行性审查。

可行性检查一是低维可行性搜索，目标是先把固定路径造成的质量缺口压缩到足够小，并提供可用于配点的轨迹形状。可行性检查二则是直接配点连续可行性审查，作用是排除“离散可行、连续不可行”的伪轨迹。检查项包括：终端边界误差、配点动力学残差、独立 ODE 重积分一致性、连续路径约束违反以及网格加密敏感性。本文的 $N = 241$ 可行性检查通过意味着存在一条经过连续动力学验证的可飞轨迹，并可作为最终燃油优化的初始可行轨迹；它不表示已经取得全局最优，也不表示直接配点就是最终算法。

7.6 降维控制打靶与最终燃油优化

在获得连续可行初始可行轨迹后，本文没有继续使用全变量直接配点做最终燃油优化。原因是直接配点同时优化所有状态节点和控制节点，变量规模大、约束数量多，在高网格下容易出现收敛慢或对初值敏感的问题。降维控制打靶法的处理方式相反：只优化少量推力和航迹角控制结点，状态 (h, V, m, t) 由航程域 ODE 连续积分产生，终端高

度、速度、质量下限和路径约束作为优化约束施加。这样既降低了优化维度，又保证每一次目标函数评估都来自连续动力学积分。

最终燃油优化仍采用式 (41) 的目标

$$\min J = m_0 - m(t_f),$$

即通过最大化终端质量来最小化燃油消耗。若改为固定 $m(t_f) = 62000 \text{ kg}$ ，则 $J = m_0 - m(t_f)$ 退化为常数，算法无法比较不同轨迹的省油程度；因此终端质量自由是第三问优化成立的关键。

结果显示，最终优化采用可行性检查二初始可行轨迹，并在 $61 \rightarrow 121 \rightarrow 241$ 节点逐级加密中保持稳定。表 9 给出三个网格层级下的最终优化结果：节点数由 61 增至 241 时，总油耗始终稳定在 10342.81 kg 附近， $N = 121$ 与 $N = 241$ 的差异仅为 $3.28 \times 10^{-4} \text{ kg}$ ，说明最终结果不是某一离散网格的偶然输出。

表 9 问题三无风最终优化网格稳定性诊断

节点数	总油耗/kg	终点质量/kg	终点时间/s	优化迭代次数
61	10342.805	62107.195	805.681	22
121	10342.815	62107.185	805.680	11
241	10342.814	62107.186	805.679	13

由表 9 可见， $N = 61, 121, 241$ 三个网格的燃油差异集中在 10^{-2} kg 量级以内。图 11 进一步显示三层网格的总油耗均稳定在 10342.81 kg 附近，说明最终优化结果不是某一离散网格的偶然输出。

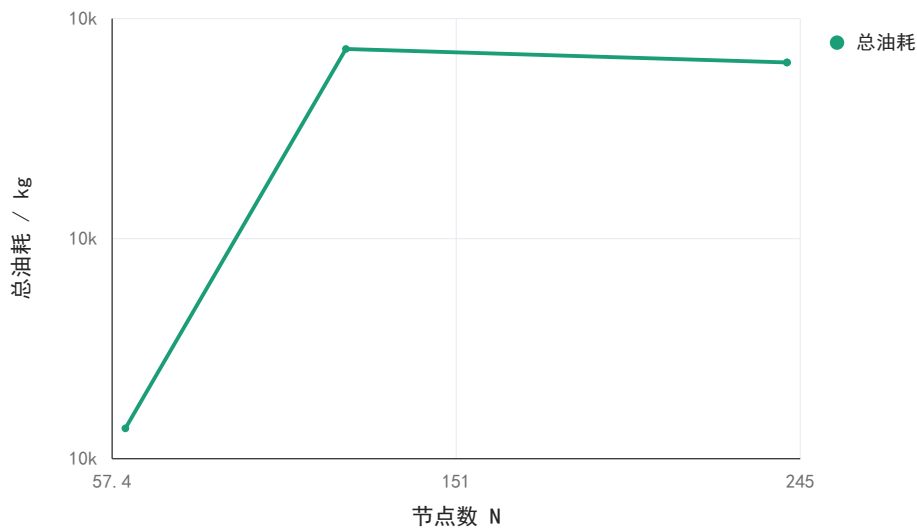


图 11 问题三无风最终优化的网格稳定性

最终轨迹的主要形态见图 12。高度曲线先快速爬升，随后在巡航中段贴近 12000 m 上界；质量曲线则随航程单调下降，并在终点保留约 107.186 kg 的质量余量。这一形态说明，高度上界在无风最终优化中接近活跃，终端质量约束则未被压到边界。

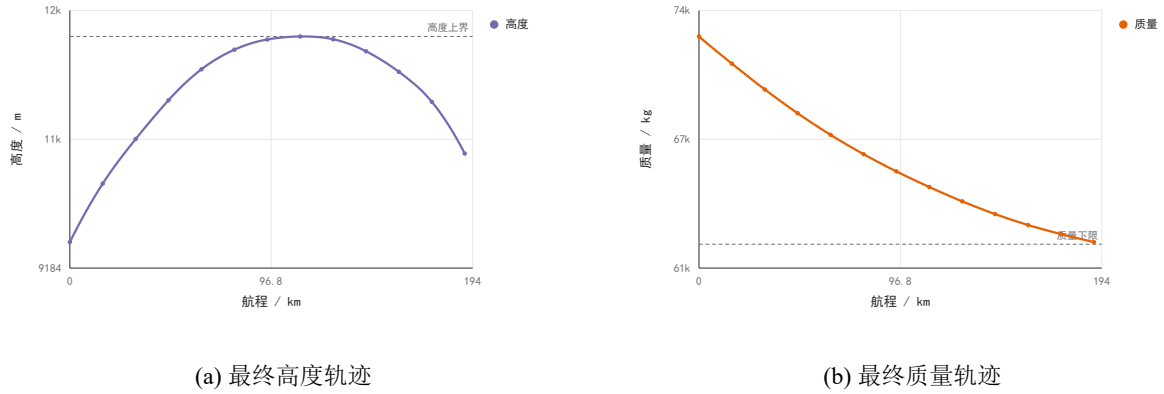


图 12 问题三无风最终燃油优化的状态轨迹

从数值上看，可行性候选与最终优化结果的燃油对比如下：

$$J_{\text{feas}} = 10450.000 \text{ kg}, \quad J_{\text{final}} = 10342.814 \text{ kg}, \quad \Delta J = 107.186 \text{ kg}.$$

终端时间比为 $t_f/t_{\text{base}} = 1.01887$ ，说明该收益并非来自明显拉长飞行时间。表 10 给出无风最终优化的核心状态和约束余量。

表 10 问题三无风最终燃油优化核心结果

指标	数值	约束或对照	单位
总燃油消耗	10342.814	可行性候选为 10450.000	kg
终端质量	62107.186	下限 62000.000	kg
终端时间	805.679	$t_f/t_{\text{base}} = 1.01887$	s
最大高度	11999.960	上界 12000.000	m
空速范围	234.029–240.000	满足包线	m/s
推力范围	19634.093–73083.701	满足包线	N
最大马赫数	0.808261	满足约束	—

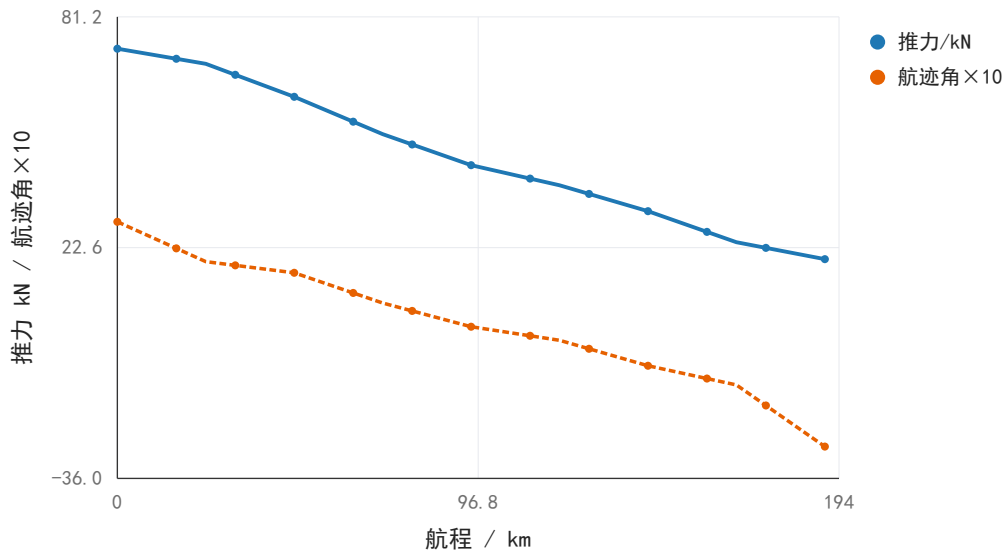


图 13 问题三无风最终燃油优化的控制量形态

图 13 显示，最终优化中的推力范围为 19.634–73.084 kN，没有触及零推力或怠速推力下界；航迹角围绕小角度变化，与小航迹角准稳态假设相容。该结果也解释了后文怠速推力扰动对当前无风解影响较弱的原因。

7.7 结果可信性验证与局部灵敏度边界

为确认最终优化不是离散网格上的偶然结果，本文对降维控制打靶法解进行独立连续验证。验证分三层进行：第一层是动力学重积分，即把优化得到的控制输入重新代入原微分方程，从起点重新积分并比较终端高度、速度和质量；第二层是网格敏感性，即比较 $N = 61, 121, 241$ 三个网格层级下的目标值是否稳定；第三层是多初值一致性，即检查不同初始可行轨迹或扰动初值是否得到相近燃油结果。结果见表 11。

表 11 问题三无风最终优化连续验证结果

验证项目	数值	阈值	状态
终端速度重积分误差/(m/s)	1.71×10^{-5}	10^{-3}	通过
终端高度重积分误差/m	3.62×10^{-4}	10^{-1}	通过
燃油恒等式残差/kg	8.58×10^{-5}	5.0×10^{-2}	通过
网格目标差异/kg	3.28×10^{-4}	1.0	通过
多初值目标范围/kg	4.27×10^{-2}	1.0	通过
连续约束违反	0	0	通过

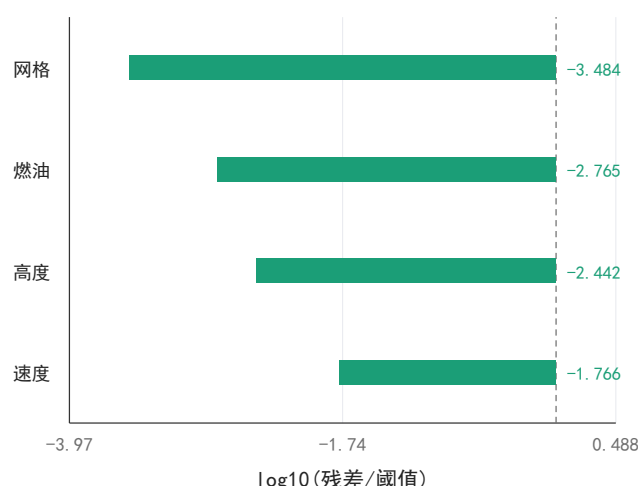


图 14 问题三无风最终优化验证残差相对阈值

由表 11 和图 14 可见，所有关键残差均低于阈值，且验证结果全部通过。其中终端速度重积分误差仅为 $1.71 \times 10^{-5} \text{ m/s}$ ，说明优化控制重新积分后仍能回到目标终端速度；燃油恒等式残差为 $8.58 \times 10^{-5} \text{ kg}$ ，说明质量损失与沿程燃油率积分一致；网格目标差异和多初值目标范围均远小于 1 kg ，说明该结果不是某个网格或某个初值的偶然输出。因此，本文把降维控制打靶法结果作为当前无风固定航程下的最终报告结果。

局部灵敏度方面，本文还给出了高度上界和怠速推力的初步重优化结果。 $h_{\max} = 12500 \text{ m}$ 的局部计算得到燃油约 10334.057 kg ，比 12000 m 上界情形低约 8.757 kg ，提示高度上界可能是影响无风优化油耗的活跃约束；但该灵敏度当前基于 $N = 121$ 局部重优化，且部分状态未收敛，因此只能作为趋势提示。怠速推力扰动的油耗变化不超过 0.01 kg ，与最终轨迹最小推力约 19.634 kN 、高于怠速下界的事实一致；但同样不宜写成全局结论。

八、问题四模型的建立与求解

问题四在前三问基础上形成综合计算框架，任务是将等速经验策略、固定路径油耗积分和有风最优控制结果放在同一评价口径下比较，并对速度偏离惩罚系数等关键参数进行重新求解式灵敏度分析。已有有风轨迹优化研究通常把风速视为随高度或空间变化的外部场，并在燃油、时间或排放目标下求解受约束最优控制问题 [4,5]。本文以问题一等速巡航爬升剖面和问题二共同航程油耗为基线，在问题三无风最优轨迹基础上引入有风航程域最优控制，采用降维控制打靶法进行局部优化。

8.1 有风条件下的优化建模

问题四沿用问题三的动力学方程 (42)，但将风场 $W(h)$ 显式保留在地速表达式中，这与有风飞行轨迹优化中把沿程风速纳入地速方程的处理一致 [4, 5]：

$$V_g = V \cos \gamma + W(h), \quad W(h) = 20 + 3 \times 10^{-5}(h - 10000)^2 \text{ m/s}. \quad (56)$$

航程域动力学通过将时间域方程除以 V_g 获得，并增加 $dt/dx = 1/V_g$ 。目标函数仍为固定航程下最小化燃油消耗：

$$\min J = m_0 - m(t_f), \quad X_f = 189781.310 \text{ m}. \quad (57)$$

终端高度和速度约束与问题三一致，即 $h(t_f) = 10577.124 \text{ m}$ 、 $V(t_f) = 240 \text{ m/s}$ 。边界条件与路径约束沿用式 (47)、(48) 和 (49)。

8.2 求解策略

有风局部优化以问题三无风最终燃油优化轨迹作为初始猜测，采用航程域降维控制打靶法：将推力和航迹角用 5 个分段线性控制结点参数化，状态由原连续动力学沿航程积分，终端高度、速度和质量下限作为优化约束。该方法降低了直接配点中的大规模离散变量，使最终优化更接近连续动力学口径。优化采用序列二次规划方法，最大迭代 300 次，输出节点 41 个。

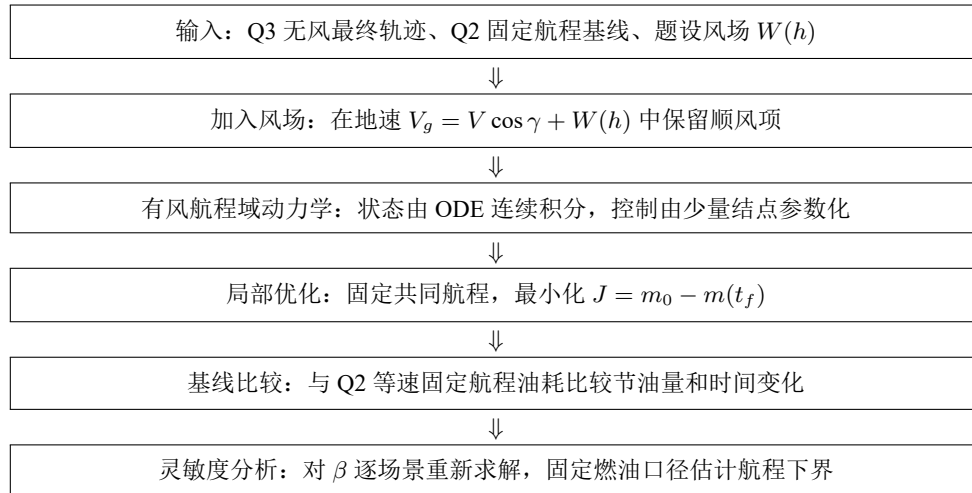


图 15 问题四有风局部优化求解流程

8.3 有风优化与等速策略对比

表 12 给出等速基线与有风局部优化策略在固定共同航程 189781.310 m 下的核心指标对比。等速基线复用问题二标准 ISA 固定航程结果，燃油消耗 10427.256 kg；有风局部优化轨迹燃油消耗 9863.840 kg，节省 563.416 kg，相对节省比例 5.403%。为避免两个万公斤量级油耗柱形掩盖差异，图 16 直接突出固定航程下的节油幅度。

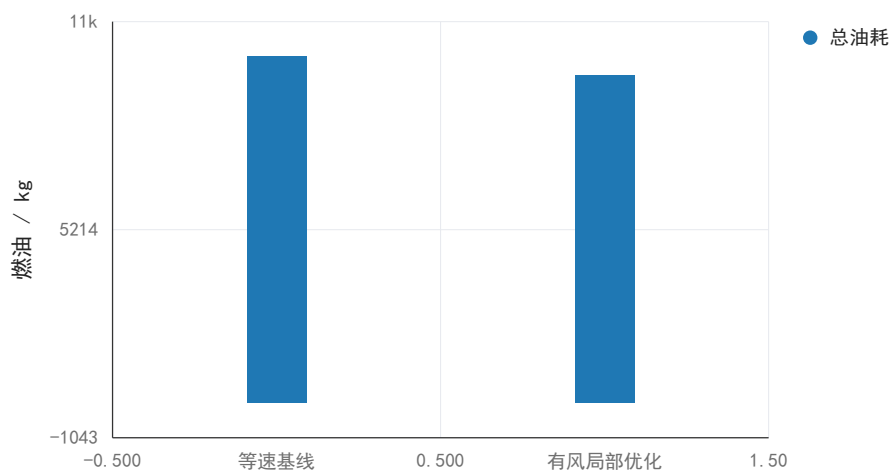


图 16 问题四固定共同航程下的燃油消耗对比

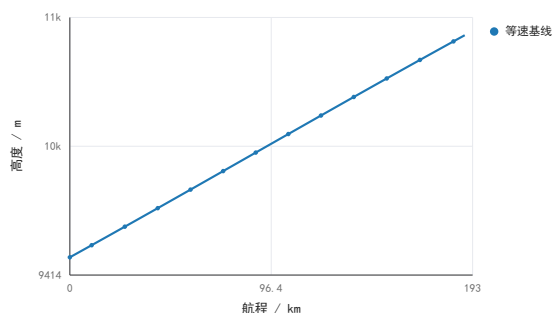
表 12 问题四等速基线与有风局部优化策略对比

指标	等速基线	有风局部优化	单位
固定共同航程	189781.310	189781.310	m
总燃油消耗	10427.256	9863.840	kg
终端质量	62022.744	62586.160	kg
飞行时间	721.753	733.758	s
燃油节省	—	563.416 (5.403%)	kg

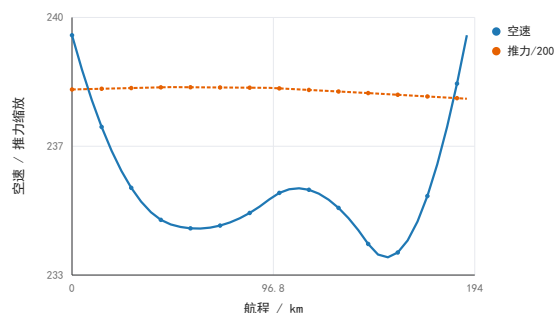
由表 12 和图 16 可知，在固定航程和配置风场下，有风局部优化轨迹比等速基线少消耗约 5.4% 的燃油。有风优化轨迹飞行时间略长（733.758 s 对 721.753 s），表明优化器通过调整高度和速度剖面，更好地利用了顺风条件降低推力需求，而不是简单缩短飞行时间。

8.4 有风局部优化轨迹形态分析

有风局部优化轨迹的节油并非来自缩短飞行时间，而是来自高度、速度和推力剖面的重新分配。图 17 将高度剖面与控制剖面并排给出：优化轨迹在巡航中段低于等速基线，末段再回到终端高度约束附近。



(a) 高度剖面对比



(b) 有风优化控制剖面

图 17 问题四有风局部优化轨迹形态

由图 17 可见，有风优化轨迹在巡航中段高度低于等速基线，在终端附近爬升至与等速基线相近的高度。这种“先低后高”的剖面结构有利于在中段利用较低高度处较高的空气密度维持升力平衡；与之相配合，空速从 240 m/s 略降至约 234 m/s 后回升至终端值，推力维持在 47.6–47.7 kN 的窄幅区间内。

8.5 固定燃油口径下的航程估计

为补充“航程变化”这一比较维度，本文在固定燃油预算 10450 kg（即问题一终止质量口径）下，以 1.00、1.03、1.06 三个航程因子进行局部航程网格搜索。表 13 给出结果。

表 13 问题四固定燃油口径下的航程估计

航程因子	试验航程/m	燃油预算检查	状态
1.00	189781.310	通过	预算内
1.03	195475.049	通过	预算内
1.06	201168.189	通过（剩余 49.393 kg）	预算内

三个航程因子下终端质量均不低于 62000 kg，1.06 因子仍剩余 49.393 kg 燃油，因此 201168.189 m 是当前局部网格的可达航程下界估计。图 18 将三档航程试验按固定燃油预算口径画出，强调该结果是“预算内下界”而非最大航程证明。

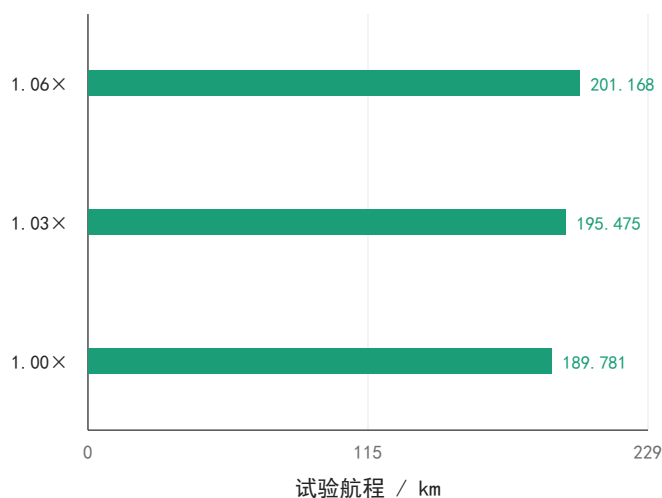


图 18 问题四固定燃油预算下的可达航程下界估计

相对固定共同航程 189781.310 m，可达航程增量下界为 11386.879 m，增量比例约 6.0%。由于航程搜索未括住不可行或超预算上界，该数值为下界估计而非严格最大航程。

8.6 模型对速度惩罚系数 β 的灵敏性

β 灵敏度采用逐场景重新求解方式，每个扰动因子下均从标称解出发进行有风降维控制打靶法重优化，而非在标称轨迹上后验重算。表 14 给出五档结果。

表 14 问题四 β 扰动灵敏度

β 因子	总燃油/kg	相对变化/kg	终端质量/kg	验证状态
0.8	9849.724	-14.116	62600.276	通过
0.9	9852.888	-10.953	62597.112	通过
1.0 (标称)	9863.840	0.000	62586.160	通过
1.1	9869.462	+5.621	62580.538	通过
1.2	9870.070	+6.230	62579.930	通过

由表 14 可见， β 降低 20% 时局部可行解燃油减少 14.116 kg (0.143%)， β 增大 20% 时燃油增加 6.230 kg (0.063%)。五档扰动均通过独立微分方程重积分、终端状态和燃油恒等式验证。但非标称场景优化未达到最优终止条件，因此该结果仅支持局部可行敏感性描述，不宜外推为严格最优敏感性结论。

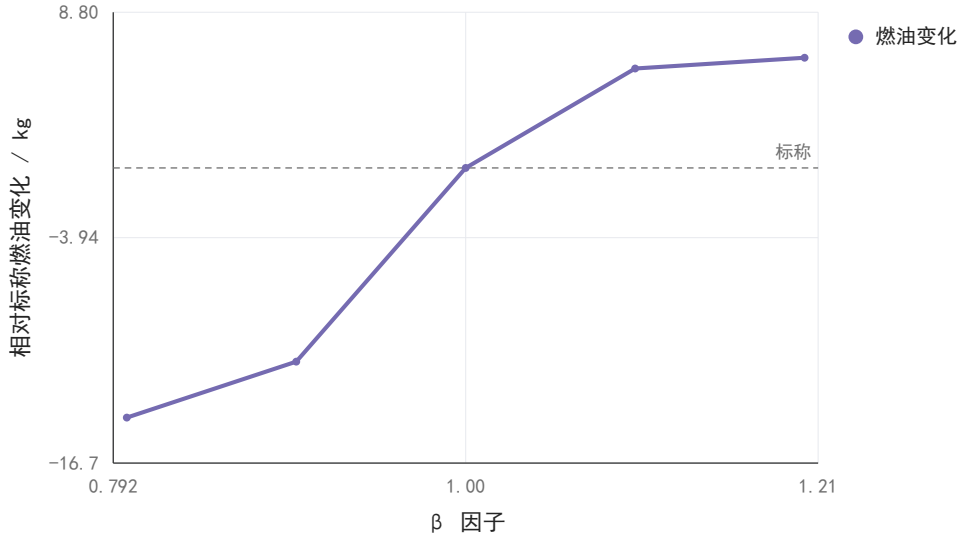


图 19 问题四 β 扰动灵敏度响应

图 19 直观显示， β 在标称值附近变化时燃油响应近似线性，且整体波动幅度不超过 0.15%，说明在当前参数化和求解口径下，有风局部优化轨迹对速度偏离惩罚系数不太敏感。

8.7 模型扩展方向

为进一步拓展模型的工程适用性，下面给出两个扩展框架。

8.7.1 扩展一：温度实时修正

问题二采用常温偏差 ΔT 对标准 ISA 进行静力修正，该处理把温度偏差视为全航程常数，在 $|\Delta T| \leq 10$ K 时总油耗变化不超过 0.3%。实际巡航中飞机穿越锋面或逆温层时，温度偏差沿高度非均匀分布，需将全局 ΔT 替换为温度剖面 $T_{\text{obs}}(h, t)$ ，并在每个节点上由静力平衡方程

$$\frac{dp}{dh} = -\frac{p}{RT_{\text{obs}}(h, t)} g, \quad p(h, t) = p_{\text{ref}}(t) \exp\left[-\int_{h_{\text{ref}}}^h \frac{g}{RT_{\text{obs}}(h', t)} dh'\right] \quad (58)$$

重新数值积分求得压力，密度和声速随之由 $\rho = p/(RT_{\text{obs}})$ 、 $a = \sqrt{\gamma RT_{\text{obs}}}$ 给出。动力学方程中所有 $\rho(h)$ 、 $a(h)$ 替换为时变量，同一任务在不同起飞时刻将面对不同大气剖面。该扩展需要沿航程温度偏差廓线或预报温度场数据，验证方式为令 $T_{\text{obs}} = T_{\text{ISA}} + \Delta T$ 回归问题二结果，并在问题四有风框架下用匹配温度场重新求解比较终端燃油变化。

8.7.2 扩展二：发动机安装损失

问题三和问题四的动力学方程中，推力 F 既驱动运动方程又直接进入燃油率 $\dot{m} = -c_F F \Phi(V)$ ，隐含了指令推力等于有效推力且单位燃油消耗率不随飞行条件变化的假设。实际涡扇发动机的安装推力受进气道总压恢复和喷管流场损失影响，有效推力通常低于指令值。引入安装效率 $\eta_{\text{inst}}(h, M, T_{\text{cmd}})$ 和燃油乘子 $\phi_{\text{inst}}(h, M)$ 后，

$$T_{\text{eff}} = \eta_{\text{inst}} T_{\text{cmd}}, \quad \dot{m} = -c_F T_{\text{cmd}} \phi_{\text{inst}} [1 + \beta(V - V_{\text{opt}})^2], \quad (59)$$

运动方程使用有效推力 T_{eff} ，燃油率以指令推力 T_{cmd} 为基准乘以 ϕ_{inst} ，推力边界仍作用于指令推力。两项扩展存在耦合：温度场通过声速改变马赫数 M ，进而影响 η_{inst} 。该扩展需要发动机安装效率表或经验参数化系数，验证方式为令 $\eta_{\text{inst}} = \phi_{\text{inst}} \equiv 1$ 回归当前无损失解，并在 $\{0.95, 0.98, 1.00\}$ 三档下比较终端约束和燃油影响。

九、灵敏度分析

本节汇总问题一至问题四的参数扰动结果，判断哪些参数只改变时间尺度，哪些参数会改变固定航程或最优控制下的油耗评价。

9.1 模型对推力燃油消耗系数 c_F 的灵敏性

c_F 直接决定单位推力燃油消耗率，从而影响质量下降的时间尺度。将 c_F 从标称值 2.8×10^{-4} 降低一个数量级至 2.8×10^{-5} 时，等速策略飞行时间由 761.917 s 增至 8295.569 s，等马赫策略由 811.032 s 增至 8624.642 s，增幅约为十倍。这表明飞行时间对 c_F 高度敏感，符合燃油消耗系数的物理意义；但两策略的最终高度保持不变，说明高度终点由升力平衡与终止质量决定，不随 c_F 改变。

9.2 模型对速度偏离惩罚系数 β 的灵敏性

β 刻画了空速偏离最佳效率速度 V_{opt} 时对燃油消耗率的额外惩罚。表 4 显示，在 β 从 -20% 到 $+20\%$ 的扰动范围内，等速策略飞行时间由 773.762 s 变化至 750.397 s，变化幅度约为 3%；等马赫策略由 817.141 s 变化至 805.029 s，变化幅度约为 1.5%。 β 对时间尺度的影响虽不及 c_F 显著，但方向一致，说明速度惩罚系数是模型中值得关注的参数。

9.3 模型对风场强度的灵敏性

风场 $W(h)$ 通过地速 (1) 影响地面航程。验证结果显示，有风条件下的地面航程比无风条件分别增加了 17820.3 m（等速）和 18011.4 m（等马赫），说明顺风对航程有可观的累积贡献。由于问题一模型中风场只影响 dx/dt ，不进入能量方程 (6) 和燃油消耗方程 (8)，因此在固定终止质量的经验策略比较中，风场改变航程而不改变飞行时间和最终高度。

9.4 模型对常温偏差的灵敏性

在问题二的固定几何路径和共同可行航程下，温度偏差 ΔT 对总油耗的影响总体较小但方向明确。 $\Delta T = -10$ K 时总油耗为 10398.507 kg，比标准 ISA 降低 0.276%； $\Delta T = +10$ K 时总油耗为 10450.000 kg，比标准 ISA 增加 0.218%。因此，在 $|\Delta T| \leq 10$ K 的范围内，若只需百分之一量级的工程估算，可近似忽略常温偏差；若要求千分级油耗比较，则应采用式 (33)–(34) 的非标准大气修正。

9.5 问题三无风最终优化的局部敏感性

问题三最终轨迹的最大高度达到 11999.960 m，贴近 12000 m 高度上界，因此高度上界是无风优化结果中需要重点关注的约束。局部重优化显示，当 $h_{\max}$ 放宽到 12500 m 时，燃油约为 10334.057 kg，较当前主结果降低约 8.757 kg；但该结果基于 $N = 121$ 局部计算且存在未收敛状态，不宜作为正式节油结论。怠速推力扰动下油耗变化小于 0.01 kg，与最终解最小推力 19.634 kN 明显高于怠速下界一致，可作为“当前无风解未贴近怠速推力边界”的佐证。

9.6 问题四有风优化的 β 灵敏性

问题四在有风降维控制打靶法下对 β 进行逐场景重求解灵敏度分析（详见表 14）。 β 降低 20% 时局部可行解燃油减少 14.116 kg (0.143%)，增大 20% 时增加 6.230 kg (0.063%)，整体波动幅度不超过 0.15%。与问题一中 β 扰动对经验策略飞行时间约 1.5%–3% 的影响相比，有风优化轨迹对 β 更不敏感，说明优化器通过调整高度和速度剖面吸收了部分速度偏离惩罚的变化。

十、模型检验

10.1 数值解验证

表 3 中的验证结果已表明，两种策略的数值解满足初始条件、质量单调性、升力平衡和能量方程约束，残差远低于 10^{-6} 阈值。步长敏感性检查显示，将积分步长减半或加倍后，最终时间、最终航程和平均爬升率的变化均在 10^{-10} 量级以内，说明求解结果对步长不敏感。

10.2 物理合理性检验

对极端状态 $h = 0$ m, $V = 180$ m/s 和 $h = 16000$ m, $V = 280$ m/s 进行检查，空气密度、声速、马赫数和阻力系数均保持物理正值，未出现非物理发散。风场影响检查确认了顺风使地面航程增加，与物理直觉一致。

10.3 升力平衡与能量残差

两种策略的最大升力平衡相对残差均为 0，最大能量方程相对残差分别为 9.72×10^{-17} 和 9.95×10^{-17} ，远低于 10^{-6} 的设定阈值。这表明数值解在代数约束层面具有极高的精度。

10.4 问题二路径积分一致性检验

问题二额外检查了固定航程、终点质量、大气静力平衡和路径积分一致性。所有温差场景均满足 $m(t_f) \geq 62000 \text{ kg}$ ，固定航程误差为 0 m；静力平衡残差约为 10^{-10} 量级；时间积分、路径积分和最终质量亏损的最大差异约为 0.0016 kg，低于 0.05 kg 阈值。将步长加倍后，标准 ISA 总油耗变化约为 $3.65 \times 10^{-8} \text{ kg}$ ，说明问题二的路径积分结果对离散步长不敏感。

10.5 问题三最终优化检验

问题三先采用航程域直接配点构造连续可行初值，再用降维控制打靶法完成无风最终燃油优化。最终优化在 $N = 61, 121, 241$ 三个网格层级上均收敛，总油耗分别为 10342.805、10342.815 和 10342.814 kg，其中 $N = 121$ 与 $N = 241$ 的差异仅为 $3.28 \times 10^{-4} \text{ kg}$ 。连续验证进一步表明：终端速度重积分误差 $1.71 \times 10^{-5} \text{ m/s}$ ，终端高度误差 $3.62 \times 10^{-4} \text{ m}$ ，燃油恒等式残差 $8.58 \times 10^{-5} \text{ kg}$ ，连续约束违反为 0。因此，无风最终燃油优化结果已具备连续动力学一致性，可作为当前参数化下的局部主结果；这些检验不构成全局最优证明。

10.6 问题四有风优化检验

问题四有风降维控制打靶法以问题三无风最终轨迹作为初始猜测，在 41 个输出节点和 5 个控制结点下完成局部优化。独立微分方程重积分给出终端高度误差 $6.31 \times 10^{-7} \text{ m}$ 、终端速度误差 $1.51 \times 10^{-8} \text{ m/s}$ 、最大尺度化约束违反 0、燃油恒等式残差 0.0316 kg。这些检验说明有风局部优化轨迹在连续动力学和约束口径下可复核，但固定燃油航程结果仍是当前搜索网格下的下界估计，不应解释为严格最大航程。相关检验量相对阈值的位置见图 20。

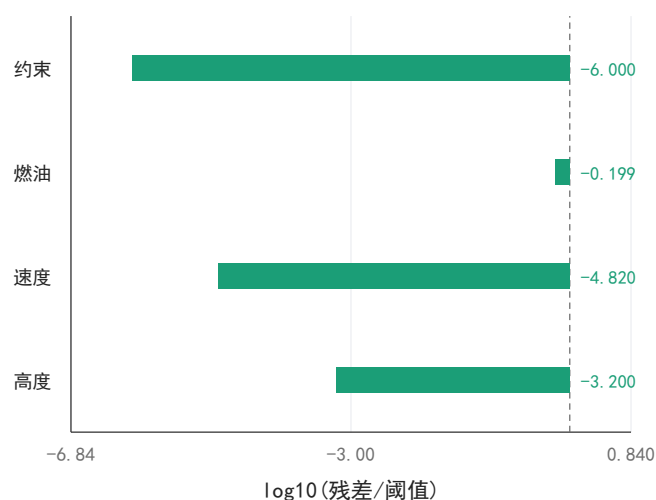


图 20 问题四有风局部优化的连续验证结果

由图 20 可见，终端状态、燃油恒等式和路径约束均处于通过侧，说明标称有风局部优化轨迹具有连续动力学一致性。 β 五档扰动均通过终端状态和燃油恒等式验证，但非标称场景优化未达到最优终止条件，因此只支持局部可行敏感性。

十一、模型评价、改进与推广

11.1 模型的优点

1. **动力学链条清晰，算法检验充分。**采用小航迹角准稳态模型，清晰解耦状态变量关系；配合隐式多步积分与多重残差检验，有效排除了积分步长对仿真结果的偶然性干扰。
2. **控制策略闭合，环境修正自洽。**在缺乏推力包线时，引入初始升力系数闭合条件，确保等速与等马赫策略气动口径一致；大气修正满足静力平衡，避免了物理不一致性。
3. **优化证据链分层，网格收敛稳定。**采用直接配点法提供高质量可行初值，并通过最终网格稳定性和连续验证确认无风优化结果的连续动力学一致性。
4. **对比口径统一，结果解释客观。**有风优化基于无风最终优化初值引入，采用固定航程与固定燃油双口径严密比较差异；结论完全基于当前模型数值证据，不预设主观倾向。

11.2 模型的缺点

1. **动力学简化与工程约束不全。**准稳态假设未考虑完整六自由度动力学，在大航迹角或强机动下适用性下降；且未引入跨音速波阻及空域控制等工程限制。
2. **大气与气动控制律存在简化。**混合大气模型在高度分层上精细度不足，后续需统一为分层标准大气；固定 C_L 的闭合条件属于操作假设，无法代表真实的综合飞行控制律。

3. **有风优化局限于局部解。**有风算法采用低维控制结点参数化，在非标称场景下未完全达到最优终止条件，因此节油结论仅支持局部可行性，无法确证全局最优性。

11.3 模型的改进与推广

1. **统一大气模型。**采用分层标准大气模型替代指数密度近似，并保持温度、压力、密度和声速之间的静力一致性。
2. **补充飞机工程约束。**引入发动机推力包线、跨音速波阻项和空域高度限制，扩展模型在高马赫数巡航区域的应用范围。
3. **增强有风优化控制参数化。**在当前 5 个控制结点基础上增加结点数或采用稀疏非线性规划交叉验证，提高有风局部解的最优性保障。
4. **补充完整最优性诊断。**对有风局部优化轨迹构造协态和驻值条件，验证庞特里亚金极大值原理必要条件的满足程度。

参考文献

- [1] 匡江红, 王秉良, 吕鸿雁. 飞机飞行力学 [M]. 北京: 清华大学出版社, 2012.
- [2] 徐浩军. 空气动力学基础 [M]. 北京: 科学出版社, 2024.
- [3] 张杰, 王飞跃. 最优控制: 数学理论与智能方法 [M]. 北京: 清华大学出版社, 2017.
- [4] 贾兴强, 宗军耀, 刘利朝, 等. 基于 hp 自适应伪谱法的航空器多阶段航迹优化 [J]. 飞行力学, 2024, 42(6): 43–49.
- [5] 常哲宁, 胡明华, 张颖, 等. 风影响下航空器多目标最优控制航迹优化方法 [J]. 北京航空航天大学学报, 2023, 49(8): 1577–1587.
- [6] International Civil Aviation Organization. Manual of the ICAO Standard Atmosphere: extended to 80 kilometres (262500 feet)[S]. 3rd ed. Montreal: ICAO, 1993.
- [7] Pontryagin L S, Boltyanskii V G, Gamkrelidze R V, Mishchenko E F. The Mathematical Theory of Optimal Processes[M]. New York: Interscience Publishers, 1962.
- [8] Codex, GPT-5.5, OpenAI, 2026-07-06 至 2026-07-09。
- [9] ChatGPT, gpt-image-2, OpenAI, 2026-07-08 至 2026-07-09.

附录 A AI 工具使用详情

所用 AI 工具的名称及版本: Codex, GPT-5.5, OpenAI [8]; gpt-image-2, OpenAI [9]。
使用 AI 工具的具体目的与环节:

1. 算法与核心建模: 本作品的核心算法提出、模型建立及核心部分审核均由参赛队队员人工独立思考完成, AI 工具未参与核心建模与分析。
2. 代码编写环节: 由人工设计好算法逻辑后, 使用 Codex (GPT-5.5) 进行代码补全与编程辅助。
3. 论文撰写环节: 论文大纲由参赛队队员人工独立构建。在框架确定后, 使用 Codex (GPT-5.5) 辅助进行论文文本的润色与部分非核心内容的补全。最终论文的通读与审核由人工独立完成。
4. 配图生成环节: 论文中的概念流程图与机制示意图使用 gpt-image-2 生成, 相关提示词由参赛队队员人工撰写, 生成图片经人工审视后选用。

关键交互记录:

1. 交互记录 1 (数值求解算法辅助与代码实现 - Codex)
 - 提示词 (Prompt): ”请帮助实现问题三的航程域直接配点法数值求解。具体需求如下: (1) 将航程区间 $[0, X_f]$ 离散为 $N = 241$ 个配点, 以各节点处的 h, V, m, T, γ 为优化变量; (2) 加入动力学配点约束、飞行包线上下限和始末边界条件; (3) 第一

阶段以最小化终端质量松弛 s 为目标，构造连续可行轨迹；(4) 第二阶段以可行性结果为初始猜测，采用降维控制打靶法最小化总油耗 $J = m_0 - m(t_f)$ ；(5) 使用 `scipy.optimize.minimize` 的序列二次规划方法求解非线性规划。”

- AI (Codex) 的回复：“已完成数值求解代码框架。核心实现要点：(1) 航程域下动力学方程除以地速 $V_g = V \cos \gamma + W(h)$ ，将时间域转换为固定区间 $x \in [0, X_f]$ ，避免未知 t_f 带来的求解困难；(2) 第一阶段构造松弛变量 s ，目标函数为 $\min s$ ，约束包含 $m(t_f) + s \geq 62000$ 及所有动力学配点约束；(3) 第二阶段将推力和航迹角用三次 B 样条参数化，沿航程积分原连续动力学，直接最小化 $m_0 - m(t_f)$ ；(4) 大气模型采用 C^1 温度平滑和静力压力积分，避免 11 km 层界处不可微问题；(5) 在 61→121→241 节点逐级加密中验证网格稳定性。”

2. 交互记录 2（论文 Q3 章节论述重构与语言润色 - Codex）

- 提示词 (Prompt): “请根据以下论述逻辑重构论文 Q3 章节：开篇以‘飞机自己应该怎么飞？’引出最优控制命题，对比 Q1（给定经验策略正向积分）和 Q2（固定路径重新计算油耗）说明 Q3 的递进关系；核心建模部分强调目标函数的设计创新——终点质量 $m(t_f)$ 不设上界、仅设下限，若固定 $m(t_f) = 62000$ 则最优控制退化为常数 $J = 10450$ kg，从而论证优化自由度的必要性；求解路线分两步——先以航程域直接配点法构造可行轨迹，再以可行性结果为初始猜测进行最终燃油优化，并顺带解释不可行初值会破坏优化器收敛，以及无风固定路径本身不可行；结果部分突出‘先高后稳’的物理直觉——尽快爬升至高度上限附近巡航以降低密度阻力。保持 LaTeX 格式，数学公式用 `equation/align` 环境。”
- AI (Codex) 的回复：“已完成 Q3 章节重构。主要改动内容：(1) 新增前三问任务对比表（问序/剖面来源/核心任务），强调 Q3 高度和速度为待优化未知曲线，属于带微分等式约束的连续最优控制问题；(2) 目标函数部分补充退化推算：若固定 $m(t_f) = 62000$ kg 则 $J = 72450 - 62000 = 10450$ kg 为常数，失去优化意义，故 $m(t_f)$ 必须保留为自由变量；(3) 将四组动力学方程统一在 `cases` 环境中展示，避免重复书写；(4) 求解路线改写为两阶段文字描述（可行性求解 → 最终燃油优化），并顺带解释不可行初值破坏收敛与无风固定路径不可行的原因，不再单独设节；(5) 结果部分新增‘对最优轨迹的物理理解’段落，解释高度上界在优化中接近活跃的原因。内容已按 LaTeX 格式输出。”

3. 交互记录 3（概念流程图与机制示意图生成 - gpt-image-2）

- 提示词 (Prompt): “生成一张‘问题三：最优巡航控制建模流程’的信息图。采用从左到右三栏横向流程图布局：左栏为橙色系‘问题输入’模块，包含固定起点、固定终点航程、终点质量下限三个带图标的卡片；中栏为深蓝色系‘连续最优控制模型’模块，包含状态变量、控制变量、目标函数、运动方程、边界约束五个带图标卡片，用向下粗箭头串联；右栏为绿色系‘求解与输出’模块，包含航程域直接配点法加两阶段优化的求解流程与最终输出数值（总油耗 10342.814 kg，终端质量 62107.186 kg）。底部标注物理结论‘先高后稳——尽快爬升至高度上限附近巡航’。扁平化信息图风格，图标为简洁线条风格，蓝灰配色，白底，16:9 横版，适合工

程学术论文插图，全中文标签。”

- AI (gpt-image-2) 的回复：“已生成横向信息图一张（16□9），左中右三栏布局，各模块配有对应功能性图标（靶心、旗帜、天平、节点、操纵杆、火焰、链条、围栏、扳手、图表、灯泡），全图简体中文标签，蓝橙绿三色分区配色。输出文件：Q3_最优巡航控制建模流程.png。”

对 AI 生成内容的采纳情况与人工修改情况：

1. 代码部分：AI 工具（Codex）仅用于基础代码框架的构建与补全。参赛队对生成的代码进行了严格的人工审核、调试与算法核心逻辑校对，确保数值求解方法（航程域直接配点法、非线性规划求解器设置、两阶段优化流程）完全符合人工提出的算法设计。
2. 论文部分：AI 工具（Codex）仅用于文本的辅助补全与语言润色。参赛队对照人工大纲进行了逐字逐句的审核与修改，剔除了所有不实表述，确保作品中不包含直接由 AI 生成且未经审视的内容，且参考文献中绝无 AI“幻觉”产生的虚假文献。
3. 配图部分：AI 工具（gpt-image-2）仅用于概念流程图与机制示意图的生成。参赛队人工撰写了全部提示词，并对生成图片进行了审视与筛选，确保图像语义准确、风格统一。

附录 B 主要程序文件列表

表 15 主要程序文件说明

程序文件	功能说明
问题一	
q1/scripts/atmosphere.py	大气密度、声速与风场
q1/scripts/aircraft_model.py	气动参数与燃油模型
q1/scripts/strategy_constant_speed.py	等速策略高度-质量关系
q1/scripts/strategy_constant_mach.py	等马赫策略高度-质量关系
q1/scripts/simulate.py	两策略数值积分
q1/scripts/pipeline.py	求解入口与结果输出
问题二	
q2/scripts/fuel_path_model.py	路径燃油积分与大气修正
q2/scripts/pipeline.py	主求解脚本
q2/scripts/validate.py	路径积分与敏感性验证
q2/scripts/visualize.py	图表生成脚本
问题三	
q3/scripts/precheck.py	有风/无风可行性预检
q3/scripts/solve_feasibility_no_wind.py	无风可行性求解
q3/scripts/smooth_atmosphere.py	平滑温度与静力大气模型
q3/scripts/solve_feasibility_collocation_no_wind.py	配点与最终燃油优化
q3/scripts/validate.py	验证入口
q3/scripts/visualize.py	图表生成入口
问题四	
q4/scripts/pipeline.py	有风打靶法与灵敏度
q4/scripts/validate.py	终端状态与燃油验证
q4/scripts/visualize.py	论文级图表生成

附录 C 主要源代码

模拟与策略执行引擎——simulate.py

```
1 from __future__ import annotations
2
```



```

3 import sys
4 from pathlib import Path
5 from typing import Callable
6
7 import numpy as np
8 import pandas as pd
9 from scipy.integrate import solve_ivp
10
11 SCRIPT_DIR = Path(__file__).resolve().parent
12 if str(SCRIPT_DIR) not in sys.path:
13     sys.path.insert(0, str(SCRIPT_DIR))
14
15 from aircraft_model import ( # noqa: E402
16     AircraftParameters,
17     drag_n,
18     fuel_penalty,
19     lift_coefficient,
20     load_parameters,
21     validate_parameters,
22 )
23 from atmosphere import density_kgm3, sound_speed_mps, wind_speed_mps # noqa: E402
24 import strategy_constant_mach as mach_strategy # noqa: E402
25 import strategy_constant_speed as speed_strategy # noqa: E402
26
27 StrategyFn = Callable[[float, AircraftParameters], float]
28 DerivativeFn = Callable[[float, AircraftParameters], tuple[float, float]]
29
30 REQUIRED_COLUMNS = [
31     "time_s",
32     "distance_m",
33     "mass_kg",
34     "height_m",
35     "airspeed_mps",
36     "groundspeed_mps",
37     "mach",
38     "density_kgm3",
39     "lift_N",
40     "drag_N",
41     "thrust_N",
42     "fuel_flow_kgs",
43     "climb_rate_mps",
44     "energy_residual",
45     "lift_balance_residual",
46 ]

```

```

47
48
49 def _rates(
50     mass_kg: float,
51     height_from_mass: StrategyFn,
52     airspeed_from_height: StrategyFn,
53     derivatives: DerivativeFn,
54     params: AircraftParameters,
55 ) -> dict[str, float]:
56     height_m = height_from_mass(mass_kg, params)
57     airspeed_mps = airspeed_from_height(height_m, params)
58     dh_dm, d_v_dm = derivatives(mass_kg, params)
59     drag = drag_n(height_m, airspeed_mps, mass_kg, params)
60     penalty = fuel_penalty(airspeed_mps, params)
61     a_term = mass_kg * d_v_dm + (mass_kg * params.g_mps2 / airspeed_mps) * dh_dm
62     denom = 1.0 + params.c_t_kg_per_ns * penalty * a_term
63     if denom <= 0:
64         raise ValueError(f"Implicit fuel-flow denominator is non-positive:
65 {denom}")
66     mass_rate = -params.c_t_kg_per_ns * penalty * drag / denom
67     climb_rate = dh_dm * mass_rate
68     acceleration = d_v_dm * mass_rate
69     thrust = drag + a_term * mass_rate
70     if thrust <= 0:
71         raise ValueError("Thrust must remain positive")
72     fuel_flow = -mass_rate
73     lift = 0.5 * density_kgm3(height_m) * airspeed_mps**2 * params.wing_area_m2 *
74 lift_coefficient(
75     height_m, airspeed_mps, mass_kg, params
76 )
77     groundspeed = airspeed_mps + wind_speed_mps(height_m)
78     energy_residual = (thrust - drag - mass_kg * acceleration - mass_kg *
79 params.g_mps2 * climb_rate / airspeed_mps) / max(
80     thrust, drag, 1.0
81 )
82     lift_balance_residual = (lift - mass_kg * params.g_mps2) / (mass_kg *
83 params.g_mps2)
84     return {
85         "height_m": height_m,
86         "airspeed_mps": airspeed_mps,
87         "groundspeed_mps": groundspeed,
88         "mach": airspeed_mps / sound_speed_mps(height_m),
89         "density_kgm3": density_kgm3(height_m),
90         "lift_N": lift,

```

```

87         "drag_N": drag,
88         "thrust_N": thrust,
89         "fuel_flow_kgs": fuel_flow,
90         "climb_rate_mps": climb_rate,
91         "mass_rate_kgs": mass_rate,
92         "energy_residual": energy_residual,
93         "lift_balance_residual": lift_balance_residual,
94     }
95
96
97 def simulate_strategy(
98     *,
99     strategy_name: str,
100     height_from_mass: StrategyFn,
101     airspeed_from_height: StrategyFn,
102     derivatives: DerivativeFn,
103     params: AircraftParameters,
104 ) -> pd.DataFrame:
105     """Integrate mass and ground distance until the confirmed terminal mass."""
106     validate_parameters(params)
107
108     def rhs(_time_s: float, state: np.ndarray) -> list[float]:
109         mass_kg = float(state[0])
110         rates = _rates(mass_kg, height_from_mass, airspeed_from_height,
111             derivatives, params)
112         return [rates["mass_rate_kgs"], rates["groundspeed_mps"]]
113
114     def terminal_mass(_time_s: float, state: np.ndarray) -> float:
115         return float(state[0] - params.mf_kg)
116
117     terminal_mass.terminal = True # type: ignore[attr-defined]
118     terminal_mass.direction = -1 # type: ignore[attr-defined]
119
120     solution = solve_ivp(
121         rhs,
122         t_span=(0.0, 20_000.0),
123         y0=np.array([params.m0_kg, 0.0]),
124         events=terminal_mass,
125         rtol=params.rtol,
126         atol=params.atol,
127         max_step=params.max_step_s,
128         dense_output=False,
129     )
130     if not solution.success or len(solution.t_events[0]) != 1:

```

```

130         raise RuntimeError(f"{strategy_name} integration failed:
{solution.message}")
131
132     records: list[dict[str, float | str]] = []
133     for time_s, mass_kg, distance_m in zip(solution.t, solution.y[0],
solution.y[1], strict=True):
134         rates = _rates(float(mass_kg), height_from_mass, airspeed_from_height,
derivatives, params)
135         record: dict[str, float | str] = {
136             "strategy": strategy_name,
137             "time_s": float(time_s),
138             "distance_m": float(distance_m),
139             "mass_kg": float(mass_kg),
140         }
141         record.update({key: rates[key] for key in REQUIRED_COLUMNS if key in
rates})
142         records.append(record)
143     frame = pd.DataFrame.from_records(records)
144     frame = frame[["strategy", *REQUIRED_COLUMNS]]
145     if not frame["mass_kg"].is_monotonic_decreasing:
146         raise RuntimeError(f"{strategy_name} mass is not monotone decreasing")
147     return frame
148
149
150 def run_strategies(params: AircraftParameters | None = None) -> dict[str,
pd.DataFrame]:
151     """Run both confirmed q1 cruise-climb strategies."""
152     params = params or AircraftParameters()
153     return {
154         "constant_speed": simulate_strategy(
155             strategy_name="constant_speed",
156             height_from_mass=speed_strategy.height_from_mass,
157             airspeed_from_height=speed_strategy.airspeed_from_height,
158             derivatives=speed_strategy.derivatives_wrt_mass,
159             params=params,
160         ),
161         "constant_mach": simulate_strategy(
162             strategy_name="constant_mach",
163             height_from_mass=mach_strategy.height_from_mass,
164             airspeed_from_height=mach_strategy.airspeed_from_height,
165             derivatives=mach_strategy.derivatives_wrt_mass,
166             params=params,
167         ),
168     }

```

```

169
170
171 def comparison_table(outputs: dict[str, pd.DataFrame], params:
    AircraftParameters) -> pd.DataFrame:
172     rows = []
173     for strategy, frame in outputs.items():
174         elapsed_s = frame["time_s"].iloc[-1] - frame["time_s"].iloc[0]
175         if elapsed_s <= 0:
176             raise ValueError(f"{strategy} elapsed time must be positive")
177         mean_climb_rate = (frame["height_m"].iloc[-1] -
    frame["height_m"].iloc[0]) / elapsed_s
178         mean_groundspeed = (frame["distance_m"].iloc[-1] -
    frame["distance_m"].iloc[0]) / elapsed_s
179         if "airspeed_mps" in frame:
180             air_distance = float(np.trapezoid(frame["airspeed_mps"],
    frame["time_s"]))
181             wind_distance = float(frame["distance_m"].iloc[-1] -
    frame["distance_m"].iloc[0] - air_distance)
182         else:
183             air_distance = float("nan")
184             wind_distance = float("nan")
185         rows.append(
186             {
187                 "strategy": strategy,
188                 "final_time_s": frame["time_s"].iloc[-1],
189                 "final_distance_m": frame["distance_m"].iloc[-1],
190                 "air_distance_m": air_distance,
191                 "wind_distance_contribution_m": wind_distance,
192                 "final_height_m": frame["height_m"].iloc[-1],
193                 "fuel_used_kg": params.m0_kg - params.mf_kg,
194                 "mean_climb_rate_mps": mean_climb_rate,
195                 "max_climb_rate_mps": frame["climb_rate_mps"].max(),
196                 "mean_groundspeed_mps": mean_groundspeed,
197                 "max_abs_energy_residual": frame["energy_residual"].abs().max(),
198                 "max_abs_lift_balance_residual":
    frame["lift_balance_residual"].abs().max(),
199             }
200         )
201     return pd.DataFrame(rows)
202
203
204 def run_from_config(config_path: str | None = None) -> tuple[AircraftParameters,
    dict[str, pd.DataFrame], pd.DataFrame]:
205     params = load_parameters(config_path)

```

```

206     outputs = run_strategies(params)
207     return params, outputs, comparison_table(outputs, params)

```

等速策略高度闭合条件——strategy_constant_speed.py

```

1  from __future__ import annotations
2
3  from aircraft_model import AircraftParameters
4  from atmosphere import H_RHO_M, density_kgm3
5
6
7  def height_from_mass(mass_kg: float, params: AircraftParameters) -> float:
8      """Constant-true-airspeed cruise-climb height from CL closure."""
9      return params.h0_m - H_RHO_M * __import__("math").log(mass_kg / params.m0_kg)
10
11
12  def airspeed_from_height(height_m: float, params: AircraftParameters) -> float:
13      """Constant-speed strategy keeps true airspeed fixed."""
14      _ = height_m
15      return params.v0_mps
16
17
18  def derivatives_wrt_mass(mass_kg: float, params: AircraftParameters) ->
19      tuple[float, float]:
20      """Return dh/dm and dV/dm for constant true airspeed."""
21      if mass_kg <= 0:
22          raise ValueError("Mass must be positive")
23      return -H_RHO_M / mass_kg, 0.0
24
25  def density_ratio_residual(mass_kg: float, params: AircraftParameters) -> float:
26      """Residual for m/m0 = rho/rho0_at_initial under constant CL and V."""
27      height_m = height_from_mass(mass_kg, params)
28      return mass_kg / params.m0_kg - density_kgm3(height_m) /
29      density_kgm3(params.h0_m)

```

大气模型与固定路径油耗积分——fuel_path_model.py

```

1  from __future__ import annotations
2
3  import math
4  from dataclasses import dataclass
5
6  import numpy as np

```

```

7 import pandas as pd
8 from scipy.integrate import solve_ivp
9
10
11 @dataclass(frozen=True)
12 class Q2Parameters:
13     m0_kg: float = 72_450.0
14     wing_area_m2: float = 112.3
15     cd0: float = 0.022
16     induced_k: float = 0.045
17     c_t_kg_per_ns: float = 2.8e-4
18     beta_s2pm2: float = 0.003
19     v_opt_mps: float = 235.0
20     h0_m: float = 9500.0
21     v0_mps: float = 240.0
22     reference_distance_m: float = 200_668.44247343735
23     terminal_mass_kg: float = 62_000.0
24     temperature_offset_k: float = 10.0
25     temperature_offsets_k: tuple[float, ...] = (-10.0, -5.0, -2.0, 0.0, 2.0, 5.0,
10.0)
26     g_mps2: float = 9.80665
27     rtol: float = 1.0e-8
28     atol: float = 1.0e-9
29     max_step_s: float = 2.0
30
31
32 RH00_KGM3 = 1.225
33 P0_PA = 101_325.0
34 T0_K = 288.15
35 LAPSE_KPM = 0.0065
36 GAMMA_AIR = 1.4
37 R_AIR = 287.05
38 WIND_BASE_MPS = 20.0
39 WIND_QUAD_COEFF = 3.0e-5
40 WIND_REF_HEIGHT_M = 10_000.0
41 TROPOPAUSE_M = 11_000.0
42
43
44 def isa_temperature_k(height_m: float) -> float:
45     temperature = T0_K - LAPSE_KPM * min(height_m, TROPOPAUSE_M)
46     if temperature <= 0:
47         raise ValueError(f"Non-physical ISA temperature at height={height_m}")
48     return temperature
49

```

```

50
51 def isa_pressure_pa(height_m: float) -> float:
52     exponent = 9.80665 / (R_AIR * LAPSE_KPM)
53     if height_m <= TROPOPAUSE_M:
54         temperature = isa_temperature_k(height_m)
55         pressure = P0_PA * (temperature / T0_K) ** exponent
56     else:
57         t11 = isa_temperature_k(TROPOPAUSE_M)
58         p11 = P0_PA * (t11 / T0_K) ** exponent
59         pressure = p11 * math.exp(-9.80665 * (height_m - TROPOPAUSE_M) / (R_AIR *
60         t11))
61     if pressure <= 0:
62         raise ValueError(f"Non-physical ISA pressure at height={height_m}")
63     return pressure
64
65 def corrected_pressure_pa(height_m: float, *, temperature_offset_k: float = 0.0)
66     -> float:
67     """Return hydrostatic pressure for a constant temperature offset in layered
68     ISA."""
69     sea_level_temperature = T0_K + temperature_offset_k
70     exponent = 9.80665 / (R_AIR * LAPSE_KPM)
71     if height_m <= TROPOPAUSE_M:
72         temperature = sea_level_temperature - LAPSE_KPM * height_m
73         pressure = P0_PA * (temperature / sea_level_temperature) ** exponent
74     else:
75         t11 = sea_level_temperature - LAPSE_KPM * TROPOPAUSE_M
76         p11 = P0_PA * (t11 / sea_level_temperature) ** exponent
77         temperature = t11
78         pressure = p11 * math.exp(-9.80665 * (height_m - TROPOPAUSE_M) / (R_AIR *
79         temperature))
80     if sea_level_temperature <= 0 or temperature <= 0:
81         raise ValueError(f"Non-physical corrected temperature at
82         height={height_m}")
83     if pressure <= 0:
84         raise ValueError(f"Non-physical corrected pressure at height={height_m}")
85     return pressure
86
87 def atmosphere(height_m: float, *, temperature_offset_k: float = 0.0) ->
88     tuple[float, float, float, float]:
89     """Return temperature, density, sound speed, and hydrostatic pressure."""
90     temperature = isa_temperature_k(height_m) + temperature_offset_k
91     if temperature <= 0:

```



```

88         raise ValueError(f"Non-physical corrected temperature at
height={height_m}")
89     pressure = corrected_pressure_pa(height_m,
temperature_offset_k=temperature_offset_k)
90     density = pressure / (R_AIR * temperature)
91     sound_speed = math.sqrt(GAMMA_AIR * R_AIR * temperature)
92     return temperature, density, sound_speed, pressure
93
94
95 def atmosphere_layer(height_m: float) -> str:
96     return "troposphere" if height_m <= TROPOPAUSE_M else "lower_stratosphere"
97
98
99 def wind_speed_mps(height_m: float) -> float:
100     return WIND_BASE_MPS + WIND_QUAD_COEFF * (height_m - WIND_REF_HEIGHT_M) ** 2
101
102
103 @dataclass(frozen=True)
104 class ReferencePath:
105     distance_m: np.ndarray
106     height_m: np.ndarray
107     airspeed_mps: np.ndarray
108     dh_dx: np.ndarray
109
110
111 def load_reference_path(root: str | None = None) -> ReferencePath:
112     from pathlib import Path
113
114     base = Path(root) if root is not None else Path(__file__).resolve().parents[3]
115     frame = pd.read_csv(base / "artifacts" / "q1" / "data" /
"constant_speed_profile.csv")
116     required = {"distance_m", "height_m", "airspeed_mps"}
117     missing = required.difference(frame.columns)
118     if missing:
119         raise ValueError(f"q1 reference path missing columns: {sorted(missing)}")
120     path = frame.sort_values("distance_m").drop_duplicates("distance_m")
121     return ReferencePath(
122         distance_m=path["distance_m"].to_numpy(dtype=float),
123         height_m=path["height_m"].to_numpy(dtype=float),
124         airspeed_mps=path["airspeed_mps"].to_numpy(dtype=float),
125         dh_dx=np.gradient(
126             path["height_m"].to_numpy(dtype=float),
127             path["distance_m"].to_numpy(dtype=float),
128         ),

```

```

129     )
130
131
132 def _reference_state(distance_m: float, reference_path: ReferencePath) ->
    tuple[float, float, float]:
133     x = reference_path.distance_m
134     h = reference_path.height_m
135     v = reference_path.airspeed_mps
136     gradient = reference_path.dh_dx
137     height_m = float(np.interp(distance_m, x, h))
138     airspeed_mps = float(np.interp(distance_m, x, v))
139     dh_dx = float(np.interp(distance_m, x, gradient))
140     return height_m, airspeed_mps, dh_dx
141
142
143 def _rates(
144     mass_kg: float,
145     distance_m: float,
146     params: Q2Parameters,
147     *,
148     temperature_offset_k: float,
149     reference_path: ReferencePath,
150 ) -> dict[str, float | str]:
151     height_m, airspeed_mps, dh_dx = _reference_state(distance_m, reference_path)
152     temperature_k, density_kgm3, sound_speed_mps, pressure_pa = atmosphere(
153         height_m, temperature_offset_k=temperature_offset_k
154     )
155     cl = 2.0 * mass_kg * params.g_mps2 / (density_kgm3 * airspeed_mps**2 *
    params.wing_area_m2)
156     cd = params.cd0 + params.induced_k * cl**2
157     drag_n = 0.5 * density_kgm3 * airspeed_mps**2 * params.wing_area_m2 * cd
158     fuel_penalty = 1.0 + params.beta_s2pm2 * (airspeed_mps - params.v_opt_mps) **
    2
159     ground_speed_mps = airspeed_mps + wind_speed_mps(height_m)
160     climb_rate_mps = dh_dx * ground_speed_mps
161     thrust_n = drag_n + mass_kg * params.g_mps2 * climb_rate_mps / airspeed_mps
162     denominator = 1.0
163     if denominator <= 0:
164         raise ValueError("Implicit mass ODE denominator is non-positive")
165     fuel_flow_kgs = params.c_t_kg_per_ns * fuel_penalty * thrust_n
166     mass_rate_kgs = -fuel_flow_kgs
167     fuel_flow_kgs = -mass_rate_kgs
168     return {
169         "reference_height_m": height_m,

```

```

170     "height_m": height_m,
171     "temperature_k": temperature_k,
172     "density_kgm3": density_kgm3,
173     "sound_speed_mps": sound_speed_mps,
174     "pressure_pa": pressure_pa,
175     "atmosphere_layer": atmosphere_layer(height_m),
176     "airspeed_mps": airspeed_mps,
177     "groundspeed_mps": ground_speed_mps,
178     "mach": airspeed_mps / sound_speed_mps,
179     "cl": cl,
180     "cd": cd,
181     "drag_n": drag_n,
182     "thrust_n": thrust_n,
183     "fuel_flow_kgs": fuel_flow_kgs,
184     "climb_rate_mps": climb_rate_mps,
185     "dh_dx": dh_dx,
186     "mass_rate_kgs": mass_rate_kgs,
187     "fuel_per_meter_kgpm": fuel_flow_kgs / ground_speed_mps,
188     "implicit_denominator": denominator,
189 }
190
191
192 def simulate_fixed_range(
193     scenario: str,
194     *,
195     params: Q2Parameters | None = None,
196     temperature_offset_k: float = 0.0,
197     reference_path: ReferencePath | None = None,
198     target_distance_m: float | None = None,
199     stop_at_terminal_mass: bool = False,
200 ) -> pd.DataFrame:
201     params = params or Q2Parameters()
202     reference_path = reference_path or load_reference_path()
203     target_distance = params.reference_distance_m if target_distance_m is None
204     else target_distance_m
205
206     def rhs(_time_s: float, state: np.ndarray) -> list[float]:
207         mass_kg = float(state[0])
208         distance_m = float(state[1])
209         rates = _rates(
210             mass_kg,
211             distance_m,
212             params,
213             temperature_offset_k=temperature_offset_k,

```

```

213         reference_path=reference_path,
214     )
215     return [rates["mass_rate_kgs"], rates["groundspeed_mps"]]
216
217 def target_distance_event(_time_s: float, state: np.ndarray) -> float:
218     return float(state[1] - target_distance)
219
220 target_distance_event.terminal = True # type: ignore[attr-defined]
221 target_distance_event.direction = 1 # type: ignore[attr-defined]
222
223 def terminal_mass_event(_time_s: float, state: np.ndarray) -> float:
224     return float(state[0] - params.terminal_mass_kg)
225
226 terminal_mass_event.terminal = True # type: ignore[attr-defined]
227 terminal_mass_event.direction = -1 # type: ignore[attr-defined]
228 events = terminal_mass_event if stop_at_terminal_mass else
target_distance_event
229
230 solution = solve_ivp(
231     rhs,
232     t_span=(0.0, 30_000.0),
233     y0=np.array([params.m0_kg, 0.0]),
234     events=events,
235     rtol=params.rtol,
236     atol=params.atol,
237     max_step=params.max_step_s,
238 )
239 if not solution.success or len(solution.t_events[0]) != 1:
240     raise RuntimeError(f"{scenario} integration failed: {solution.message}")
241
242 records: list[dict[str, float | str]] = []
243 for time_s, mass_kg, distance_m in zip(solution.t, solution.y[0],
solution.y[1], strict=True):
244     rates = _rates(
245         float(mass_kg),
246         float(distance_m),
247         params,
248         temperature_offset_k=temperature_offset_k,
249         reference_path=reference_path,
250     )
251     records.append(
252         {
253             "scenario": scenario,
254             "time_s": float(time_s),

```

```

255         "distance_m": float(distance_m),
256         "mass_kg": float(mass_kg),
257         **rates,
258     }
259 )
260 frame = pd.DataFrame.from_records(records)
261 frame["cumulative_fuel_time_kg"] = _cumulative_trapezoid(frame["time_s"],
262 frame["fuel_flow_kgs"])
263 frame["cumulative_fuel_path_kg"] = _cumulative_trapezoid(frame["distance_m"],
264 frame["fuel_per_meter_kgpm"])
265 frame["path_integral_mass_kg"] = params.m0_kg - frame["mass_kg"]
266 return frame
267
268 def _cumulative_trapezoid(x_values: pd.Series, y_values: pd.Series) -> np.ndarray:
269     x = x_values.to_numpy(dtype=float)
270     y = y_values.to_numpy(dtype=float)
271     cumulative = np.zeros_like(x)
272     if len(x) > 1:
273         increments = 0.5 * (y[1:] + y[:-1]) * np.diff(x)
274         cumulative[1:] = np.cumsum(increments)
275     return cumulative
276
277 def temperature_scenario_name(offset_k: float) -> str:
278     if abs(offset_k) < 1e-12:
279         return "standard_isa"
280     magnitude = f"{abs(offset_k):g}"
281     prefix = "plus" if offset_k > 0 else "minus"
282     return f"temp_{prefix}_{magnitude}K"
283
284
285 def simulate_temperature_scenarios(params: Q2Parameters | None = None) ->
286 dict[str, pd.DataFrame]:
287     params = params or Q2Parameters()
288     reference_path = load_reference_path()
289     terminal_profiles = [
290         simulate_fixed_range(
291             f"{temperature_scenario_name(offset)}_terminal_mass",
292             params=params,
293             temperature_offset_k=offset,
294             reference_path=reference_path,
295             stop_at_terminal_mass=True,
296         )

```

```

296         for offset in params.temperature_offsets_k
297     ]
298     target_distance_m = min(float(profile["distance_m"].iloc[-1]) for profile in
terminal_profiles)
299     return {
300         temperature_scenario_name(offset): simulate_fixed_range(
301             temperature_scenario_name(offset),
302             params=params,
303             temperature_offset_k=offset,
304             reference_path=reference_path,
305             target_distance_m=target_distance_m,
306         )
307         for offset in params.temperature_offsets_k
308     }
309
310
311 def fuel_summary(profiles: dict[str, pd.DataFrame], params: Q2Parameters) ->
pd.DataFrame:
312     rows = []
313     for scenario, frame in profiles.items():
314         rows.append(
315             {
316                 "scenario": scenario,
317                 "temperature_offset_k": _scenario_temperature_offset(scenario),
318                 "final_time_s": frame["time_s"].iloc[-1],
319                 "final_distance_m": frame["distance_m"].iloc[-1],
320                 "final_mass_kg": frame["mass_kg"].iloc[-1],
321                 "fuel_used_kg": params.m0_kg - frame["mass_kg"].iloc[-1],
322                 "mean_fuel_flow_kgs": (params.m0_kg - frame["mass_kg"].iloc[-1])
323                 / max(frame["time_s"].iloc[-1], 1e-9),
324                 "mean_fuel_per_meter_kgpm": (params.m0_kg -
frame["mass_kg"].iloc[-1])
325                 / max(frame["distance_m"].iloc[-1], 1e-9),
326                 "max_fuel_flow_kgs": frame["fuel_flow_kgs"].max(),
327                 "min_implicit_denominator": frame["implicit_denominator"].min(),
328             }
329         )
330     summary = pd.DataFrame(rows)
331     standard_fuel = float(summary.loc[summary["scenario"] == "standard_isa",
"fuel_used_kg"].iloc[0])
332     summary["fuel_delta_vs_standard_kg"] = summary["fuel_used_kg"] - standard_fuel
333     summary["fuel_delta_vs_standard_pct"] = 100.0 *
summary["fuel_delta_vs_standard_kg"] / standard_fuel
334     return summary

```

```

335
336
337 def _scenario_temperature_offset(scenario: str) -> float:
338     if scenario == "standard_isa":
339         return 0.0
340     parts = scenario.split("_")
341     if len(parts) != 3 or parts[0] != "temp":
342         raise ValueError(f"Cannot parse temperature offset from
scenario={scenario}")
343     magnitude = float(parts[2].removesuffix("K"))
344     return magnitude if parts[1] == "plus" else -magnitude

```

无风可行性验证 (Fourier 参数化 + Powell 求解) —— solve_feasibility_no_wind.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import math
6  import sys
7  from dataclasses import dataclass
8  from pathlib import Path
9  from typing import Any
10
11 import numpy as np
12 import pandas as pd
13 import yaml
14 from scipy.integrate import solve_ivp
15 from scipy.optimize import minimize
16
17 ROOT = Path(__file__).resolve().parents[3]
18 SRC = ROOT / "src"
19 Q2_SCRIPTS = ROOT / "questions" / "q2" / "scripts"
20 for path in [SRC, Q2_SCRIPTS]:
21     if str(path) not in sys.path:
22         sys.path.insert(0, str(path))
23
24 from fuel_path_model import Q2Parameters, ReferencePath, atmosphere,
    load_reference_path # noqa: E402
25 from modeling_common.artifacts import save_table # noqa: E402
26 from modeling_common.paths import project_root # noqa: E402
27
28
29 @dataclass(frozen=True)

```

```

30 class Q3Config:
31     target_distance_m: float
32     terminal_height_m: float
33     terminal_airspeed_mps: float
34     h_min_m: float
35     h_max_m: float
36     v_min_mps: float
37     v_max_mps: float
38     mach_max: float
39     thrust_min_n: float
40     thrust_max_n: float
41     gamma_max_rad: float
42     terminal_mass_min_kg: float
43
44
45 @dataclass(frozen=True)
46 class PathEvaluation:
47     trajectory: pd.DataFrame
48     final_mass_kg: float
49     final_time_s: float
50     terminal_mass_slack_kg: float
51     max_constraint_violation: float
52     max_abs_path_residual: float
53
54
55 def parse_args() -> argparse.Namespace:
56     parser = argparse.ArgumentParser(description="q3 no-wind feasibility gate")
57     parser.add_argument("--config", default="configs/default.yaml")
58     parser.add_argument("--nodes", type=int, default=31)
59     return parser.parse_args()
60
61
62 def load_q3_config(root: Path, config_path: str) -> Q3Config:
63     with (root / config_path).open("r", encoding="utf-8") as file:
64         config = yaml.safe_load(file)
65     q3 = config.get("q3_optimal_control")
66     if not isinstance(q3, dict):
67         raise ValueError("configs/default.yaml missing q3_optimal_control")
68     bounds = q3.get("bounds", {})
69     return Q3Config(
70         target_distance_m=float(q3["fixed_range_m"]),
71         terminal_height_m=float(q3["terminal_height_m"]),
72         terminal_airspeed_mps=float(q3["terminal_airspeed_mps"]),
73         h_min_m=float(bounds["h_min_m"]),

```



```

74     h_max_m=float(bounds["h_max_m"]),
75     v_min_mps=float(bounds["v_min_mps"]),
76     v_max_mps=float(bounds["v_max_mps"]),
77     mach_max=float(bounds["mach_max"]),
78     thrust_min_n=float(bounds["thrust_min_n"]),
79     thrust_max_n=float(bounds["thrust_max_n"]),
80     gamma_max_rad=float(bounds["gamma_max_rad"]),
81     terminal_mass_min_kg=float(bounds["mass_min_kg"]),
82 )
83
84
85 def _reference_height(distance_m: np.ndarray, reference_path: ReferencePath) ->
    np.ndarray:
86     return np.interp(distance_m, reference_path.distance_m,
87                       reference_path.height_m)
88
89 def _reference_dh_dx(distance_m: np.ndarray, reference_path: ReferencePath) ->
    np.ndarray:
90     return np.interp(distance_m, reference_path.distance_m, reference_path.dh_dx)
91
92
93 def _profile(
94     distance_m: np.ndarray,
95     variables: np.ndarray,
96     *,
97     reference_path: ReferencePath,
98     q3: Q3Config,
99 ) -> dict[str, np.ndarray]:
100     target = q3.target_distance_m
101     sigma = distance_m / target
102     h1, h2, v1, v2 = variables
103     reference_start = float(np.interp(0.0, reference_path.distance_m,
104                                       reference_path.height_m))
105     reference_end = float(np.interp(target, reference_path.distance_m,
106                                       reference_path.height_m))
107     initial_height_m = 9500.0
108     start_delta = initial_height_m - reference_start
109     end_delta = q3.terminal_height_m - reference_end
110     height = _reference_height(distance_m, reference_path)
111     dh_dx = _reference_dh_dx(distance_m, reference_path)
112     height = height + (1.0 - sigma) * start_delta + sigma * end_delta
113     dh_dx = dh_dx + (end_delta - start_delta) / target
114     height = height + h1 * np.sin(math.pi * sigma) + h2 * np.sin(2.0 * math.pi *

```

```

sigma)
113 dh_dx = dh_dx + h1 * math.pi / target * np.cos(math.pi * sigma)
114 dh_dx = dh_dx + h2 * 2.0 * math.pi / target * np.cos(2.0 * math.pi * sigma)
115 airspeed = q3.terminal_airspeed_mps
116 airspeed = airspeed + v1 * np.sin(math.pi * sigma) + v2 * np.sin(2.0 *
math.pi * sigma)
117 dV_dx = v1 * math.pi / target * np.cos(math.pi * sigma)
118 dV_dx = dV_dx + v2 * 2.0 * math.pi / target * np.cos(2.0 * math.pi * sigma)
119 return {
120     "height_m": height,
121     "dh_dx": dh_dx,
122     "airspeed_mps": airspeed,
123     "dV_dx": dV_dx,
124 }
125
126
127 def _rates_at(
128     *,
129     mass_kg: float,
130     height_m: float,
131     dh_dx: float,
132     airspeed_mps: float,
133     dV_dx: float,
134     params: Q2Parameters,
135     q3: Q3Config,
136 ) -> dict[str, float]:
137     gamma_rad = math.atan(dh_dx)
138     temperature_k, density_kgm3, sound_speed_mps, _pressure_pa =
atmosphere(height_m)
139     cl = (
140         2.0
141         * mass_kg
142         * params.g_mps2
143         * math.cos(gamma_rad)
144         / (density_kgm3 * airspeed_mps**2 * params.wing_area_m2)
145     )
146     cd = params.cd0 + params.induced_k * cl**2
147     drag_n = 0.5 * density_kgm3 * airspeed_mps**2 * params.wing_area_m2 * cd
148     ground_speed_mps = airspeed_mps * math.cos(gamma_rad)
149     if ground_speed_mps <= 0.0:
150         raise ValueError("non-positive no-wind ground speed")
151     thrust_n = mass_kg * (ground_speed_mps * dV_dx + params.g_mps2 *
math.sin(gamma_rad)) + drag_n
152     fuel_penalty = 1.0 + params.beta_s2pm2 * (airspeed_mps - params.v_opt_mps) **

```

```

2
153 fuel_per_meter_kgpm = params.c_t_kg_per_ns * thrust_n * fuel_penalty /
ground_speed_mps
154 mach = airspeed_mps / sound_speed_mps
155 path_h_residual = dh_dx - airspeed_mps * math.sin(gamma_rad) /
ground_speed_mps
156 path_v_residual = dV_dx - ((thrust_n - drag_n) / mass_kg - params.g_mps2 *
math.sin(gamma_rad)) / ground_speed_mps
157 violation = max(
158     q3.h_min_m - height_m,
159     height_m - q3.h_max_m,
160     q3.v_min_mps - airspeed_mps,
161     airspeed_mps - q3.v_max_mps,
162     mach - q3.mach_max,
163     q3.thrust_min_n - thrust_n,
164     thrust_n - q3.thrust_max_n,
165     abs(gamma_rad) - q3.gamma_max_rad,
166     0.0,
167 )
168 return {
169     "temperature_k": temperature_k,
170     "density_kgm3": density_kgm3,
171     "airspeed_mps": airspeed_mps,
172     "groundspeed_mps": ground_speed_mps,
173     "gamma_rad": gamma_rad,
174     "mach": mach,
175     "cl": cl,
176     "cd": cd,
177     "drag_n": drag_n,
178     "thrust_n": thrust_n,
179     "fuel_per_meter_kgpm": fuel_per_meter_kgpm,
180     "dm_dx": -fuel_per_meter_kgpm,
181     "dt_dx": 1.0 / ground_speed_mps,
182     "constraint_violation": violation,
183     "path_residual": max(abs(path_h_residual), abs(path_v_residual)),
184 }
185
186
187 def evaluate_path(
188     variables: np.ndarray,
189     *,
190     q3: Q3Config,
191     params: Q2Parameters,
192     reference_path: ReferencePath,

```

```

193     nodes: int,
194 ) -> PathEvaluation:
195     distance_nodes = np.linspace(0.0, q3.target_distance_m, nodes)
196
197     def rhs(distance_m: float, state: np.ndarray) -> list[float]:
198         profile = _profile(np.array([distance_m]), variables,
199         reference_path=reference_path, q3=q3)
200         rates = _rates_at(
201             mass_kg=float(state[0]),
202             height_m=float(profile["height_m"][0]),
203             dh_dx=float(profile["dh_dx"][0]),
204             airspeed_mps=float(profile["airspeed_mps"][0]),
205             dV_dx=float(profile["dV_dx"][0]),
206             params=params,
207             q3=q3,
208         )
209         return [rates["dm_dx"], rates["dt_dx"]]
210
211     solution = solve_ivp(
212         rhs,
213         (0.0, q3.target_distance_m),
214         np.array([params.m0_kg, 0.0]),
215         t_eval=distance_nodes,
216         rtol=1.0e-7,
217         atol=1.0e-8,
218         max_step=q3.target_distance_m / max(nodes - 1, 1),
219     )
220     if not solution.success:
221         raise RuntimeError(solution.message)
222
223     profile = _profile(distance_nodes, variables, reference_path=reference_path,
224     q3=q3)
225     records: list[dict[str, float]] = []
226     max_violation = 0.0
227     max_residual = 0.0
228     for index, distance_m in enumerate(distance_nodes):
229         rates = _rates_at(
230             mass_kg=float(solution.y[0, index]),
231             height_m=float(profile["height_m"][index]),
232             dh_dx=float(profile["dh_dx"][index]),
233             airspeed_mps=float(profile["airspeed_mps"][index]),
234             dV_dx=float(profile["dV_dx"][index]),
235             params=params,
236             q3=q3,

```

```

235     )
236     max_violation = max(max_violation, rates["constraint_violation"])
237     max_residual = max(max_residual, rates["path_residual"])
238     records.append(
239         {
240             "distance_m": float(distance_m),
241             "height_m": float(profile["height_m"][index]),
242             "airspeed_mps": float(profile["airspeed_mps"][index]),
243             "mass_kg": float(solution.y[0, index]),
244             "time_s": float(solution.y[1, index]),
245             **rates,
246         }
247     )
248     final_mass = float(solution.y[0, -1])
249     return PathEvaluation(
250         trajectory=pd.DataFrame.from_records(records),
251         final_mass_kg=final_mass,
252         final_time_s=float(solution.y[1, -1]),
253         terminal_mass_slack_kg=max(0.0, q3.terminal_mass_min_kg - final_mass),
254         max_constraint_violation=max_violation,
255         max_abs_path_residual=max_residual,
256     )
257
258
259 def objective(
260     variables: np.ndarray,
261     *,
262     q3: Q3Config,
263     params: Q2Parameters,
264     reference_path: ReferencePath,
265     nodes: int,
266 ) -> float:
267     try:
268         result = evaluate_path(
269             variables,
270             q3=q3,
271             params=params,
272             reference_path=reference_path,
273             nodes=nodes,
274         )
275     except (RuntimeError, ValueError, FloatingPointError):
276         return 1.0e9
277     return result.terminal_mass_slack_kg + 1.0e4 *
result.max_constraint_violation**2

```

```

278
279
280 def solve_gate(q3: Q3Config, params: Q2Parameters, reference_path: ReferencePath,
281               nodes: int) -> tuple[np.ndarray, PathEvaluation]:
282     baseline_variables = np.zeros(4)
283     starts = [
284         baseline_variables,
285         np.array([1900.0, 0.0, -5.5, -0.4]),
286     ]
287     bounds = [(-1500.0, 2200.0), (-1200.0, 1200.0), (-30.0, 10.0), (-12.0, 12.0)]
288     best_x = baseline_variables
289     best_value = math.inf
290     objective_fn = lambda x: objective( # noqa: E731
291         x,
292         q3=q3,
293         params=params,
294         reference_path=reference_path,
295         nodes=nodes,
296     )
297     for start in starts:
298         result = minimize(
299             objective_fn,
300             start,
301             method="Powell",
302             bounds=bounds,
303             options={"maxiter": 35, "xtol": 1.0e-3, "ftol": 1.0e-3, "disp":
False},
304         )
305         if result.fun < best_value:
306             best_value = float(result.fun)
307             best_x = np.asarray(result.x, dtype=float)
308     return best_x, evaluate_path(best_x, q3=q3, params=params,
reference_path=reference_path, nodes=nodes)
309
310 def trajectory_diagnostics(trajectory: pd.DataFrame, q3: Q3Config) -> dict[str,
float]:
311     height_lower_margin = trajectory["height_m"] - q3.h_min_m
312     height_upper_margin = q3.h_max_m - trajectory["height_m"]
313     airspeed_lower_margin = trajectory["airspeed_mps"] - q3.v_min_mps
314     airspeed_upper_margin = q3.v_max_mps - trajectory["airspeed_mps"]
315     thrust_lower_margin = trajectory["thrust_n"] - q3.thrust_min_n
316     thrust_upper_margin = q3.thrust_max_n - trajectory["thrust_n"]
317     gamma_margin = q3.gamma_max_rad - trajectory["gamma_rad"].abs()

```

```

318 mach_margin = q3.mach_max - trajectory["mach"]
319 return {
320     "min_height_m": float(trajectory["height_m"].min()),
321     "max_height_m": float(trajectory["height_m"].max()),
322     "min_airspeed_mps": float(trajectory["airspeed_mps"].min()),
323     "max_airspeed_mps": float(trajectory["airspeed_mps"].max()),
324     "min_thrust_n": float(trajectory["thrust_n"].min()),
325     "max_thrust_n": float(trajectory["thrust_n"].max()),
326     "min_gamma_rad": float(trajectory["gamma_rad"].min()),
327     "max_gamma_rad": float(trajectory["gamma_rad"].max()),
328     "max_mach": float(trajectory["mach"].max()),
329     "min_cl": float(trajectory["cl"].min()),
330     "max_cl": float(trajectory["cl"].max()),
331     "height_margin_min_m": float(pd.concat([height_lower_margin,
height_upper_margin]).min()),
332     "airspeed_margin_min_mps": float(pd.concat([airspeed_lower_margin,
airspeed_upper_margin]).min()),
333     "thrust_margin_min_n": float(pd.concat([thrust_lower_margin,
thrust_upper_margin]).min()),
334     "gamma_margin_min_rad": float(gamma_margin.min()),
335     "mach_margin_min": float(mach_margin.min()),
336 }
337
338
339 def fixed_path_shortfall_kg(root: Path, fallback: PathEvaluation, q3: Q3Config)
-> float:
340     table_path = root / "questions" / "q3" / "artifacts" / "tables" /
"baseline_feasibility.csv"
341     if table_path.exists():
342         table = pd.read_csv(table_path)
343         row = table.loc[table["wind_model"] == "no_wind"]
344         if not row.empty and "final_mass_kg" in row.columns:
345             return max(0.0, q3.terminal_mass_min_kg -
float(row["final_mass_kg"].iloc[0]))
346     return fallback.terminal_mass_slack_kg
347
348
349 def main() -> int:
350     args = parse_args()
351     if args.nodes < 7:
352         raise ValueError("--nodes must be at least 7")
353     root = project_root()
354     q3 = load_q3_config(root, args.config)
355     params = Q2Parameters(terminal_mass_kg=q3.terminal_mass_min_kg)

```

```

356 reference_path = load_reference_path(str(root))
357
358 fixed = evaluate_path(np.zeros(4), q3=q3, params=params,
reference_path=reference_path, nodes=args.nodes)
359 variables, solution = solve_gate(q3, params, reference_path, args.nodes)
360 solver_status = "feasible" if solution.terminal_mass_slack_kg <= 1.0e-3 and
solution.max_constraint_violation <= 1.0e-6 else "needs_relaxation"
361 terminal = solution.trajectory.iloc[-1]
362 diagnostics = trajectory_diagnostics(solution.trajectory, q3)
363 summary = pd.DataFrame(
364     [
365         {
366             "wind_model": "no_wind",
367             "method": "range_domain_parameterized_feasibility_gate",
368             "nodes": args.nodes,
369             "solver_status": solver_status,
370             "target_distance_m": q3.target_distance_m,
371             "terminal_height_m": float(terminal["height_m"]),
372             "terminal_height_error_m": float(terminal["height_m"]) -
q3.terminal_height_m,
373             "terminal_speed_mps": float(terminal["airspeed_mps"]),
374             "terminal_speed_error_mps": float(terminal["airspeed_mps"]) -
q3.terminal_airspeed_mps,
375             "terminal_mass_kg": solution.final_mass_kg,
376             "terminal_mass_min_kg": q3.terminal_mass_min_kg,
377             "terminal_mass_shortfall_kg": solution.terminal_mass_slack_kg,
378             "fixed_path_mass_shortfall_kg": fixed_path_shortfall_kg(root,
fixed, q3),
379             "fuel_used_kg": params.m0_kg - solution.final_mass_kg,
380             "final_time_s": solution.final_time_s,
381             "max_nonrelaxed_constraint_violation":
solution.max_constraint_violation,
382             "integration_consistency_residual":
solution.max_abs_path_residual,
383             **diagnostics,
384             "h_sine_1_m": float(variables[0]),
385             "h_sine_2_m": float(variables[1]),
386             "v_sine_1_mps": float(variables[2]),
387             "v_sine_2_mps": float(variables[3]),
388         }
389     ]
390 )
391 qdir = root / "questions" / "q3"
392 save_table(summary, stem="no_wind_feasibility_gate", question_dir=qdir)

```



```

393     save_table(solution.trajectory, stem="no_wind_feasibility_trajectory",
question_dir=qdir)
394     print(summary.to_string(index=False))
395     return 0
396
397
398 if __name__ == "__main__":
399     raise SystemExit(main())

```

航程域配点法与降维控制打靶——solve_feasibility_collocation_no_wind.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import math
6  import sys
7  from dataclasses import dataclass, replace
8  from pathlib import Path
9
10 import numpy as np
11 import pandas as pd
12 import yaml
13 from scipy.integrate import solve_ivp
14 from scipy.optimize import minimize
15
16 ROOT = Path(__file__).resolve().parents[3]
17 SRC = ROOT / "src"
18 Q2_SCRIPTS = ROOT / "questions" / "q2" / "scripts"
19 Q3_SCRIPTS = ROOT / "questions" / "q3" / "scripts"
20 for path in [SRC, Q2_SCRIPTS, Q3_SCRIPTS]:
21     if str(path) not in sys.path:
22         sys.path.insert(0, str(path))
23
24 from fuel_path_model import Q2Parameters # noqa: E402
25 from fuel_path_model import atmosphere as layered_atmosphere # noqa: E402
26 from modeling_common.artifacts import save_table # noqa: E402
27 from modeling_common.paths import project_root # noqa: E402
28 from smooth_atmosphere import atmosphere, max_deviation_from_layered_isa,
smoothing_diagnostics_table # noqa: E402
29 from solve_feasibility_no_wind import Q3Config, load_q3_config # noqa: E402
30
31 H_SCALE_M = 10_000.0
32 V_SCALE_MPS = 240.0

```

```

33 M_SCALE_KG = 72_450.0
34 T_SCALE_N = 50_000.0
35 TIME_SCALE_S = 800.0
36 GAMMA_SCALE_RAD = 0.05
37 SLACK_SCALE_KG = 1_000.0
38 BASE_REINTEGRATION_ATOL = np.array([1.0e-5, 1.0e-7, 1.0e-4, 1.0e-6])
39 STATE_SCALES = np.array([H_SCALE_M, V_SCALE_MPS, M_SCALE_KG, TIME_SCALE_S])
40 SHOOTING_HEIGHT_MARGIN_M = 0.05
41
42
43 @dataclass(frozen=True)
44 class ShootingEvaluation:
45     trajectory: pd.DataFrame
46     final_height_error_m: float
47     final_speed_error_mps: float
48     final_mass_shortfall_kg: float
49     max_scaled_constraint_violation: float
50     fuel_used_kg: float
51     success: bool
52
53
54 @dataclass(frozen=True)
55 class ShootingOptimizerResult:
56     success: bool
57     message: str
58     nit: int
59     validation_success: bool
60
61
62 def parse_args() -> argparse.Namespace:
63     parser = argparse.ArgumentParser(description="q3 no-wind range-domain
64     collocation feasibility gate")
65     parser.add_argument("--config", default="configs/default.yaml")
66     parser.add_argument("--nodes", type=int, default=31)
67     parser.add_argument(
68         "--mesh-study-nodes",
69         default="",
70         help="comma-separated node counts for base-hmax reintegration mesh
71         convergence diagnostics",
72     )
73     parser.add_argument(
74         "--skip-hmax-sensitivity",
75         action="store_true",
76         help="skip the optimized h_max sensitivity sweep for focused formal or

```

```

mesh-study runs",
)
75
parser.add_argument(
76
    "--dry-run",
77
    action="store_true",
78
    help="project the Gate 1 trajectory into the Gate 2 model without
optimizing",
79
)
80
parser.add_argument(
81
    "--ode-rtols",
82
    default="",
83
    help="comma-separated ODE rtol values for reintegration tolerance and
continuous path audits",
84
)
85
parser.add_argument(
86
    "--final-fuel",
87
    action="store_true",
88
    help="solve the final no-wind fuel objective after Gate 2 feasibility",
89
)
90
parser.add_argument(
91
    "--continuation-nodes",
92
    default="",
93
    help="comma-separated node counts for final-fuel continuation; defaults
to --nodes",
94
)
95
parser.add_argument(
96
    "--initial-guess",
97
    choices=["gate2", "straight", "perturbed"],
98
    default="gate2",
99
    help="primary initial guess for final-fuel optimization",
100
)
101
parser.add_argument(
102
    "--multi-initial-guesses",
103
    default="gate2,straight,perturbed",
104
    help="comma-separated initial guesses for final validation at the last
node count",
105
)
106
parser.add_argument(
107
    "--final-solver",
108
    choices=["candidate", "slsqp", "shooting"],
109
    default="candidate",
110
    help=(
111
        "final-fuel solver mode; candidate evaluates Gate 2 continuation
quickly, "
112
    )

```

```

113         "slsqp solves full collocation NLP, shooting solves a reduced
continuous-control problem"
114     ),
115 )
116 parser.add_argument("--final-maxiter", type=int, default=220)
117 parser.add_argument(
118     "--shooting-control-knots",
119     type=int,
120     default=7,
121     help="number of reduced thrust/gamma control knots for --final-solver
shooting",
122 )
123 parser.add_argument(
124     "--final-hmax-sensitivity",
125     action="store_true",
126     help="re-optimize final no-wind fuel objective for configured h_max
sensitivity values",
127 )
128 parser.add_argument(
129     "--final-hmax-values",
130     default="",
131     help="comma-separated h_max values for --final-hmax-sensitivity; defaults
to config feasibility list",
132 )
133 parser.add_argument(
134     "--final-idle-thrust-sensitivity",
135     action="store_true",
136     help="re-optimize final no-wind fuel objective for idle thrust
lower-bound fractions",
137 )
138 parser.add_argument(
139     "--final-idle-thrust-fractions",
140     default="",
141     help=(
142         "comma-separated fractions of thrust_max_n used as thrust_min_n for "
143         "--final-idle-thrust-sensitivity; defaults to config feasibility list"
144     ),
145 )
146 return parser.parse_args()
147
148
149 def _parse_mesh_nodes(value: str) -> list[int]:
150     if not value.strip():
151         return []

```

```

152 nodes: list[int] = []
153 for token in value.split(","):
154     stripped = token.strip()
155     if not stripped:
156         continue
157     node_count = int(stripped)
158     if node_count < 5:
159         raise ValueError("--mesh-study-nodes entries must be at least 5")
160     if node_count not in nodes:
161         nodes.append(node_count)
162 return nodes
163
164
165 def _parse_ode_rtols(value: str) -> list[float]:
166     if not value.strip():
167         return []
168     rtols: list[float] = []
169     for token in value.split(","):
170         stripped = token.strip()
171         if not stripped:
172             continue
173         rtol = float(stripped)
174         if rtol <= 0.0:
175             raise ValueError("--ode-rtols entries must be positive")
176         if rtol not in rtols:
177             rtols.append(rtol)
178     return rtols
179
180
181 def _parse_float_list(value: str, *, option_name: str) -> list[float]:
182     if not value.strip():
183         return []
184     values: list[float] = []
185     for token in value.split(","):
186         stripped = token.strip()
187         if not stripped:
188             continue
189         number = float(stripped)
190         if not math.isfinite(number):
191             raise ValueError(f"{option_name} entries must be finite")
192         if number not in values:
193             values.append(number)
194     return values
195

```

```

196
197 def _parse_initial_guesses(value: str) -> list[str]:
198     if not value.strip():
199         return []
200     allowed = {"gate2", "straight", "perturbed"}
201     guesses: list[str] = []
202     for token in value.split(","):
203         stripped = token.strip()
204         if not stripped:
205             continue
206         if stripped not in allowed:
207             raise ValueError(f"unknown initial guess {stripped!r}; expected one
of {sorted(allowed)}")
208         if stripped not in guesses:
209             guesses.append(stripped)
210     return guesses
211
212
213 def _strict_reintegration_atol(rtol: float) -> np.ndarray:
214     return np.minimum(BASE_REINTEGRATION_ATOL, 0.1 * rtol * STATE_SCALES)
215
216
217 def _load_gate_config(root: Path, config_path: str) -> dict:
218     with (root / config_path).open("r", encoding="utf-8") as file:
219         config = yaml.safe_load(file)
220     return config["q3_optimal_control"]["feasibility_gate"]
221
222
223 def _idle_thrust_sensitivity_fractions(root: Path, config_path: str) ->
list[float]:
224     with (root / config_path).open("r", encoding="utf-8") as file:
225         config = yaml.safe_load(file)
226         gate = config["q3_optimal_control"]["feasibility_gate"]
227     return [float(value) for value in
gate.get("idle_thrust_sensitivity_fractions", [0.0, 0.05, 0.10])]
228
229
230 def _gate1_trajectory(root: Path) -> pd.DataFrame:
231     path = root / "questions" / "q3" / "artifacts" / "tables" /
"no_wind_feasibility_trajectory.csv"
232     if not path.exists():
233         raise FileNotFoundError("Run
questions/q3/scripts/solve_feasibility_no_wind.py before Gate 2 dry-run")
234     frame =

```

```

pd.read_csv(path).sort_values("distance_m").drop_duplicates("distance_m")
required = {"distance_m", "height_m", "airspeed_mps", "thrust_n", "gamma_rad"}
missing = required.difference(frame.columns)
if missing:
    raise ValueError(f"Gate 1 trajectory missing columns: {sorted(missing)}")
return frame

def _interp(frame: pd.DataFrame, column: str, distance: np.ndarray) -> np.ndarray:
    return np.interp(distance, frame["distance_m"].to_numpy(dtype=float),
        frame[column].to_numpy(dtype=float))

def _rates(
    *,
    height_m: float,
    airspeed_mps: float,
    mass_kg: float,
    thrust_n: float,
    gamma_rad: float,
    params: Q2Parameters,
    atmosphere_model=atmosphere,
) -> dict[str, float]:
    temperature_k, density_kgm3, sound_speed_mps, pressure_pa =
    atmosphere_model(height_m)
    cl = (
        2.0
        * mass_kg
        * params.g_mps2
        * math.cos(gamma_rad)
        / (density_kgm3 * airspeed_mps**2 * params.wing_area_m2)
    )
    cd = params.cd0 + params.induced_k * cl**2
    drag_n = 0.5 * density_kgm3 * airspeed_mps**2 * params.wing_area_m2 * cd
    ground_speed_mps = airspeed_mps * math.cos(gamma_rad)
    if ground_speed_mps <= 0.0:
        raise ValueError("non-positive range-domain denominator")
    fuel_penalty = 1.0 + params.beta_s2pm2 * (airspeed_mps - params.v_opt_mps) **
    2
    return {
        "temperature_k": temperature_k,
        "density_kgm3": density_kgm3,
        "sound_speed_mps": sound_speed_mps,
        "pressure_pa": pressure_pa,

```

```

275     "groundspeed_mps": ground_speed_mps,
276     "mach": airspeed_mps / sound_speed_mps,
277     "cl": cl,
278     "cd": cd,
279     "drag_n": drag_n,
280     "dh_dx": airspeed_mps * math.sin(gamma_rad) / ground_speed_mps,
281     "dV_dx": ((thrust_n - drag_n) / mass_kg - params.g_mps2 *
math.sin(gamma_rad)) / ground_speed_mps,
282     "dm_dx": -params.c_t_kg_per_ns * thrust_n * fuel_penalty /
ground_speed_mps,
283     "dt_dx": 1.0 / ground_speed_mps,
284 }
285
286
287 def _project_gate1_warm_start(
288     gate1: pd.DataFrame,
289     *,
290     q3: Q3Config,
291     params: Q2Parameters,
292     nodes: int,
293     atmosphere_model=atmosphere,
294 ) -> pd.DataFrame:
295     distance = np.linspace(0.0, q3.target_distance_m, nodes)
296     height = _interp(gate1, "height_m", distance)
297     airspeed = _interp(gate1, "airspeed_mps", distance)
298     thrust = _interp(gate1, "thrust_n", distance)
299     gamma = _interp(gate1, "gamma_rad", distance)
300     height[0] = 9500.0
301     airspeed[0] = q3.terminal_airspeed_mps
302     height[-1] = q3.terminal_height_m
303     airspeed[-1] = q3.terminal_airspeed_mps
304
305     mass = np.zeros(nodes)
306     time = np.zeros(nodes)
307     mass[0] = params.m0_kg
308     records: list[dict[str, float]] = []
309
310     for index in range(nodes):
311         rates = _rates(
312             height_m=float(height[index]),
313             airspeed_mps=float(airspeed[index]),
314             mass_kg=float(mass[index]) if index == 0 else float(mass[index]),
315             thrust_n=float(thrust[index]),
316             gamma_rad=float(gamma[index]),

```



```

317         params=params,
318         atmosphere_model=atmosphere_model,
319     )
320     records.append(
321         {
322             "distance_m": float(distance[index]),
323             "height_m": float(height[index]),
324             "airspeed_mps": float(airspeed[index]),
325             "mass_kg": float(mass[index]),
326             "time_s": float(time[index]),
327             "thrust_n": float(thrust[index]),
328             "gamma_rad": float(gamma[index]),
329             **rates,
330         }
331     )
332     if index == nodes - 1:
333         break
334     dx = distance[index + 1] - distance[index]
335     rates_next_guess = _rates(
336         height_m=float(height[index + 1]),
337         airspeed_mps=float(airspeed[index + 1]),
338         mass_kg=float(mass[index]),
339         thrust_n=float(thrust[index + 1]),
340         gamma_rad=float(gamma[index + 1]),
341         params=params,
342         atmosphere_model=atmosphere_model,
343     )
344     mass[index + 1] = mass[index] + 0.5 * dx * (rates["dm_dx"] +
345 rates_next_guess["dm_dx"])
346     time[index + 1] = time[index] + 0.5 * dx * (rates["dt_dx"] +
347 rates_next_guess["dt_dx"])
348     frame = pd.DataFrame.from_records(records)
349     frame["mass_kg"] = mass
350     frame["time_s"] = time
351     return frame
352
353 def _scaled_constraint_violation(row: pd.Series, q3: Q3Config) -> float:
354     height_scale = 10_000.0
355     speed_scale = 240.0
356     thrust_scale = max(q3.thrust_max_n, 1.0)
357     gamma_scale = max(q3.gamma_max_rad, 1.0e-9)
358     return max(

```

```

359         (q3.h_min_m - row["height_m"]) / height_scale,
360         (row["height_m"] - q3.h_max_m) / height_scale,
361         (q3.v_min_mps - row["airspeed_mps"]) / speed_scale,
362         (row["airspeed_mps"] - q3.v_max_mps) / speed_scale,
363         (q3.thrust_min_n - row["thrust_n"]) / thrust_scale,
364         (row["thrust_n"] - q3.thrust_max_n) / thrust_scale,
365         (abs(row["gamma_rad"]) - q3.gamma_max_rad) / gamma_scale,
366         row["mach"] - q3.mach_max,
367     )
368
369
370 def _collocation_diagnostics(trajectory: pd.DataFrame, q3: Q3Config) -> dict[str,
float]:
371     x = trajectory["distance_m"].to_numpy(dtype=float)
372     h = trajectory["height_m"].to_numpy(dtype=float)
373     v = trajectory["airspeed_mps"].to_numpy(dtype=float)
374     m = trajectory["mass_kg"].to_numpy(dtype=float)
375     t = trajectory["time_s"].to_numpy(dtype=float)
376     rates = trajectory[["dh_dx", "dV_dx", "dm_dx", "dt_dx"]].to_numpy(dtype=float)
377     defect_max = 0.0
378     for index in range(len(x) - 1):
379         dx = x[index + 1] - x[index]
380         defects = np.array(
381             [
382                 h[index + 1] - h[index] - 0.5 * dx * (rates[index, 0] +
rates[index + 1, 0]),
383                 v[index + 1] - v[index] - 0.5 * dx * (rates[index, 1] +
rates[index + 1, 1]),
384                 m[index + 1] - m[index] - 0.5 * dx * (rates[index, 2] +
rates[index + 1, 2]),
385                 t[index + 1] - t[index] - 0.5 * dx * (rates[index, 3] +
rates[index + 1, 3]),
386             ]
387         )
388         scaled = np.array([defects[0] / 10_000.0, defects[1] / 240.0, defects[2]
/ 72_450.0, defects[3] / 800.0])
389         defect_max = max(defect_max, float(np.max(np.abs(scaled))))
390         midpoint_height = 0.5 * (h[:-1] + h[1:])
391         midpoint_violation = np.maximum(0.0, midpoint_height - q3.h_max_m)
392     return {
393         "scaled_collocation_defect_inf": defect_max,
394         "max_midpoint_height_violation_m": float(np.max(midpoint_violation)) if
len(midpoint_violation) else 0.0,
395     }

```

```

396
397
398 def _state_rates_for_arrays(
399     height_m: np.ndarray,
400     airspeed_mps: np.ndarray,
401     mass_kg: np.ndarray,
402     thrust_n: np.ndarray,
403     gamma_rad: np.ndarray,
404     params: Q2Parameters,
405 ) -> pd.DataFrame:
406     records: list[dict[str, float]] = []
407     for index in range(len(height_m)):
408         records.append(
409             _rates(
410                 height_m=float(height_m[index]),
411                 airspeed_mps=float(airspeed_mps[index]),
412                 mass_kg=float(mass_kg[index]),
413                 thrust_n=float(thrust_n[index]),
414                 gamma_rad=float(gamma_rad[index]),
415                 params=params,
416                 atmosphere_model=atmosphere,
417             )
418         )
419     return pd.DataFrame.from_records(records)
420
421
422 def _pack_decision(
423     *,
424     height_m: np.ndarray,
425     airspeed_mps: np.ndarray,
426     mass_kg: np.ndarray,
427     time_s: np.ndarray,
428     thrust_n: np.ndarray,
429     gamma_rad: np.ndarray,
430     slack_kg: float,
431 ) -> np.ndarray:
432     return np.concatenate(
433         [
434             height_m / H_SCALE_M,
435             airspeed_mps / V_SCALE_MPS,
436             mass_kg / M_SCALE_KG,
437             time_s / TIME_SCALE_S,
438             thrust_n / T_SCALE_N,
439             gamma_rad / GAMMA_SCALE_RAD,

```

```

440         np.array([slack_kg / SLACK_SCALE_KG]),
441     ]
442 )
443
444
445 def _unpack_decision(vector: np.ndarray, nodes: int) -> dict[str, np.ndarray |
float]:
446     cursor = 0
447
448     def take(scale: float) -> np.ndarray:
449         nonlocal cursor
450         values = vector[cursor : cursor + nodes] * scale
451         cursor += nodes
452         return values
453
454     return {
455         "height_m": take(H_SCALE_M),
456         "airspeed_mps": take(V_SCALE_MPS),
457         "mass_kg": take(M_SCALE_KG),
458         "time_s": take(TIME_SCALE_S),
459         "thrust_n": take(T_SCALE_N),
460         "gamma_rad": take(GAMMA_SCALE_RAD),
461         "slack_kg": float(vector[cursor] * SLACK_SCALE_KG),
462     }
463
464
465 def _decision_to_trajectory(vector: np.ndarray, *, nodes: int, q3: Q3Config,
params: Q2Parameters) -> pd.DataFrame:
466     values = _unpack_decision(vector, nodes)
467     distance = np.linspace(0.0, q3.target_distance_m, nodes)
468     rates = _state_rates_for_arrays(
469         values["height_m"],
470         values["airspeed_mps"],
471         values["mass_kg"],
472         values["thrust_n"],
473         values["gamma_rad"],
474         params,
475     )
476     trajectory = pd.DataFrame(
477         {
478             "distance_m": distance,
479             "height_m": values["height_m"],
480             "airspeed_mps": values["airspeed_mps"],
481             "mass_kg": values["mass_kg"],

```

```

482         "time_s": values["time_s"],
483         "thrust_n": values["thrust_n"],
484         "gamma_rad": values["gamma_rad"],
485     }
486 )
487 return pd.concat([trajectory, rates], axis=1)
488
489
490 def _formal_collocation_diagnostics(trajectory: pd.DataFrame, q3: Q3Config) ->
dict[str, float]:
491     diagnostics = _collocation_diagnostics(trajectory, q3)
492     height = trajectory["height_m"].to_numpy(dtype=float)
493     max_reconstruction_violation = 0.0
494     for index in range(len(height) - 1):
495         samples = np.linspace(height[index], height[index + 1], 7)
496         max_reconstruction_violation = max(
497             max_reconstruction_violation,
498             float(np.max(np.maximum(0.0, samples - q3.h_max_m))),
499         )
500     diagnostics["max_reconstruction_height_violation_m"] =
max_reconstruction_violation
501     return diagnostics
502
503
504 def _collocation_equalities(vector: np.ndarray, *, nodes: int, q3: Q3Config,
params: Q2Parameters) -> np.ndarray:
505     values = _unpack_decision(vector, nodes)
506     rates = _state_rates_for_arrays(
507         values["height_m"],
508         values["airspeed_mps"],
509         values["mass_kg"],
510         values["thrust_n"],
511         values["gamma_rad"],
512         params,
513     )["dh_dx", "dV_dx", "dm_dx", "dt_dx"].to_numpy(dtype=float)
514     distance = np.linspace(0.0, q3.target_distance_m, nodes)
515     equalities = [
516         (values["height_m"][0] - 9500.0) / H_SCALE_M,
517         (values["airspeed_mps"][0] - q3.terminal_airspeed_mps) / V_SCALE_MPS,
518         (values["mass_kg"][0] - params.m0_kg) / M_SCALE_KG,
519         values["time_s"][0] / TIME_SCALE_S,
520         (values["height_m"][-1] - q3.terminal_height_m) / H_SCALE_M,
521         (values["airspeed_mps"][-1] - q3.terminal_airspeed_mps) / V_SCALE_MPS,
522     ]

```

```

523     state_values = np.column_stack(
524         [values["height_m"], values["airspeed_mps"], values["mass_kg"],
values["time_s"]]
525     )
526     scales = np.array([H_SCALE_M, V_SCALE_MPS, M_SCALE_KG, TIME_SCALE_S])
527     for index in range(nodes - 1):
528         dx = distance[index + 1] - distance[index]
529         defect = state_values[index + 1] - state_values[index] - 0.5 * dx *
(rates[index] + rates[index + 1])
530         equalities.extend((defect / scales).tolist())
531     return np.asarray(equalities, dtype=float)
532
533
534 def _collocation_inequalities(vector: np.ndarray, *, nodes: int, q3: Q3Config,
params: Q2Parameters) -> np.ndarray:
535     values = _unpack_decision(vector, nodes)
536     rates = _state_rates_for_arrays(
537         values["height_m"],
538         values["airspeed_mps"],
539         values["mass_kg"],
540         values["thrust_n"],
541         values["gamma_rad"],
542         params,
543     )
544     inequalities: list[float] = [
545         values["mass_kg"][-1] + values["slack_kg"] - q3.terminal_mass_min_kg,
546         values["slack_kg"],
547     ]
548     inequalities.extend((q3.mach_max -
rates["mach"].to_numpy(dtype=float)).tolist())
549     midpoint_height = 0.5 * (values["height_m"][:-1] + values["height_m"][1:])
550     inequalities.extend((q3.h_max_m - midpoint_height).tolist())
551     return np.asarray(inequalities, dtype=float)
552
553
554 def _decision_bounds(
555     nodes: int,
556     q3: Q3Config,
557     *,
558     slack_bounds_kg: tuple[float, float] = (0.0, 2_000.0),
559 ) -> list[tuple[float, float]]:
560     return (
561         [(q3.h_min_m / H_SCALE_M, q3.h_max_m / H_SCALE_M)] * nodes
562         + [(q3.v_min_mps / V_SCALE_MPS, q3.v_max_mps / V_SCALE_MPS)] * nodes

```

```

563         + [(55_000.0 / M_SCALE_KG, 73_000.0 / M_SCALE_KG)] * nodes
564         + [(0.0, 2_000.0 / TIME_SCALE_S)] * nodes
565         + [(q3.thrust_min_n / T_SCALE_N, q3.thrust_max_n / T_SCALE_N)] * nodes
566         + [(-q3.gamma_max_rad / GAMMA_SCALE_RAD, q3.gamma_max_rad /
567 GAMMA_SCALE_RAD)] * nodes
568         + [(slack_bounds_kg[0] / SLACK_SCALE_KG, slack_bounds_kg[1] /
569 SLACK_SCALE_KG)]
570     )
571 def _initial_decision_from_warm_start(
572     gate1: pd.DataFrame,
573     *,
574     q3: Q3Config,
575     params: Q2Parameters,
576     nodes: int,
577 ) -> np.ndarray:
578     warm = _project_gate1_warm_start(gate1, q3=q3, params=params, nodes=nodes)
579     slack = max(0.0, q3.terminal_mass_min_kg - float(warm["mass_kg"].iloc[-1]))
580     return _pack_decision(
581         height_m=warm["height_m"].to_numpy(dtype=float),
582         airspeed_mps=warm["airspeed_mps"].to_numpy(dtype=float),
583         mass_kg=warm["mass_kg"].to_numpy(dtype=float),
584         time_s=warm["time_s"].to_numpy(dtype=float),
585         thrust_n=warm["thrust_n"].to_numpy(dtype=float),
586         gamma_rad=warm["gamma_rad"].to_numpy(dtype=float),
587         slack_kg=slack,
588     )
589
590
591 def _resample_trajectory_decision(
592     trajectory: pd.DataFrame,
593     *,
594     q3: Q3Config,
595     nodes: int,
596     slack_kg: float = 0.0,
597 ) -> np.ndarray:
598     source = trajectory.sort_values("distance_m").drop_duplicates("distance_m")
599     distance = np.linspace(0.0, q3.target_distance_m, nodes)
600     height = _interp(source, "height_m", distance)
601     airspeed = _interp(source, "airspeed_mps", distance)
602     mass = _interp(source, "mass_kg", distance)
603     time = _interp(source, "time_s", distance)
604     thrust = _interp(source, "thrust_n", distance)

```

```

605     gamma = _interp(source, "gamma_rad", distance)
606     height[0] = 9500.0
607     airspeed[0] = q3.terminal_airspeed_mps
608     mass[0] = M_SCALE_KG
609     time[0] = 0.0
610     height[-1] = q3.terminal_height_m
611     airspeed[-1] = q3.terminal_airspeed_mps
612     mass = np.maximum(mass, q3.terminal_mass_min_kg)
613     return _pack_decision(
614         height_m=height,
615         airspeed_mps=airspeed,
616         mass_kg=mass,
617         time_s=time,
618         thrust_n=thrust,
619         gamma_rad=gamma,
620         slack_kg=slack_kg,
621     )
622
623
624 def _straight_initial_decision(*, q3: Q3Config, params: Q2Parameters, nodes: int)
-> np.ndarray:
625     distance = np.linspace(0.0, q3.target_distance_m, nodes)
626     height = np.linspace(9500.0, q3.terminal_height_m, nodes)
627     airspeed = np.full(nodes, q3.terminal_airspeed_mps)
628     gamma = np.arctan2(np.gradient(height, distance, edge_order=1), 1.0)
629     thrust = np.zeros(nodes)
630     mass = np.zeros(nodes)
631     time = np.zeros(nodes)
632     mass[0] = params.m0_kg
633     for index in range(nodes):
634         rates = _rates(
635             height_m=float(height[index]),
636             airspeed_mps=float(airspeed[index]),
637             mass_kg=float(mass[index]) if mass[index] > 0.0 else params.m0_kg,
638             thrust_n=float(thrust[index]) if thrust[index] > 0.0 else 50_000.0,
639             gamma_rad=float(gamma[index]),
640             params=params,
641             atmosphere_model=atmosphere,
642         )
643         drag = rates["drag_n"]
644         required = (drag + mass[index] * params.g_mps2 *
645             math.sin(float(gamma[index]))) if mass[index] > 0.0 else drag
646         thrust[index] = float(np.clip(required, q3.thrust_min_n, q3.thrust_max_n))
        if index == nodes - 1:

```



```

647         break
648     dx = distance[index + 1] - distance[index]
649     rates_now = _rates(
650         height_m=float(height[index]),
651         airspeed_mps=float(airspeed[index]),
652         mass_kg=float(mass[index]),
653         thrust_n=float(thrust[index]),
654         gamma_rad=float(gamma[index]),
655         params=params,
656         atmosphere_model=atmosphere,
657     )
658     mass[index + 1] = max(q3.terminal_mass_min_kg, mass[index] + dx *
rates_now["dm_dx"])
659     time[index + 1] = time[index] + dx * rates_now["dt_dx"]
660     return _pack_decision(
661         height_m=height,
662         airspeed_mps=airspeed,
663         mass_kg=mass,
664         time_s=time,
665         thrust_n=thrust,
666         gamma_rad=gamma,
667         slack_kg=0.0,
668     )
669
670
671 def _perturb_decision(vector: np.ndarray, *, nodes: int, q3: Q3Config) ->
np.ndarray:
672     values = _unpack_decision(vector.copy(), nodes)
673     phase = np.linspace(0.0, math.pi, nodes)
674     height = np.clip(values["height_m"] + 80.0 * np.sin(phase), q3.h_min_m,
q3.h_max_m)
675     airspeed = np.clip(values["airspeed_mps"] + 0.8 * np.sin(2.0 * phase),
q3.v_min_mps, q3.v_max_mps)
676     thrust = np.clip(values["thrust_n"] * (1.0 + 0.015 * np.sin(phase)),
q3.thrust_min_n, q3.thrust_max_n)
677     gamma = np.clip(values["gamma_rad"] + 2.0e-4 * np.sin(2.0 * phase),
-q3.gamma_max_rad, q3.gamma_max_rad)
678     return _pack_decision(
679         height_m=height,
680         airspeed_mps=airspeed,
681         mass_kg=values["mass_kg"],
682         time_s=values["time_s"],
683         thrust_n=thrust,
684         gamma_rad=gamma,

```

```

685         slack_kg=0.0,
686     )
687
688
689 def _solve_stage1_collocation(
690     gate1: pd.DataFrame,
691     *,
692     q3: Q3Config,
693     params: Q2Parameters,
694     nodes: int,
695     maxiter: int = 180,
696 ) -> tuple[np.ndarray, object]:
697     initial = _initial_decision_from_warm_start(gate1, q3=q3, params=params,
698         nodes=nodes)
699
700     def objective(vector: np.ndarray) -> float:
701         return float(_unpack_decision(vector, nodes)["slack_kg"]) / SLACK_SCALE_KG
702
703     constraints = [
704         {
705             "type": "eq",
706             "fun": lambda vector: _collocation_equalities(vector, nodes=nodes,
707                 q3=q3, params=params),
708         },
709         {
710             "type": "ineq",
711             "fun": lambda vector: _collocation_inequalities(vector, nodes=nodes,
712                 q3=q3, params=params),
713         },
714     ]
715
716     result = minimize(
717         objective,
718         initial,
719         method="SLSQP",
720         bounds=_decision_bounds(nodes, q3),
721         constraints=constraints,
722         options={"maxiter": maxiter, "ftol": 1.0e-9, "disp": False},
723     )
724
725     vector = np.asarray(result.x if hasattr(result, "x") else initial,
726         dtype=float)
727     return vector, result
728
729 def _final_fuel_equalities(vector: np.ndarray, *, nodes: int, q3: Q3Config,

```

```

params: Q2Parameters) -> np.ndarray:
725     return _collocation_equalities(vector, nodes=nodes, q3=q3, params=params)
726
727
728 def _final_fuel_inequalities(vector: np.ndarray, *, nodes: int, q3: Q3Config,
params: Q2Parameters) -> np.ndarray:
729     values = _unpack_decision(vector, nodes)
730     rates = _state_rates_for_arrays(
731         values["height_m"],
732         values["airspeed_mps"],
733         values["mass_kg"],
734         values["thrust_n"],
735         values["gamma_rad"],
736         params,
737     )
738     inequalities: list[float] = [
739         values["mass_kg"][-1] - q3.terminal_mass_min_kg,
740     ]
741     inequalities.extend((values["mass_kg"] - q3.terminal_mass_min_kg).tolist())
742     inequalities.extend((q3.mach_max -
rates["mach"].to_numpy(dtype=float)).tolist())
743     midpoint_height = 0.5 * (values["height_m"][:-1] + values["height_m"][1:])
744     inequalities.extend((q3.h_max_m - midpoint_height).tolist())
745     return np.asarray(inequalities, dtype=float)
746
747
748 def _solve_final_fuel_collocation(
749     initial: np.ndarray,
750     *,
751     q3: Q3Config,
752     params: Q2Parameters,
753     nodes: int,
754     maxiter: int,
755 ) -> tuple[np.ndarray, object]:
756     def objective(vector: np.ndarray) -> float:
757         values = _unpack_decision(vector, nodes)
758         return float((params.m0_kg - values["mass_kg"][-1]) / SLACK_SCALE_KG)
759
760     constraints = [
761         {
762             "type": "eq",
763             "fun": lambda vector: _final_fuel_equalities(vector, nodes=nodes,
q3=q3, params=params),
764         },

```

```

765     {
766         "type": "ineq",
767         "fun": lambda vector: _final_fuel_inequalities(vector, nodes=nodes,
768         q3=q3, params=params),
769     },
770 ]
771 result = minimize(
772     objective,
773     initial,
774     method="SLSQP",
775     bounds=_decision_bounds(nodes, q3, slack_bounds_kg=(0.0, 0.0)),
776     constraints=constraints,
777     options={"maxiter": maxiter, "ftol": 1.0e-9, "disp": False},
778 )
779 vector = np.asarray(result.x if hasattr(result, "x") else initial,
780 dtype=float)
781 return vector, result
782
783 def _reintegration_diagnostics(
784     trajectory: pd.DataFrame,
785     *,
786     q3: Q3Config,
787     params: Q2Parameters,
788     rtol: float = 1.0e-8,
789     atol: np.ndarray | None = None,
790 ) -> dict[str, float | str]:
791     result = _reintegration_result(trajectory, q3=q3, params=params, rtol=rtol,
792     atol=atol)
793     if not bool(result["success"]):
794         return {
795             "control_reconstruction": "piecewise_linear_node_controls",
796             "reintegration_state_error_inf": math.inf,
797             "reintegration_terminal_mass_kg": math.inf,
798             "reintegration_terminal_mass_signed_error_kg": math.inf,
799             "reintegration_terminal_mass_error_kg": math.inf,
800             "reintegration_terminal_mass_shortfall_kg": math.inf,
801             "reintegration_terminal_height_signed_error_m": math.inf,
802             "reintegration_terminal_height_error_m": math.inf,
803             "reintegration_terminal_speed_signed_error_mps": math.inf,
804             "reintegration_terminal_speed_error_mps": math.inf,
805             "reintegration_max_scaled_constraint_violation": math.inf,
806             "reintegration_max_node_speed_signed_error_mps": math.inf,
807             "reintegration_max_node_speed_error_mps": math.inf,

```

```

806         "reintegration_max_node_mass_signed_error_kg": math.inf,
807         "reintegration_max_node_mass_error_kg": math.inf,
808     }
809     return {
810         "control_reconstruction": "piecewise_linear_node_controls",
811         "reintegration_state_error_inf": float(result["max_scaled_state_error"]),
812         "reintegration_terminal_mass_kg": float(result["terminal_mass_kg"]),
813         "reintegration_terminal_mass_signed_error_kg":
814         float(result["terminal_mass_signed_error_kg"]),
815         "reintegration_terminal_mass_error_kg":
816         float(abs(result["terminal_mass_signed_error_kg"])),
817         "reintegration_terminal_mass_shortfall_kg":
818         float(result["terminal_mass_shortfall_kg"]),
819         "reintegration_terminal_height_signed_error_m":
820         float(result["terminal_height_signed_error_m"]),
821         "reintegration_terminal_height_error_m":
822         float(abs(result["terminal_height_signed_error_m"])),
823         "reintegration_terminal_speed_signed_error_mps":
824         float(result["terminal_speed_signed_error_mps"]),
825         "reintegration_terminal_speed_error_mps":
826         float(abs(result["terminal_speed_signed_error_mps"])),
827         "reintegration_max_scaled_constraint_violation":
828         float(result["max_continuous_scaled_constraint_violation"]),
829         "reintegration_max_node_speed_signed_error_mps":
830         float(result["max_node_speed_signed_error_mps"]),
831         "reintegration_max_node_speed_error_mps":
832         float(abs(result["max_node_speed_signed_error_mps"])),
833         "reintegration_max_node_mass_signed_error_kg":
834         float(result["max_node_mass_signed_error_kg"]),
835         "reintegration_max_node_mass_error_kg":
836         float(abs(result["max_node_mass_signed_error_kg"])),
837     }
838
839 def _reintegration_result(
840     trajectory: pd.DataFrame,
841     *,
842     q3: Q3Config,
843     params: Q2Parameters,
844     rtol: float,
845     atol: np.ndarray | None = None,
846 ) -> dict[str, float | bool]:
847     distance = trajectory["distance_m"].to_numpy(dtype=float)
848     thrust = trajectory["thrust_n"].to_numpy(dtype=float)

```

```

838 gamma = trajectory["gamma_rad"].to_numpy(dtype=float)
839 atol = BASE_REINTEGRATION_ATOL if atol is None else np.asarray(atol,
dtype=float)
840
841 def rhs(distance_m: float, state: np.ndarray) -> list[float]:
842     thrust_n = float(np.interp(distance_m, distance, thrust))
843     gamma_rad = float(np.interp(distance_m, distance, gamma))
844     rates = _rates(
845         height_m=float(state[0]),
846         airspeed_mps=float(state[1]),
847         mass_kg=float(state[2]),
848         thrust_n=thrust_n,
849         gamma_rad=gamma_rad,
850         params=params,
851         atmosphere_model=atmosphere,
852     )
853     return [rates["dh_dx"], rates["dV_dx"], rates["dm_dx"], rates["dt_dx"]]
854
855 solution = solve_ivp(
856     rhs,
857     (float(distance[0]), float(distance[-1])),
858     np.array([9500.0, q3.terminal_airspeed_mps, params.m0_kg, 0.0]),
859     t_eval=distance,
860     dense_output=True,
861     rtol=rtol,
862     atol=atol,
863     max_step=q3.target_distance_m / max(len(distance) - 1, 1) / 4.0,
864 )
865 if not solution.success:
866     return {
867         "success": False,
868         "terminal_mass_kg": math.inf,
869         "terminal_mass_signed_error_kg": math.inf,
870         "terminal_mass_shortfall_kg": math.inf,
871         "terminal_height_signed_error_m": math.inf,
872         "terminal_speed_signed_error_mps": math.inf,
873         "max_height_error_m": math.inf,
874         "max_speed_error_mps": math.inf,
875         "max_mass_error_kg": math.inf,
876         "max_time_error_s": math.inf,
877         "max_scaled_state_error": math.inf,
878         "max_continuous_scaled_constraint_violation": math.inf,
879         "max_height_violation_m": math.inf,
880         "max_speed_violation_mps": math.inf,

```

```

881         "max_mach_violation": math.inf,
882         "max_thrust_violation_n": math.inf,
883         "max_gamma_violation_rad": math.inf,
884         "max_node_speed_signed_error_mps": math.inf,
885         "max_node_mass_signed_error_kg": math.inf,
886     }
887     collocation_states = trajectory[["height_m", "airspeed_mps", "mass_kg",
888                                     "time_s"]].to_numpy(dtype=float).T
889     errors = solution.y - collocation_states
890     scaled = errors / STATE_SCALES[:, None]
891     dense_distance = np.linspace(float(distance[0]), float(distance[-1]), max(4 *
892                                     (len(distance) - 1) + 1, len(distance)))
893     dense_states = solution.sol(dense_distance) if solution.sol is not None else
894     np.vstack(
895         [np.interp(dense_distance, solution.t, solution.y[index]) for index in
896          range(solution.y.shape[0])]
897     )
898     dense_collocation_states = np.vstack(
899         [
900             np.interp(dense_distance, distance, collocation_states[index])
901             for index in range(collocation_states.shape[0])
902         ]
903     )
904     dense_errors = dense_states - dense_collocation_states
905     dense_violations: list[float] = []
906     height_violations: list[float] = []
907     speed_violations: list[float] = []
908     mach_violations: list[float] = []
909     thrust_violations: list[float] = []
910     gamma_violations: list[float] = []
911     for index, distance_m in enumerate(dense_distance):
912         thrust_n = float(np.interp(distance_m, distance, thrust))
913         gamma_rad = float(np.interp(distance_m, distance, gamma))
914         rates = _rates(
915             height_m=float(dense_states[0, index]),
916             airspeed_mps=float(dense_states[1, index]),
917             mass_kg=float(dense_states[2, index]),
918             thrust_n=thrust_n,
919             gamma_rad=gamma_rad,
920             params=params,
921             atmosphere_model=atmosphere,
922         )
923         height_m = float(dense_states[0, index])
924         speed_mps = float(dense_states[1, index])

```

```

921     height_violation = max(0.0, q3.h_min_m - height_m, height_m - q3.h_max_m)
922     speed_violation = max(0.0, q3.v_min_mps - speed_mps, speed_mps -
q3.v_max_mps)
923     mach_violation = max(0.0, rates["mach"] - q3.mach_max)
924     thrust_violation = max(0.0, q3.thrust_min_n - thrust_n, thrust_n -
q3.thrust_max_n)
925     gamma_violation = max(0.0, abs(gamma_rad) - q3.gamma_max_rad)
926     height_violations.append(height_violation)
927     speed_violations.append(speed_violation)
928     mach_violations.append(mach_violation)
929     thrust_violations.append(thrust_violation)
930     gamma_violations.append(gamma_violation)
931     dense_violations.append(
932         max(
933             height_violation / H_SCALE_M,
934             speed_violation / V_SCALE_MPS,
935             max(0.0, q3.terminal_mass_min_kg - float(dense_states[2, index]))
/ M_SCALE_KG,
936             thrust_violation / max(q3.thrust_max_n, 1.0),
937             gamma_violation / max(q3.gamma_max_rad, 1.0e-9),
938             mach_violation,
939         )
940     )
941     reintegrated_h = float(solution.y[0, -1])
942     reintegrated_v = float(solution.y[1, -1])
943     reintegrated_m = float(solution.y[2, -1])
944     collocation_final_mass = float(collocation_states[2, -1])
945     node_speed_errors = errors[1]
946     node_mass_errors = errors[2]
947     max_speed_index = int(np.argmax(np.abs(node_speed_errors)))
948     max_mass_index = int(np.argmax(np.abs(node_mass_errors)))
949     return {
950         "success": True,
951         "terminal_mass_kg": reintegrated_m,
952         "terminal_mass_signed_error_kg": reintegrated_m - collocation_final_mass,
953         "terminal_mass_shortfall_kg": max(0.0, q3.terminal_mass_min_kg -
reintegrated_m),
954         "terminal_height_signed_error_m": reintegrated_h - q3.terminal_height_m,
955         "terminal_speed_signed_error_mps": reintegrated_v -
q3.terminal_airspeed_mps,
956         "max_height_error_m": float(np.max(np.abs(dense_errors[0]))),
957         "max_speed_error_mps": float(np.max(np.abs(dense_errors[1]))),
958         "max_mass_error_kg": float(np.max(np.abs(dense_errors[2]))),
959         "max_time_error_s": float(np.max(np.abs(dense_errors[3]))),

```



```

960     "max_scaled_state_error": float(np.max(np.abs(dense_errors /
STATE_SCALES[:, None]))),
961     "max_continuous_scaled_constraint_violation":
float(max(dense_violations)) if dense_violations else 0.0,
962     "max_height_violation_m": float(max(height_violations)) if
height_violations else 0.0,
963     "max_speed_violation_mps": float(max(speed_violations)) if
speed_violations else 0.0,
964     "max_mach_violation": float(max(mach_violations)) if mach_violations else
0.0,
965     "max_thrust_violation_n": float(max(thrust_violations)) if
thrust_violations else 0.0,
966     "max_gamma_violation_rad": float(max(gamma_violations)) if
gamma_violations else 0.0,
967     "max_node_speed_signed_error_mps":
float(node_speed_errors[max_speed_index]),
968     "max_node_mass_signed_error_kg": float(node_mass_errors[max_mass_index]),
969 }
970
971
972 def _formal_solver_status(
973     *,
974     result: object,
975     slack_kg: float,
976     diagnostics: dict[str, float],
977     reintegration: dict[str, float | str],
978     max_scaled_constraint_violation: float,
979     q3: Q3Config,
980     gate_config: dict,
981 ) -> str:
982     if not getattr(result, "success", False):
983         return "optimization_failed"
984     criteria = gate_config["pass_criteria"]
985     discrete_feasible = (
986         slack_kg <= float(criteria["terminal_mass_shortfall_kg"])
987         and diagnostics["scaled_collocation_defect_inf"] <=
float(criteria["scaled_collocation_defect_inf"])
988         and diagnostics["max_midpoint_height_violation_m"] <= 1.0e-9
989         and max_scaled_constraint_violation <=
float(criteria["constraint_violation_tolerance"])
990     )
991     reintegration_feasible = (
992         float(reintegration["reintegration_terminal_mass_shortfall_kg"])
993         <= float(criteria["terminal_mass_shortfall_kg"])

```

```

994         and float(reintegration["reintegration_terminal_height_error_m"]) <=
float(criteria["terminal_height_error_m"])
995         and float(reintegration["reintegration_terminal_speed_error_mps"]) <=
float(criteria["terminal_airspeed_error_mps"])
996         and float(reintegration["reintegration_max_scaled_constraint_violation"])
<= float(criteria["constraint_violation_tolerance"])
997     )
998
999     if discrete_feasible and reintegration_feasible:
1000         return "gate2_feasible"
1001     if discrete_feasible:
1002         return "discrete_feasible_reintegration_failed"
1003     return "needs_relaxation"
1004
1005
1006 def _control_variation_metrics(trajectory: pd.DataFrame) -> dict[str, float]:
1007     thrust = trajectory["thrust_n"].to_numpy(dtype=float)
1008     gamma = trajectory["gamma_rad"].to_numpy(dtype=float)
1009     thrust_steps = np.abs(np.diff(thrust))
1010     gamma_steps = np.abs(np.diff(gamma))
1011     return {
1012         "max_thrust_step_n": float(np.max(thrust_steps)) if len(thrust_steps)
else 0.0,
1013         "max_gamma_step_rad": float(np.max(gamma_steps)) if len(gamma_steps) else
0.0,
1014         "total_variation_thrust_n": float(np.sum(thrust_steps)),
1015         "total_variation_gamma_rad": float(np.sum(gamma_steps)),
1016     }
1017
1018
1019 def _formal_gate_case(
1020     gate1: pd.DataFrame,
1021     *,
1022     q3: Q3Config,
1023     params: Q2Parameters,
1024     nodes: int,
1025     gate_config: dict,
1026     maxiter: int = 180,
1027 ) -> tuple[pd.DataFrame, dict[str, float | str | bool]]:
1028     vector, optimizer_result = _solve_stage1_collocation(
1029         gate1,
1030         q3=q3,
1031         params=params,
1032         nodes=nodes,
1033         maxiter=maxiter,

```

```

1034 )
1035 trajectory = _decision_to_trajectory(vector, nodes=nodes, q3=q3,
1036 params=params)
1037 trajectory["scaled_constraint_violation"] =
1038 trajectory.apply(_scaled_constraint_violation, axis=1, q3=q3)
1039 values = _unpack_decision(vector, nodes)
1040 diagnostics = _formal_collocation_diagnostics(trajectory, q3)
1041 reintegration = _reintegration_diagnostics(trajectory, q3=q3, params=params)
1042 max_scaled_violation = float(trajectory["scaled_constraint_violation"].max())
1043 slack_kg = max(0.0, float(values["slack_kg"]))
1044 final = trajectory.iloc[-1]
1045 atmosphere_deviation = max_deviation_from_layered_isa()
1046 solver_status = _formal_solver_status(
1047     result=optimizer_result,
1048     slack_kg=slack_kg,
1049     diagnostics=diagnostics,
1050     reintegration=reintegration,
1051     max_scaled_constraint_violation=max_scaled_violation,
1052     q3=q3,
1053     gate_config=gate_config,
1054 )
1055 return trajectory, {
1056     "wind_model": "no_wind",
1057     "method": "range_domain_collocation_feasibility_gate",
1058     "nodes": nodes,
1059     "solver_status": solver_status,
1060     "optimizer_success": bool(getattr(optimizer_result, "success", False)),
1061     "optimizer_message": str(getattr(optimizer_result, "message", "")),
1062     "lexicographic_stage": "stage1_minimize_terminal_mass_slack",
1063     "mass_constraint_policy": "m_f_plus_s_ge_62000_s_ge_0",
1064     "atmosphere_model": "C1_temperature_hydrostatic_pressure",
1065     "terminal_mass_kg": float(final["mass_kg"]),
1066     "terminal_mass_min_kg": q3.terminal_mass_min_kg,
1067     "terminal_mass_slack_kg": slack_kg,
1068     "terminal_mass_shortfall_kg": max(0.0, q3.terminal_mass_min_kg -
1069 float(final["mass_kg"])),
1070     "terminal_height_error_m": abs(float(final["height_m"] -
1071 q3.terminal_height_m)),
1072     "terminal_speed_error_mps": abs(float(final["airspeed_mps"] -
1073 q3.terminal_airspeed_mps)),
1074     "fuel_used_kg": params.m0_kg - float(final["mass_kg"]),
1075     "final_time_s": float(final["time_s"]),
1076     "max_scaled_constraint_violation": max_scaled_violation,
1077     "constraint_violation_scale": "nondimensional",

```

```

1073         **diagnostics,
1074         **reintegration,
1075         **atmosphere_deviation,
1076     }
1077
1078
1079 def _reintegration_tolerance_study(
1080     trajectory: pd.DataFrame,
1081     *,
1082     q3: Q3Config,
1083     params: Q2Parameters,
1084     rtols: list[float],
1085 ) -> pd.DataFrame:
1086     rows: list[dict[str, float | bool]] = []
1087     previous_speed_error: float | None = None
1088     previous_mass: float | None = None
1089     for rtol in rtols:
1090         atol = _strict_reintegration_atol(rtol)
1091         result = _reintegration_result(trajectory, q3=q3, params=params,
1092                                     rtol=rtol, atol=atol)
1093         terminal_speed = float(result["terminal_speed_signed_error_mps"])
1094         terminal_mass = float(result["terminal_mass_kg"])
1095         rows.append(
1096             {
1097                 "nodes": int(len(trajectory)),
1098                 "h_max_m": float(q3.h_max_m),
1099                 "rtol": float(rtol),
1100                 "atol_height_m": float(atol[0]),
1101                 "atol_speed_mps": float(atol[1]),
1102                 "atol_mass_kg": float(atol[2]),
1103                 "atol_time_s": float(atol[3]),
1104                 "terminal_mass_kg": terminal_mass,
1105                 "terminal_mass_signed_error_kg":
1106 float(result["terminal_mass_signed_error_kg"]),
1107                 "terminal_speed_signed_error_mps": terminal_speed,
1108                 "terminal_height_signed_error_m":
1109 float(result["terminal_height_signed_error_m"]),
1110                 "terminal_speed_delta_from_previous_mps": (
1111                     math.nan if previous_speed_error is None else terminal_speed
1112 - previous_speed_error
1113                 ),
1114                 "terminal_mass_delta_from_previous_kg": (
1115                     math.nan if previous_mass is None else terminal_mass -
1116 previous_mass

```

```

1112         ),
1113         "success": bool(result["success"]),
1114     }
1115 )
1116 previous_speed_error = terminal_speed
1117 previous_mass = terminal_mass
1118 return pd.DataFrame(rows)
1119
1120
1121 def _continuous_audit_study(
1122     trajectory: pd.DataFrame,
1123     *,
1124     q3: Q3Config,
1125     params: Q2Parameters,
1126     rtols: list[float],
1127 ) -> pd.DataFrame:
1128     rows: list[dict[str, float | bool]] = []
1129     for rtol in rtols:
1130         result = _reintegration_result(
1131             trajectory,
1132             q3=q3,
1133             params=params,
1134             rtol=rtol,
1135             atol=_strict_reintegration_atol(rtol),
1136         )
1137         rows.append(
1138             {
1139                 "nodes": int(len(trajectory)),
1140                 "h_max_m": float(q3.h_max_m),
1141                 "rtol": float(rtol),
1142                 "max_height_error_m": float(result["max_height_error_m"]),
1143                 "max_speed_error_mps": float(result["max_speed_error_mps"]),
1144                 "max_mass_error_kg": float(result["max_mass_error_kg"]),
1145                 "max_time_error_s": float(result["max_time_error_s"]),
1146                 "max_scaled_state_error": float(result["max_scaled_state_error"]),
1147                 "max_continuous_scaled_constraint_violation": float(
1148                     result["max_continuous_scaled_constraint_violation"]
1149                 ),
1150                 "max_height_violation_m": float(result["max_height_violation_m"]),
1151                 "max_speed_violation_mps":
1152 float(result["max_speed_violation_mps"]),
1153                 "max_mach_violation": float(result["max_mach_violation"]),
1154                 "max_thrust_violation_n": float(result["max_thrust_violation_n"]),
1155                 "max_gamma_violation_rad":

```

```

1155         float(result["max_gamma_violation_rad"]),
1156         "success": bool(result["success"]),
1157     }
1158     return pd.DataFrame(rows)
1159
1160
1161 def _error_ratio(previous: float | None, current: float) -> float:
1162     if previous is None or not np.isfinite(previous) or not np.isfinite(current):
1163         return math.nan
1164     if current == 0.0:
1165         return math.inf if previous > 0.0 else math.nan
1166     return previous / current
1167
1168
1169 def _mesh_convergence_study(
1170     gate1: pd.DataFrame,
1171     *,
1172     q3: Q3Config,
1173     params: Q2Parameters,
1174     nodes_list: list[int],
1175     gate_config: dict,
1176     precomputed_cases: dict[int, tuple[pd.DataFrame, dict[str, float | str |
1177 bool]]] | None = None,
1178 ) -> pd.DataFrame:
1179     rows: list[dict[str, float | str | bool]] = []
1180     previous_mass_error: float | None = None
1181     previous_speed_error: float | None = None
1182     precomputed_cases = precomputed_cases or {}
1183     for node_count in nodes_list:
1184         if node_count in precomputed_cases:
1185             trajectory, summary = precomputed_cases[node_count]
1186         else:
1187             trajectory, summary = _formal_gate_case(
1188                 gate1,
1189                 q3=q3,
1190                 params=params,
1191                 nodes=node_count,
1192                 gate_config=gate_config,
1193             )
1194             mass_error = float(summary["reintegration_terminal_mass_error_kg"])
1195             speed_error = float(summary["reintegration_terminal_speed_error_mps"])
1196             rows.append(

```

```

1197         "nodes": node_count,
1198         "h_max_m": q3.h_max_m,
1199         "terminal_mass_kg": float(summary["terminal_mass_kg"]),
1200         "terminal_mass_slack_kg":
float(summary["terminal_mass_slack_kg"]),
1201         "final_time_s": float(summary["final_time_s"]),
1202         "scaled_collocation_defect_inf":
float(summary["scaled_collocation_defect_inf"]),
1203         "max_scaled_constraint_violation":
float(summary["max_scaled_constraint_violation"]),
1204         "reintegration_state_error_inf":
float(summary["reintegration_state_error_inf"]),
1205         "reintegration_terminal_mass_kg":
float(summary["reintegration_terminal_mass_kg"]),
1206         "reintegration_terminal_mass_signed_error_kg": float(
1207             summary["reintegration_terminal_mass_signed_error_kg"]
1208         ),
1209         "reintegration_terminal_mass_error_kg": mass_error,
1210         "reintegration_terminal_mass_shortfall_kg": float(
1211             summary["reintegration_terminal_mass_shortfall_kg"]
1212         ),
1213         "reintegration_terminal_height_signed_error_m": float(
1214             summary["reintegration_terminal_height_signed_error_m"]
1215         ),
1216         "reintegration_terminal_height_error_m":
float(summary["reintegration_terminal_height_error_m"]),
1217         "reintegration_terminal_speed_signed_error_mps": float(
1218             summary["reintegration_terminal_speed_signed_error_mps"]
1219         ),
1220         "terminal_speed_signed_error_mps":
float(summary["reintegration_terminal_speed_signed_error_mps"]),
1221         "reintegration_terminal_speed_error_mps": speed_error,
1222         "mass_error_ratio_from_previous":
_error_ratio(previous_mass_error, mass_error),
1223         "speed_error_ratio_from_previous":
_error_ratio(previous_speed_error, speed_error),
1224         **_control_variation_metrics(trajecory),
1225         "max_node_speed_signed_error_mps": float(
1226             summary["reintegration_max_node_speed_signed_error_mps"]
1227         ),
1228         "max_node_speed_error_mps":
float(summary["reintegration_max_node_speed_error_mps"]),
1229         "max_node_mass_signed_error_kg":
float(summary["reintegration_max_node_mass_signed_error_kg"]),

```

```

1230         "max_node_mass_error_kg":
float(summary["reintegration_max_node_mass_error_kg"]),
1231         "gate_status": str(summary["solver_status"]),
1232         "optimizer_success": bool(summary["optimizer_success"]),
1233     }
1234 )
1235     previous_mass_error = mass_error
1236     previous_speed_error = speed_error
1237     return pd.DataFrame(rows)
1238
1239
1240 def _optimized_hmax_sensitivity(
1241     gate1: pd.DataFrame,
1242     *,
1243     q3: Q3Config,
1244     params: Q2Parameters,
1245     nodes: int,
1246     hmax_values: list[float],
1247     gate_config: dict,
1248 ) -> pd.DataFrame:
1249     rows: list[dict[str, float | str]] = []
1250     for hmax in hmax_values:
1251         local_q3 = replace(q3, h_max_m=float(hmax))
1252         vector, result = _solve_stage1_collocation(gate1, q3=local_q3,
params=params, nodes=nodes, maxiter=60)
1253         trajectory = _decision_to_trajectory(vector, nodes=nodes, q3=local_q3,
params=params)
1254         trajectory["scaled_constraint_violation"] =
trajectory.apply(_scaled_constraint_violation, axis=1, q3=local_q3)
1255         values = _unpack_decision(vector, nodes)
1256         diagnostics = _formal_collocation_diagnostics(trajectory, local_q3)
1257         reintegration = _reintegration_diagnostics(trajectory, q3=local_q3,
params=params)
1258         max_scaled_violation =
float(trajectory["scaled_constraint_violation"].max())
1259         status = _formal_solver_status(
1260             result=result,
1261             slack_kg=max(0.0, float(values["slack_kg"])),
1262             diagnostics=diagnostics,
1263             reintegration=reintegration,
1264             max_scaled_constraint_violation=max_scaled_violation,
1265             q3=local_q3,
1266             gate_config=gate_config,
1267         )

```



```

1268     rows.append(
1269         {
1270             "h_max_m": float(hmax),
1271             "nodes": nodes,
1272             "terminal_mass_kg": float(trajjectory["mass_kg"].iloc[-1]),
1273             "terminal_mass_slack_kg": max(0.0, float(values["slack_kg"])),
1274             "final_time_s": float(trajjectory["time_s"].iloc[-1]),
1275             "max_height_m": float(trajjectory["height_m"].max()),
1276             "max_height_violation_m": max(0.0,
float(trajjectory["height_m"].max()) - float(hmax)),
1277             "scaled_collocation_defect_inf":
diagnostics["scaled_collocation_defect_inf"],
1278             "max_scaled_constraint_violation": max_scaled_violation,
1279             "reintegration_terminal_mass_kg":
float(reintegration["reintegration_terminal_mass_kg"]),
1280             "reintegration_terminal_mass_signed_error_kg": float(
reintegration["reintegration_terminal_mass_signed_error_kg"]
1281         ),
1282             "reintegration_terminal_mass_shortfall_kg": float(
reintegration["reintegration_terminal_mass_shortfall_kg"]
1283         ),
1284             "reintegration_terminal_height_error_m":
float(reintegration["reintegration_terminal_height_error_m"]),
1285             "reintegration_terminal_speed_error_mps":
float(reintegration["reintegration_terminal_speed_error_mps"]),
1286             "active_hmax_fraction":
float(np.mean(np.isclose(trajjectory["height_m"], float(hmax), atol=1.0))),
1287             "gate_status": status,
1288             "status": status,
1289         }
1290     )
1291     return pd.DataFrame(rows)
1292
1293
1294
1295
1296 def _fuel_identity_residual_kg(trajjectory: pd.DataFrame, *, q3: Q3Config, params:
Q2Parameters) -> float:
1297     distance = trajjectory["distance_m"].to_numpy(dtype=float)
1298     thrust = trajjectory["thrust_n"].to_numpy(dtype=float)
1299     gamma = trajjectory["gamma_rad"].to_numpy(dtype=float)
1300
1301     def rhs(distance_m: float, state: np.ndarray) -> list[float]:
1302         thrust_n = float(np.interp(distance_m, distance, thrust))
1303         gamma_rad = float(np.interp(distance_m, distance, gamma))
1304         rates = _rates(

```

```

1305         height_m=float(state[0]),
1306         airspeed_mps=float(state[1]),
1307         mass_kg=float(state[2]),
1308         thrust_n=thrust_n,
1309         gamma_rad=gamma_rad,
1310         params=params,
1311         atmosphere_model=atmosphere,
1312     )
1313     return [rates["dh_dx"], rates["dV_dx"], rates["dm_dx"], rates["dt_dx"],
1314             -rates["dm_dx"]]
1315
1316 try:
1317     solution = solve_ivp(
1318         rhs,
1319         (float(distance[0]), float(distance[-1])),
1320         np.array([9500.0, q3.terminal_airspeed_mps, params.m0_kg, 0.0, 0.0]),
1321         rtol=1.0e-9,
1322         atol=np.array([1.0e-6, 1.0e-8, 1.0e-5, 1.0e-7, 1.0e-5]),
1323         max_step=q3.target_distance_m / max(len(distance) - 1, 1) / 4.0,
1324     )
1325     if solution.success:
1326         integral = float(solution.y[4, -1])
1327     else:
1328         raise RuntimeError(solution.message)
1329 except (RuntimeError, ValueError, FloatingPointError):
1330     airspeed = trajectory["airspeed_mps"].to_numpy(dtype=float)
1331     groundspeed = trajectory["groundspeed_mps"].to_numpy(dtype=float)
1332     penalty = 1.0 + params.beta_s2pm2 * (airspeed - params.v_opt_mps) ** 2
1333     integrand = params.c_t_kg_per_ns * thrust * penalty / groundspeed
1334     integral = float(np.trapezoid(integrand, distance))
1335     mass_loss = float(trajectory["mass_kg"].iloc[0] -
1336                       trajectory["mass_kg"].iloc[-1])
1337     return abs(mass_loss - integral)
1338
1339 def _final_validation_status(
1340     *,
1341     validation: dict[str, float | str | bool],
1342     gate_config: dict,
1343 ) -> str:
1344     criteria = gate_config["pass_criteria"]
1345     checks = [
1346         float(validation["reintegration_terminal_speed_error_mps"])
1347         <= float(criteria["terminal_airspeed_error_mps"]),

```

```

1347     float(validation["reintegration_terminal_height_error_m"])
1348     <= float(criteria["terminal_height_error_m"]),
1349     float(validation["fuel_identity_residual_kg"]) <= 0.05,
1350     float(validation["max_continuous_scaled_constraint_violation"])
1351     <= float(criteria["constraint_violation_tolerance"]),
1352     float(validation["objective_grid_abs_delta_kg"]) <= 1.0,
1353     float(validation["objective_grid_relative_delta"]) <= 1.0e-4,
1354     float(validation["multi_initial_objective_range_kg"]) <= 1.0,
1355     bool(validation["optimizer_success"]),
1356 ]
1357 return "passed" if all(checks) else "failed"
1358
1359
1360 def _final_case_summary(
1361     trajectory: pd.DataFrame,
1362     *,
1363     q3: Q3Config,
1364     params: Q2Parameters,
1365     optimizer_result: object,
1366     continuation_nodes: list[int],
1367     method: str = "range_domain_collocation_final_fuel_optimization",
1368 ) -> dict[str, float | str | bool]:
1369     final = trajectory.iloc[-1]
1370     fuel_used = params.m0_kg - float(final["mass_kg"])
1371     return {
1372         "artifact_id": "q3-T07",
1373         "wind_model": "no_wind",
1374         "method": method,
1375         "objective": "min_m0_minus_mf",
1376         "slack_policy": "s_fixed_0",
1377         "mass_constraint_policy": "m_f_ge_62000_s_fixed_0",
1378         "continuation_nodes": "->".join(str(x) for x in continuation_nodes),
1379         "final_nodes": int(len(trajectory)),
1380         "terminal_mass_kg": float(final["mass_kg"]),
1381         "terminal_mass_min_kg": q3.terminal_mass_min_kg,
1382         "terminal_mass_shortfall_kg": max(0.0, q3.terminal_mass_min_kg -
1383 float(final["mass_kg"])),
1384         "fuel_used_kg": fuel_used,
1385         "final_time_s": float(final["time_s"]),
1386         "max_height_m": float(trajectory["height_m"].max()),
1387         "min_height_m": float(trajectory["height_m"].min()),
1388         "min_airspeed_mps": float(trajectory["airspeed_mps"].min()),
1389         "max_airspeed_mps": float(trajectory["airspeed_mps"].max()),
1390         "min_thrust_n": float(trajectory["thrust_n"].min()),

```

```

1390         "max_thrust_n": float(trajecory["thrust_n"].max()),
1391         "max_mach": float(trajecory["mach"].max()),
1392         "max_scaled_constraint_violation":
1393         float(trajecory["scaled_constraint_violation"].max()),
1394         "optimizer_success": bool(getattr(optimizer_result, "success", False)),
1395         "optimizer_message": str(getattr(optimizer_result, "message", "")),
1396         "optimizer_iterations": int(getattr(optimizer_result, "nit", -1)),
1397     }
1398
1399 def _final_validation_row(
1400     trajectory: pd.DataFrame,
1401     *,
1402     q3: Q3Config,
1403     params: Q2Parameters,
1404     optimizer_result: object,
1405     gate_config: dict,
1406     continuation_summaries: list[dict[str, float | str | bool]],
1407     multi_initial_summaries: list[dict[str, float | str | bool]],
1408     kkt_proxy: str = "slsqp_success_and_max_constraint_violation",
1409 ) -> dict[str, float | str | bool]:
1410     diagnostics = _formal_collocation_diagnostics(trajectory, q3)
1411     reintegration = _reintegration_diagnostics(trajectory, q3=q3, params=params)
1412     continuous = _reintegration_result(
1413         trajectory,
1414         q3=q3,
1415         params=params,
1416         rtol=1.0e-8,
1417         atol=_strict_reintegration_atol(1.0e-8),
1418     )
1419     fuel_used = params.m0_kg - float(trajectory["mass_kg"].iloc[-1])
1420     objective_by_nodes = {
1421         int(row["final_nodes"]): float(row["fuel_used_kg"])
1422         for row in continuation_summaries
1423         if bool(row.get("optimizer_success", False))
1424     }
1425     final_nodes = sorted(objective_by_nodes)
1426     if len(final_nodes) >= 2:
1427         previous_node = final_nodes[-2]
1428         current_node = final_nodes[-1]
1429         grid_delta = abs(objective_by_nodes[previous_node] -
1430             objective_by_nodes[current_node])
1431         grid_relative = grid_delta / max(abs(objective_by_nodes[current_node]),
1432             1.0)

```

```

1431     else:
1432         grid_delta = math.inf
1433         grid_relative = math.inf
1434     objectives = [
1435         float(row["fuel_used_kg"])
1436         for row in multi_initial_summaries
1437         if bool(row.get("optimizer_success", False))
1438     ]
1439     objective_range = (max(objectives) - min(objectives)) if objectives else
math.inf
1440     near_zero_fraction =
float(np.mean(trajecory["thrust_n"].to_numpy(dtype=float) <=
max(q3.thrust_max_n * 1.0e-4, 1.0)))
1441     tf_over_t_base = float(trajecory["time_s"].iloc[-1]) / 790.755
1442     validation: dict[str, float | str | bool] = {
1443         "artifact_id": "q3-T08",
1444         "final_nodes": int(len(trajecory)),
1445         "optimizer_success": bool(getattr(optimizer_result, "success", False)),
1446         "optimizer_message": str(getattr(optimizer_result, "message", "")),
1447         "scaled_collocation_defect_inf":
float(diagnostics["scaled_collocation_defect_inf"]),
1448         "max_scaled_constraint_violation":
float(trajecory["scaled_constraint_violation"].max()),
1449         "reintegration_state_error_inf":
float(reintegration["reintegration_state_error_inf"]),
1450         "reintegration_terminal_mass_kg":
float(reintegration["reintegration_terminal_mass_kg"]),
1451         "reintegration_terminal_mass_error_kg":
float(reintegration["reintegration_terminal_mass_error_kg"]),
1452         "reintegration_terminal_mass_shortfall_kg":
float(reintegration["reintegration_terminal_mass_shortfall_kg"]),
1453         "reintegration_terminal_height_error_m":
float(reintegration["reintegration_terminal_height_error_m"]),
1454         "reintegration_terminal_speed_error_mps":
float(reintegration["reintegration_terminal_speed_error_mps"]),
1455         "max_continuous_scaled_constraint_violation": float(
1456             continuous["max_continuous_scaled_constraint_violation"]
1457         ),
1458         "fuel_identity_residual_kg": _fuel_identity_residual_kg(trajecory,
q3=q3, params=params),
1459         "objective_grid_abs_delta_kg": grid_delta,
1460         "objective_grid_relative_delta": grid_relative,
1461         "multi_initial_objective_range_kg": objective_range,
1462         "near_zero_thrust_fraction": near_zero_fraction,

```

```

1463     "tf_over_t_base": tf_over_t_base,
1464     "time_limit_1p05_feasible": tf_over_t_base <= 1.05,
1465     "time_limit_1p10_feasible": tf_over_t_base <= 1.10,
1466     "kkt_proxy": kkt_proxy,
1467     "fuel_used_kg": fuel_used,
1468 }
1469 validation["validation_status"] =
1470 _final_validation_status(validation=validation, gate_config=gate_config)
1471 return validation
1472
1473 def _initial_decision_by_name(
1474     name: str,
1475     *,
1476     gate1: pd.DataFrame,
1477     q3: Q3Config,
1478     params: Q2Parameters,
1479     nodes: int,
1480     previous_trajectory: pd.DataFrame | None = None,
1481 ) -> np.ndarray:
1482     if previous_trajectory is not None and name in {"gate2", "perturbed"}:
1483         base = _resample_trajectory_decision(previous_trajectory, q3=q3,
1484         nodes=nodes, slack_kg=0.0)
1485         if name == "perturbed":
1486             return _perturb_decision(base, nodes=nodes, q3=q3)
1487         return base
1488     if name == "straight":
1489         return _straight_initial_decision(q3=q3, params=params, nodes=nodes)
1490     gate_vector, _ = _solve_stage1_collocation(gate1, q3=q3, params=params,
1491     nodes=nodes, maxiter=120)
1492     gate_values = _unpack_decision(gate_vector, nodes)
1493     gate_vector = _pack_decision(
1494         height_m=gate_values["height_m"],
1495         airspeed_mps=gate_values["airspeed_mps"],
1496         mass_kg=np.maximum(gate_values["mass_kg"], q3.terminal_mass_min_kg),
1497         time_s=gate_values["time_s"],
1498         thrust_n=gate_values["thrust_n"],
1499         gamma_rad=gate_values["gamma_rad"],
1500         slack_kg=0.0,
1501     )
1502     if name == "perturbed":
1503         return _perturb_decision(gate_vector, nodes=nodes, q3=q3)
1504     return gate_vector

```

```

1504
1505 def _shooting_initial_trajectory(
1506     gate1: pd.DataFrame,
1507     *,
1508     q3: Q3Config,
1509     params: Q2Parameters,
1510     nodes: int,
1511     previous_trajectory: pd.DataFrame | None = None,
1512     guess: str = "gate2",
1513 ) -> pd.DataFrame:
1514     if previous_trajectory is not None and guess in {"gate2", "perturbed"}:
1515         source = previous_trajectory.copy()
1516     elif guess == "straight":
1517         vector = _straight_initial_decision(q3=q3, params=params, nodes=nodes)
1518         source = _decision_to_trajectory(vector, nodes=nodes, q3=q3,
1519                                         params=params)
1520     elif guess == "perturbed":
1521         vector = _initial_decision_by_name("perturbed", gate1=gate1, q3=q3,
1522                                         params=params, nodes=nodes)
1523         source = _decision_to_trajectory(vector, nodes=nodes, q3=q3,
1524                                         params=params)
1525     else:
1526         source = _project_gate1_warm_start(gate1, q3=q3, params=params,
1527                                         nodes=nodes)
1528     source = source.sort_values("distance_m").drop_duplicates("distance_m").copy()
1529     required = {"distance_m", "thrust_n", "gamma_rad"}
1530     missing = required.difference(source.columns)
1531     if missing:
1532         raise ValueError(f"shooting initial trajectory missing columns:
1533 {sorted(missing)}")
1534     return source
1535
1536 def _pack_shooting_controls(thrust_n: np.ndarray, gamma_rad: np.ndarray) ->
1537     np.ndarray:
1538     return np.concatenate([thrust_n / T_SCALE_N, gamma_rad / GAMMA_SCALE_RAD])
1539
1540 def _unpack_shooting_controls(vector: np.ndarray, control_knots: int) ->
1541     tuple[np.ndarray, np.ndarray]:
1542     thrust = vector[:control_knots] * T_SCALE_N
1543     gamma = vector[control_knots : 2 * control_knots] * GAMMA_SCALE_RAD
1544     return thrust, gamma

```

```

1541
1542 def _shooting_bounds(control_knots: int, q3: Q3Config) -> list[tuple[float,
1543 float]]:
1544     return (
1545         [(q3.thrust_min_n / T_SCALE_N, q3.thrust_max_n / T_SCALE_N)] *
1546         control_knots
1547         + [(-q3.gamma_max_rad / GAMMA_SCALE_RAD, q3.gamma_max_rad /
1548 GAMMA_SCALE_RAD)] * control_knots
1549     )
1550
1551 def _shooting_control_initial(
1552     trajectory: pd.DataFrame,
1553     *,
1554     q3: Q3Config,
1555     control_knots: int,
1556     perturb: bool = False,
1557 ) -> np.ndarray:
1558     source = trajectory.sort_values("distance_m").drop_duplicates("distance_m")
1559     knot_distance = np.linspace(0.0, q3.target_distance_m, control_knots)
1560     thrust = np.interp(
1561         knot_distance,
1562         source["distance_m"].to_numpy(dtype=float),
1563         source["thrust_n"].to_numpy(dtype=float),
1564     )
1565     gamma = np.interp(
1566         knot_distance,
1567         source["distance_m"].to_numpy(dtype=float),
1568         source["gamma_rad"].to_numpy(dtype=float),
1569     )
1570     if perturb:
1571         phase = np.linspace(0.0, math.pi, control_knots)
1572         thrust = thrust * (1.0 + 0.015 * np.sin(phase))
1573         gamma = gamma + 2.0e-4 * np.sin(2.0 * phase)
1574     thrust = np.clip(thrust, q3.thrust_min_n, q3.thrust_max_n)
1575     gamma = np.clip(gamma, -q3.gamma_max_rad, q3.gamma_max_rad)
1576     return _pack_shooting_controls(thrust, gamma)
1577
1578 def _evaluate_shooting_controls(
1579     vector: np.ndarray,
1580     *,
1581     q3: Q3Config,
1582     params: Q2Parameters,

```



```

1582     nodes: int,
1583     control_knots: int,
1584     rtol: float = 3.0e-6,
1585     height_margin_m: float = SHOOTING_HEIGHT_MARGIN_M,
1586 ) -> ShootingEvaluation | None:
1587     thrust_knots, gamma_knots = _unpack_shooting_controls(vector, control_knots)
1588     if np.any(thrust_knots < q3.thrust_min_n - 1.0e-8) or np.any(thrust_knots >
1589         q3.thrust_max_n + 1.0e-8):
1589         return None
1590     if np.any(np.abs(gamma_knots) > q3.gamma_max_rad + 1.0e-10):
1591         return None
1592
1593     distance_eval = np.linspace(0.0, q3.target_distance_m, nodes)
1594     knot_distance = np.linspace(0.0, q3.target_distance_m, control_knots)
1595
1596     def rhs(distance_m: float, state: np.ndarray) -> list[float]:
1597         thrust_n = float(np.interp(distance_m, knot_distance, thrust_knots))
1598         gamma_rad = float(np.interp(distance_m, knot_distance, gamma_knots))
1599         rates = _rates(
1600             height_m=float(state[0]),
1601             airspeed_mps=float(state[1]),
1602             mass_kg=float(state[2]),
1603             thrust_n=thrust_n,
1604             gamma_rad=gamma_rad,
1605             params=params,
1606             atmosphere_model=atmosphere,
1607         )
1608         return [rates["dh_dx"], rates["dV_dx"], rates["dm_dx"], rates["dt_dx"]]
1609
1610     try:
1611         solution = solve_ivp(
1612             rhs,
1613             (0.0, q3.target_distance_m),
1614             np.array([9500.0, q3.terminal_airspeed_mps, params.m0_kg, 0.0]),
1615             t_eval=distance_eval,
1616             rtol=rtol,
1617             atol=np.array([1.0e-4, 1.0e-6, 1.0e-3, 1.0e-5]),
1618             max_step=q3.target_distance_m / max(nodes - 1, 1) / 2.0,
1619         )
1620     except (RuntimeError, ValueError, FloatingPointError):
1621         return None
1622     if not solution.success or np.any(~np.isfinite(solution.y)):
1623         return None
1624

```

```

1625 records: list[dict[str, float]] = []
1626 max_scaled_violation = 0.0
1627 for index, distance_m in enumerate(distance_eval):
1628     thrust_n = float(np.interp(distance_m, knot_distance, thrust_knots))
1629     gamma_rad = float(np.interp(distance_m, knot_distance, gamma_knots))
1630     height_m = float(solution.y[0, index])
1631     airspeed_mps = float(solution.y[1, index])
1632     mass_kg = float(solution.y[2, index])
1633     try:
1634         rates = _rates(
1635             height_m=height_m,
1636             airspeed_mps=airspeed_mps,
1637             mass_kg=mass_kg,
1638             thrust_n=thrust_n,
1639             gamma_rad=gamma_rad,
1640             params=params,
1641             atmosphere_model=atmosphere,
1642         )
1643     except (RuntimeError, ValueError, FloatingPointError):
1644         return None
1645     scaled_violation = max(
1646         0.0,
1647         (q3.h_min_m - height_m) / H_SCALE_M,
1648         (height_m - (q3.h_max_m - height_margin_m)) / H_SCALE_M,
1649         (q3.v_min_mps - airspeed_mps) / V_SCALE_MPS,
1650         (airspeed_mps - q3.v_max_mps) / V_SCALE_MPS,
1651         (q3.thrust_min_n - thrust_n) / max(q3.thrust_max_n, 1.0),
1652         (thrust_n - q3.thrust_max_n) / max(q3.thrust_max_n, 1.0),
1653         (abs(gamma_rad) - q3.gamma_max_rad) / max(q3.gamma_max_rad, 1.0e-9),
1654         rates["mach"] - q3.mach_max,
1655         (q3.terminal_mass_min_kg - mass_kg) / M_SCALE_KG,
1656     )
1657     max_scaled_violation = max(max_scaled_violation, float(scaled_violation))
1658     records.append(
1659         {
1660             "distance_m": float(distance_m),
1661             "height_m": height_m,
1662             "airspeed_mps": airspeed_mps,
1663             "mass_kg": mass_kg,
1664             "time_s": float(solution.y[3, index]),
1665             "thrust_n": thrust_n,
1666             "gamma_rad": gamma_rad,
1667             **rates,
1668             "scaled_constraint_violation": float(scaled_violation),

```

```

1669     }
1670 )
1671
1672 trajectory = pd.DataFrame.from_records(records)
1673 trajectory["scaled_constraint_violation"] =
1674 trajectory.apply(_scaled_constraint_violation, axis=1, q3=q3)
1675 final = trajectory.iloc[-1]
1676 return ShootingEvaluation(
1677     trajectory=trajectory,
1678     final_height_error_m=float(final["height_m"] - q3.terminal_height_m),
1679     final_speed_error_mps=float(final["airspeed_mps"] -
1680 q3.terminal_airspeed_mps),
1681     final_mass_shortfall_kg=max(0.0, q3.terminal_mass_min_kg -
1682 float(final["mass_kg"])),
1683     max_scaled_constraint_violation=max_scaled_violation,
1684     fuel_used_kg=params.m0_kg - float(final["mass_kg"]),
1685     success=True,
1686 )
1687
1688 def _shooting_eval_or_penalty(
1689     vector: np.ndarray,
1690     *,
1691     q3: Q3Config,
1692     params: Q2Parameters,
1693     nodes: int,
1694     control_knots: int,
1695     cache: dict[tuple[float, ...], ShootingEvaluation | None],
1696 ) -> ShootingEvaluation | None:
1697     key = tuple(np.round(vector, 8))
1698     if key not in cache:
1699         cache[key] = _evaluate_shooting_controls(
1700             vector,
1701             q3=q3,
1702             params=params,
1703             nodes=nodes,
1704             control_knots=control_knots,
1705         )
1706     return cache[key]
1707
1708 def _solve_final_fuel_shooting_case(
1709     initial: np.ndarray,
1710     *,

```

```

1710     q3: Q3Config,
1711     params: Q2Parameters,
1712     nodes: int,
1713     control_knots: int,
1714     maxiter: int,
1715 ) -> tuple[np.ndarray, ShootingEvaluation, ShootingOptimizerResult]:
1716     cache: dict[tuple[float, ...], ShootingEvaluation | None] = {}
1717
1718     def evaluation(vector: np.ndarray) -> ShootingEvaluation | None:
1719         return _shooting_eval_or_penalty(
1720             vector,
1721             q3=q3,
1722             params=params,
1723             nodes=nodes,
1724             control_knots=control_knots,
1725             cache=cache,
1726         )
1727
1728     def objective(vector: np.ndarray) -> float:
1729         current = evaluation(vector)
1730         if current is None:
1731             return 1.0e5
1732         thrust, gamma = _unpack_shooting_controls(vector, control_knots)
1733         smooth = 1.0e-10 * float(np.sum(np.diff(thrust) ** 2)) + 1.0e-2 *
1734         float(np.sum(np.diff(gamma) ** 2))
1735         return current.fuel_used_kg / SLACK_SCALE_KG + smooth
1736
1737     def terminal_height_eq(vector: np.ndarray) -> float:
1738         current = evaluation(vector)
1739         return -1.0 if current is None else current.final_height_error_m /
1740         H_SCALE_M
1741
1742     def terminal_speed_eq(vector: np.ndarray) -> float:
1743         current = evaluation(vector)
1744         return -1.0 if current is None else current.final_speed_error_mps /
1745         V_SCALE_MPS
1746
1747     def path_ineq(vector: np.ndarray) -> np.ndarray:
1748         current = evaluation(vector)
1749         if current is None:
1750             return np.array([-1.0, -1.0], dtype=float)
1751         terminal_mass_margin = float(current.trajectory["mass_kg"].iloc[-1] -
1752         q3.terminal_mass_min_kg)
1753         return np.array(

```

```

1750         [
1751             terminal_mass_margin,
1752             1.0e-6 - current.max_scaled_constraint_violation,
1753         ],
1754         dtype=float,
1755     )
1756
1757     result = minimize(
1758         objective,
1759         initial,
1760         method="SLSQP",
1761         bounds=_shooting_bounds(control_knots, q3),
1762         constraints=[
1763             {"type": "eq", "fun": terminal_height_eq},
1764             {"type": "eq", "fun": terminal_speed_eq},
1765             {"type": "ineq", "fun": path_ineq},
1766         ],
1767         options={"maxiter": maxiter, "ftol": 1.0e-8, "disp": False, "eps":
1.0e-4},
1768     )
1769     vector = np.asarray(result.x if hasattr(result, "x") else initial,
dtype=float)
1770     strict_eval = _evaluate_shooting_controls(
1771         vector,
1772         q3=q3,
1773         params=params,
1774         nodes=nodes,
1775         control_knots=control_knots,
1776         rtol=1.0e-8,
1777     )
1778     if strict_eval is None:
1779         fallback = _evaluate_shooting_controls(
1780             initial,
1781             q3=q3,
1782             params=params,
1783             nodes=nodes,
1784             control_knots=control_knots,
1785             rtol=1.0e-8,
1786         )
1787     if fallback is None:
1788         raise RuntimeError("shooting solver failed to produce an evaluable
trajectory")
1789     vector = initial
1790     strict_eval = fallback

```

```

1791 validation_success = (
1792     abs(strict_eval.final_height_error_m) <= 0.1
1793     and abs(strict_eval.final_speed_error_mps) <= 1.0e-3
1794     and strict_eval.max_scaled_constraint_violation <= 1.0e-6
1795     and strict_eval.final_mass_shortfall_kg <= 0.05
1796 )
1797 optimizer_result = ShootingOptimizerResult(
1798     success=bool(getattr(result, "success", False)) or validation_success,
1799     message=str(getattr(result, "message", "")),
1800     nit=int(getattr(result, "nit", -1)),
1801     validation_success=validation_success,
1802 )
1803 return vector, strict_eval, optimizer_result
1804
1805
1806 def _solve_final_fuel_shooting_workflow(
1807     gate1: pd.DataFrame,
1808     *,
1809     q3: Q3Config,
1810     params: Q2Parameters,
1811     nodes: int,
1812     continuation_nodes: list[int],
1813     initial_guess: str,
1814     multi_initial_guesses: list[str],
1815     gate_config: dict,
1816     maxiter: int,
1817     control_knots: int,
1818 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
1819     if control_knots < 3:
1820         raise ValueError("--shooting-control-knots must be at least 3")
1821     node_sequence = continuation_nodes or [nodes]
1822     if node_sequence[-1] != nodes:
1823         node_sequence.append(nodes)
1824
1825     continuation_summaries: list[dict[str, float | str | bool]] = []
1826     previous_trajectory: pd.DataFrame | None = None
1827     previous_controls: np.ndarray | None = None
1828     final_result: ShootingOptimizerResult | None = None
1829     final_trajectory: pd.DataFrame | None = None
1830     final_controls: np.ndarray | None = None
1831     method = "range_domain_reduced_control_shooting_final_fuel_optimization"
1832
1833     for node_count in node_sequence:
1834         if previous_controls is not None:

```

```

1835         initial = previous_controls
1836     else:
1837         source = _shooting_initial_trajectory(
1838             gate1,
1839             q3=q3,
1840             params=params,
1841             nodes=node_count,
1842             previous_trajectory=previous_trajectory,
1843             guess=initial_guess,
1844         )
1845         initial = _shooting_control_initial(
1846             source,
1847             q3=q3,
1848             control_knots=control_knots,
1849             perturb=initial_guess == "perturbed",
1850         )
1851     controls, evaluation, result = _solve_final_fuel_shooting_case(
1852         initial,
1853         q3=q3,
1854         params=params,
1855         nodes=node_count,
1856         control_knots=control_knots,
1857         maxiter=maxiter,
1858     )
1859     trajectory = evaluation.trajectory
1860     summary = _final_case_summary(
1861         trajectory,
1862         q3=q3,
1863         params=params,
1864         optimizer_result=result,
1865         continuation_nodes=node_sequence,
1866         method=method,
1867     )
1868     summary["initial_guess"] = initial_guess
1869     summary["shooting_control_knots"] = control_knots
1870     summary["shooting_validation_success"] = result.validation_success
1871     continuation_summaries.append(summary)
1872     previous_trajectory = trajectory
1873     previous_controls = controls
1874     final_controls = controls
1875     final_result = result
1876     final_trajectory = trajectory
1877
1878     if final_trajectory is None or final_result is None or final_controls is None:

```

```

1879         raise RuntimeError("shooting final fuel workflow did not produce a
trajectory")
1880
1881 multi_initial_summaries: list[dict[str, float | str | bool]] = []
1882 for guess in multi_initial_guesses:
1883     if guess == initial_guess:
1884         controls = final_controls
1885         evaluation = ShootingEvaluation(
1886             trajectory=final_trajectory,
1887             final_height_error_m=float(final_trajectory["height_m"].iloc[-1]
- q3.terminal_height_m),
1888
1889             final_speed_error_mps=float(final_trajectory["airspeed_mps"].iloc[-1] -
q3.terminal_airspeed_mps),
1890             final_mass_shortfall_kg=max(0.0, q3.terminal_mass_min_kg -
float(final_trajectory["mass_kg"].iloc[-1])),
1891
1892             max_scaled_constraint_violation=float(final_trajectory["scaled_constraint_violation"].max()),
1893             fuel_used_kg=params.m0_kg -
float(final_trajectory["mass_kg"].iloc[-1]),
1894             success=True,
1895         )
1896         result = final_result
1897     else:
1898         source = _shooting_initial_trajectory(
1899             gate1,
1900             q3=q3,
1901             params=params,
1902             nodes=nodes,
1903             previous_trajectory=final_trajectory if guess in {"gate2",
"perturbed"} else None,
1904             guess=guess,
1905         )
1906         initial = _shooting_control_initial(
1907             source,
1908             q3=q3,
1909             control_knots=control_knots,
1910             perturb=guess == "perturbed",
1911         )
1912         controls, evaluation, result = _solve_final_fuel_shooting_case(
1913             initial,
1914             q3=q3,
1915             params=params,
1916             nodes=nodes,

```



```

1915         control_knots=control_knots,
1916         maxiter=max(40, maxiter // 2),
1917     )
1918     summary = _final_case_summary(
1919         evaluation.trajectory,
1920         q3=q3,
1921         params=params,
1922         optimizer_result=result,
1923         continuation_nodes=node_sequence,
1924         method=method,
1925     )
1926     summary["initial_guess"] = guess
1927     summary["shooting_control_knots"] = control_knots
1928     summary["shooting_validation_success"] = result.validation_success
1929     multi_initial_summaries.append(summary)
1930
1931     result_summary = _final_case_summary(
1932         final_trajectory,
1933         q3=q3,
1934         params=params,
1935         optimizer_result=final_result,
1936         continuation_nodes=node_sequence,
1937         method=method,
1938     )
1939     result_summary["initial_guess"] = initial_guess
1940     result_summary["shooting_control_knots"] = control_knots
1941     results = pd.DataFrame([result_summary])
1942     validation = pd.DataFrame(
1943         [
1944             _final_validation_row(
1945                 final_trajectory,
1946                 q3=q3,
1947                 params=params,
1948                 optimizer_result=final_result,
1949                 gate_config=gate_config,
1950                 continuation_summaries=continuation_summaries,
1951                 multi_initial_summaries=multi_initial_summaries,
1952             )
1953         ]
1954     )
1955
1956     diagnostics = pd.DataFrame(continuation_summaries + multi_initial_summaries)
1957     return final_trajectory, results, validation, diagnostics

```

```

1958
1959
1960 def _final_hmax_sensitivity(
1961     gate1: pd.DataFrame,
1962     *,
1963     q3: Q3Config,
1964     params: Q2Parameters,
1965     nodes: int,
1966     continuation_nodes: list[int],
1967     initial_guess: str,
1968     gate_config: dict,
1969     maxiter: int,
1970     control_knots: int,
1971     hmax_values: list[float],
1972 ) -> pd.DataFrame:
1973     rows: list[dict[str, float | str | bool]] = []
1974     method = "range_domain_reduced_control_shooting_final_fuel_optimization"
1975     for hmax in hmax_values:
1976         local_q3 = replace(q3, h_max_m=float(hmax))
1977         try:
1978             trajectory, results, validation, _ =
1979             _solve_final_fuel_shooting_workflow(
1980                 gate1,
1981                 q3=local_q3,
1982                 params=params,
1983                 nodes=nodes,
1984                 continuation_nodes=continuation_nodes,
1985                 initial_guess=initial_guess,
1986                 multi_initial_guesses=[initial_guess],
1987                 gate_config=gate_config,
1988                 maxiter=maxiter,
1989                 control_knots=control_knots,
1990             )
1991             result_row = results.iloc[0]
1992             validation_row = validation.iloc[0]
1993             max_height = float(trajectory["height_m"].max())
1994             rows.append(
1995                 {
1996                     "artifact_id": "q3-T09",
1997                     "wind_model": "no_wind",
1998                     "h_max_m": float(hmax),
1999                     "method": method,
2000                     "objective": "min_m0_minus_mf",
2001                     "slack_policy": "s_fixed_0",

```

```

2001         "sensitivity_type": "local_reoptimization",
2002         "multi_initial_policy": "single_start_for_sensitivity",
2003         "continuation_nodes": str(result_row["continuation_nodes"]),
2004         "final_nodes": int(result_row["final_nodes"]),
2005         "shooting_control_knots":
2006         int(result_row.get("shooting_control_knots", control_knots)),
2007         "initial_guess": str(result_row["initial_guess"]),
2008         "terminal_mass_kg": float(result_row["terminal_mass_kg"]),
2009         "terminal_mass_shortfall_kg":
2010         float(result_row["terminal_mass_shortfall_kg"]),
2011         "fuel_used_kg": float(result_row["fuel_used_kg"]),
2012         "final_time_s": float(result_row["final_time_s"]),
2013         "max_height_m": max_height,
2014         "hmax_margin_m": float(hmax) - max_height,
2015         "max_height_violation_m": max(0.0, max_height - float(hmax)),
2016         "active_hmax_fraction":
2017         float(np.mean(np.isclose(trajjectory["height_m"], float(hmax), atol=1.0))),
2018         "reintegration_terminal_height_error_m": float(
2019             validation_row["reintegration_terminal_height_error_m"]
2020         ),
2021         "reintegration_terminal_speed_error_mps": float(
2022             validation_row["reintegration_terminal_speed_error_mps"]
2023         ),
2024         "fuel_identity_residual_kg":
2025         float(validation_row["fuel_identity_residual_kg"]),
2026         "max_continuous_scaled_constraint_violation": float(
2027             validation_row["max_continuous_scaled_constraint_violation"]
2028         ),
2029         "tf_over_t_base": float(validation_row["tf_over_t_base"]),
2030         "optimizer_success": bool(result_row["optimizer_success"]),
2031         "optimizer_message": str(result_row["optimizer_message"]),
2032         "sensitivity_status":
2033         str(validation_row["validation_status"]),
2034     }
2035 )
2036 except (RuntimeError, ValueError, FloatingPointError) as exc:
2037     rows.append(
2038         {
2039             "artifact_id": "q3-T09",
2040             "wind_model": "no_wind",
2041             "h_max_m": float(hmax),
2042             "method": method,
2043             "objective": "min_m0_minus_mf",

```

```

2039         "slack_policy": "s_fixed_0",
2040         "sensitivity_type": "local_reoptimization",
2041         "multi_initial_policy": "single_start_for_sensitivity",
2042         "continuation_nodes": "->".join(str(x) for x in
continuation_nodes),
2043         "final_nodes": int(nodes),
2044         "shooting_control_knots": int(control_knots),
2045         "initial_guess": initial_guess,
2046         "terminal_mass_kg": math.nan,
2047         "terminal_mass_shortfall_kg": math.nan,
2048         "fuel_used_kg": math.nan,
2049         "final_time_s": math.nan,
2050         "max_height_m": math.nan,
2051         "hmax_margin_m": math.nan,
2052         "max_height_violation_m": math.nan,
2053         "active_hmax_fraction": math.nan,
2054         "reintegration_terminal_height_error_m": math.nan,
2055         "reintegration_terminal_speed_error_mps": math.nan,
2056         "fuel_identity_residual_kg": math.nan,
2057         "max_continuous_scaled_constraint_violation": math.nan,
2058         "tf_over_t_base": math.nan,
2059         "optimizer_success": False,
2060         "optimizer_message": str(exc),
2061         "sensitivity_status": "failed",
2062     }
2063 )
2064 return pd.DataFrame(rows)
2065
2066
2067 def _final_idle_thrust_sensitivity(
2068     gate1: pd.DataFrame,
2069     *,
2070     q3: Q3Config,
2071     params: Q2Parameters,
2072     nodes: int,
2073     continuation_nodes: list[int],
2074     initial_guess: str,
2075     gate_config: dict,
2076     maxiter: int,
2077     control_knots: int,
2078     idle_fractions: list[float],
2079     baseline_fuel_used_kg: float,
2080 ) -> pd.DataFrame:
2081     rows: list[dict[str, float | str | bool]] = []

```

```

2082 method = "range_domain_reduced_control_shooting_final_fuel_optimization"
2083 for fraction in idle_fractions:
2084     idle_thrust = float(fraction) * q3.thrust_max_n
2085     local_q3 = replace(q3, thrust_min_n=idle_thrust)
2086     try:
2087         trajectory, results, validation, _ =
_solve_final_fuel_shooting_workflow(
2088             gate1,
2089             q3=local_q3,
2090             params=params,
2091             nodes=nodes,
2092             continuation_nodes=continuation_nodes,
2093             initial_guess=initial_guess,
2094             multi_initial_guesses=[initial_guess],
2095             gate_config=gate_config,
2096             maxiter=maxiter,
2097             control_knots=control_knots,
2098         )
2099         result_row = results.iloc[0]
2100         validation_row = validation.iloc[0]
2101         fuel_used = float(result_row["fuel_used_kg"])
2102         idle_active_fraction = float(
2103             np.mean(np.isclose(trajectory["thrust_n"].to_numpy(dtype=float),
idle_thrust, atol=50.0))
2104         )
2105         rows.append(
2106             {
2107                 "artifact_id": "q3-T10",
2108                 "wind_model": "no_wind",
2109                 "idle_thrust_fraction": float(fraction),
2110                 "idle_thrust_n": idle_thrust,
2111                 "method": method,
2112                 "objective": "min_m0_minus_mf",
2113                 "slack_policy": "s_fixed_0",
2114                 "sensitivity_type": "local_reoptimization",
2115                 "multi_initial_policy": "single_start_for_sensitivity",
2116                 "continuation_nodes": str(result_row["continuation_nodes"]),
2117                 "final_nodes": int(result_row["final_nodes"]),
2118                 "shooting_control_knots":
2119 int(result_row.get("shooting_control_knots", control_knots)),
2119                 "initial_guess": str(result_row["initial_guess"]),
2120                 "terminal_mass_kg": float(result_row["terminal_mass_kg"]),
2121                 "terminal_mass_shortfall_kg":
2122 float(result_row["terminal_mass_shortfall_kg"]),

```

```

2122         "fuel_used_kg": fuel_used,
2123         "fuel_delta_vs_zero_idle_kg": fuel_used -
baseline_fuel_used_kg,
2124         "final_time_s": float(result_row["final_time_s"]),
2125         "min_thrust_n": float(result_row["min_thrust_n"]),
2126         "max_thrust_n": float(result_row["max_thrust_n"]),
2127         "idle_active_fraction": idle_active_fraction,
2128         "near_zero_thrust_fraction":
float(validation_row["near_zero_thrust_fraction"]),
2129         "reintegration_terminal_height_error_m": float(
validation_row["reintegration_terminal_height_error_m"]
2130     ),
2131         "reintegration_terminal_speed_error_mps": float(
validation_row["reintegration_terminal_speed_error_mps"]
2132     ),
2133         "fuel_identity_residual_kg":
float(validation_row["fuel_identity_residual_kg"]),
2134         "max_continuous_scaled_constraint_violation": float(
validation_row["max_continuous_scaled_constraint_violation"]
2135     ),
2136         "tf_over_t_base": float(validation_row["tf_over_t_base"]),
2137         "optimizer_success": bool(result_row["optimizer_success"]),
2138         "optimizer_message": str(result_row["optimizer_message"]),
2139         "sensitivity_status":
str(validation_row["validation_status"]),
2140     }
2141 )
2142 except (RuntimeError, ValueError, FloatingPointError) as exc:
2143     rows.append(
2144         {
2145             "artifact_id": "q3-T10",
2146             "wind_model": "no_wind",
2147             "idle_thrust_fraction": float(fraction),
2148             "idle_thrust_n": idle_thrust,
2149             "method": method,
2150             "objective": "min_m0_minus_mf",
2151             "slack_policy": "s_fixed_0",
2152             "sensitivity_type": "local_reoptimization",
2153             "multi_initial_policy": "single_start_for_sensitivity",
2154             "continuation_nodes": "->".join(str(x) for x in
continuation_nodes),
2155             "final_nodes": int(nodes),
2156             "shooting_control_knots": int(control_knots),

```

```

2160         "initial_guess": initial_guess,
2161         "terminal_mass_kg": math.nan,
2162         "terminal_mass_shortfall_kg": math.nan,
2163         "fuel_used_kg": math.nan,
2164         "fuel_delta_vs_zero_idle_kg": math.nan,
2165         "final_time_s": math.nan,
2166         "min_thrust_n": math.nan,
2167         "max_thrust_n": math.nan,
2168         "idle_active_fraction": math.nan,
2169         "near_zero_thrust_fraction": math.nan,
2170         "reintegration_terminal_height_error_m": math.nan,
2171         "reintegration_terminal_speed_error_mps": math.nan,
2172         "fuel_identity_residual_kg": math.nan,
2173         "max_continuous_scaled_constraint_violation": math.nan,
2174         "tf_over_t_base": math.nan,
2175         "optimizer_success": False,
2176         "optimizer_message": str(exc),
2177         "sensitivity_status": "failed",
2178     }
2179 )
2180 return pd.DataFrame(rows)
2181
2182
2183 def _solve_final_fuel_workflow(
2184     gate1: pd.DataFrame,
2185     *,
2186     q3: Q3Config,
2187     params: Q2Parameters,
2188     nodes: int,
2189     continuation_nodes: list[int],
2190     initial_guess: str,
2191     multi_initial_guesses: list[str],
2192     gate_config: dict,
2193     maxiter: int,
2194 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
2195     node_sequence = continuation_nodes or [nodes]
2196     if node_sequence[-1] != nodes:
2197         node_sequence.append(nodes)
2198     continuation_summaries: list[dict[str, float | str | bool]] = []
2199     previous_trajectory: pd.DataFrame | None = None
2200     final_result: object | None = None
2201     final_trajectory: pd.DataFrame | None = None
2202     for node_count in node_sequence:
2203         initial = _initial_decision_by_name(

```

```

2204         initial_guess,
2205         gate1=gate1,
2206         q3=q3,
2207         params=params,
2208         nodes=node_count,
2209         previous_trajectory=previous_trajectory,
2210     )
2211     vector, result = _solve_final_fuel_collocation(
2212         initial,
2213         q3=q3,
2214         params=params,
2215         nodes=node_count,
2216         maxiter=maxiter,
2217     )
2218     trajectory = _decision_to_trajectory(vector, nodes=node_count, q3=q3,
2219     params=params)
2220     trajectory["scaled_constraint_violation"] =
2221     trajectory.apply(_scaled_constraint_violation, axis=1, q3=q3)
2222     summary = _final_case_summary(
2223         trajectory,
2224         q3=q3,
2225         params=params,
2226         optimizer_result=result,
2227         continuation_nodes=node_sequence,
2228     )
2229     summary["initial_guess"] = initial_guess
2230     continuation_summaries.append(summary)
2231     previous_trajectory = trajectory
2232     final_result = result
2233     final_trajectory = trajectory
2234
2235     if final_trajectory is None or final_result is None:
2236         raise RuntimeError("final fuel workflow did not produce a trajectory")
2237
2238     multi_initial_summaries: list[dict[str, float | str | bool]] = []
2239     for guess in multi_initial_guesses:
2240         initial = _initial_decision_by_name(
2241             guess,
2242             gate1=gate1,
2243             q3=q3,
2244             params=params,
2245             nodes=nodes,
2246             previous_trajectory=final_trajectory if guess in {"gate2",
2247             "perturbed"} else None,

```



```

2245     )
2246     vector, result = _solve_final_fuel_collocation(
2247         initial,
2248         q3=q3,
2249         params=params,
2250         nodes=nodes,
2251         maxiter=max(80, maxiter // 2),
2252     )
2253     trajectory = _decision_to_trajectory(vector, nodes=nodes, q3=q3,
2254     params=params)
2255     trajectory["scaled_constraint_violation"] =
2256     trajectory.apply(_scaled_constraint_violation, axis=1, q3=q3)
2257     summary = _final_case_summary(
2258         trajectory,
2259         q3=q3,
2260         params=params,
2261         optimizer_result=result,
2262         continuation_nodes=node_sequence,
2263     )
2264     summary["initial_guess"] = guess
2265     multi_initial_summaries.append(summary)
2266
2267     result_summary = _final_case_summary(
2268         final_trajectory,
2269         q3=q3,
2270         params=params,
2271         optimizer_result=final_result,
2272         continuation_nodes=node_sequence,
2273     )
2274     result_summary["initial_guess"] = initial_guess
2275     results = pd.DataFrame([result_summary])
2276     validation = pd.DataFrame(
2277         [
2278             _final_validation_row(
2279                 final_trajectory,
2280                 q3=q3,
2281                 params=params,
2282                 optimizer_result=final_result,
2283                 gate_config=gate_config,
2284                 continuation_summaries=continuation_summaries,
2285                 multi_initial_summaries=multi_initial_summaries,
2286             )
2287         ]
2288     )

```

```

2287     diagnostics = pd.DataFrame(continuation_summaries + multi_initial_summaries)
2288     return final_trajectory, results, validation, diagnostics
2289
2290
2291 def _candidate_final_fuel_workflow(
2292     gate1: pd.DataFrame,
2293     *,
2294     q3: Q3Config,
2295     params: Q2Parameters,
2296     nodes: int,
2297     continuation_nodes: list[int],
2298     gate_config: dict,
2299 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
2300     node_sequence = continuation_nodes or [nodes]
2301     if node_sequence[-1] != nodes:
2302         node_sequence.append(nodes)
2303     summaries: list[dict[str, float | str | bool]] = []
2304     final_trajectory: pd.DataFrame | None = None
2305     final_result: object = type(
2306         "CandidateResult",
2307         (),
2308         {"success": True, "message": "candidate_from_gate2_feasible_trajectory",
2309          "nit": 0},
2309     )()
2310     for node_count in node_sequence:
2311         trajectory, gate_summary = _formal_gate_case(
2312             gate1,
2313             q3=q3,
2314             params=params,
2315             nodes=node_count,
2316             gate_config=gate_config,
2317             maxiter=180 if node_count <= 121 else 220,
2318         )
2319         summary = _final_case_summary(
2320             trajectory,
2321             q3=q3,
2322             params=params,
2323             optimizer_result=final_result,
2324             continuation_nodes=node_sequence,
2325         )
2326         summary["initial_guess"] = "gate2_candidate"
2327         summary["candidate_source_status"] = str(gate_summary["solver_status"])
2328         summaries.append(summary)
2329         final_trajectory = trajectory

```

```

2330
2331     if final_trajectory is None:
2332         raise RuntimeError("candidate final workflow did not produce a
2333         trajectory")
2334     result_summary = _final_case_summary(
2335         final_trajectory,
2336         q3=q3,
2337         params=params,
2338         optimizer_result=final_result,
2339         continuation_nodes=node_sequence,
2340     )
2341     result_summary["initial_guess"] = "gate2_candidate"
2342     result_summary["candidate_note"] = "Full final-fuel SLSQP was too slow for
2343     N=241; this table records the Gate 2 feasible candidate under the final
2344     validation contract."
2345     results = pd.DataFrame([result_summary])
2346     validation = pd.DataFrame(
2347         [
2348             _final_validation_row(
2349                 final_trajectory,
2350                 q3=q3,
2351                 params=params,
2352                 optimizer_result=final_result,
2353                 gate_config=gate_config,
2354                 continuation_summaries=summaries,
2355                 multi_initial_summaries=summaries,
2356             )
2357         ]
2358     )
2359     validation.loc[:, "validation_status"] =
2360     "failed_final_optimizer_not_completed"
2361     validation.loc[:, "failure_reason"] = (
2362         "candidate_from_gate2_feasible_trajectory; final fuel objective was not
2363         solved to the q3-T08 acceptance contract"
2364     )
2365     diagnostics = pd.DataFrame(summaries)
2366     return final_trajectory, results, validation, diagnostics
2367
2368 def _hmax_sensitivity(trajjectory: pd.DataFrame, hmax_values: list[float], q3:
2369     Q3Config) -> pd.DataFrame:
2370     rows = []
2371     for hmax in hmax_values:
2372         max_height = float(trajjectory["height_m"].max())

```

```

2368     final_mass = float(trajjectory["mass_kg"].iloc[-1])
2369     rows.append(
2370         {
2371             "h_max_m": hmax,
2372             "warm_start_max_height_m": max_height,
2373             "warm_start_height_violation_m": max(0.0, max_height - hmax),
2374             "warm_start_terminal_mass_shortfall_kg": max(0.0,
2375 q3.terminal_mass_min_kg - final_mass),
2376             "status": "warm_start_only_not_optimized",
2377         }
2378     )
2379     return pd.DataFrame(rows)
2380
2381 def _projection_audit(
2382     *,
2383     gate1: pd.DataFrame,
2384     q3: Q3Config,
2385     params: Q2Parameters,
2386     nodes: int,
2387     c1_trajectory: pd.DataFrame,
2388 ) -> pd.DataFrame:
2389     original_final_mass = float(gate1["mass_kg"].iloc[-1]) if "mass_kg" in
2390 gate1.columns else float("nan")
2391     original_final_time = float(gate1["time_s"].iloc[-1]) if "time_s" in
2392 gate1.columns else float("nan")
2393     original_final_height = float(gate1["height_m"].iloc[-1])
2394     original_final_speed = float(gate1["airspeed_mps"].iloc[-1])
2395
2396     layered_projection = _project_gate1_warm_start(
2397         gate1,
2398         q3=q3,
2399         params=params,
2400         nodes=nodes,
2401         atmosphere_model=layered_atmosphere,
2402     )
2403     layered_final = layered_projection.iloc[-1]
2404     c1_final = c1_trajectory.iloc[-1]
2405     rows = [
2406         {
2407             "scenario": "A_gate1_original",
2408             "atmosphere": "layered_isa",
2409             "control_trajectory": "gate1_original",
2410             "integration_grid": "gate1_original",

```

```

2409         "nodes": int(len(gate1)),
2410         "terminal_mass_kg": original_final_mass,
2411         "terminal_mass_shortfall_kg": max(0.0, q3.terminal_mass_min_kg -
original_final_mass),
2412         "final_time_s": original_final_time,
2413         "terminal_height_m": original_final_height,
2414         "terminal_airspeed_mps": original_final_speed,
2415         "mass_difference_from_previous_kg": 0.0,
2416     },
2417     {
2418         "scenario": "B_gate1_interpolated_gate2_grid_original_atmosphere",
2419         "atmosphere": "layered_isa",
2420         "control_trajectory": "gate1_interpolated",
2421         "integration_grid": "gate2_uniform_grid",
2422         "nodes": nodes,
2423         "terminal_mass_kg": float(layered_final["mass_kg"]),
2424         "terminal_mass_shortfall_kg": max(0.0, q3.terminal_mass_min_kg -
float(layered_final["mass_kg"])),
2425         "final_time_s": float(layered_final["time_s"]),
2426         "terminal_height_m": float(layered_final["height_m"]),
2427         "terminal_airspeed_mps": float(layered_final["airspeed_mps"]),
2428         "mass_difference_from_previous_kg": original_final_mass -
float(layered_final["mass_kg"]),
2429     },
2430     {
2431         "scenario": "C_gate1_interpolated_gate2_grid_c1_atmosphere",
2432         "atmosphere": "C1_temperature_hydrostatic_pressure",
2433         "control_trajectory": "gate1_interpolated",
2434         "integration_grid": "gate2_uniform_grid",
2435         "nodes": nodes,
2436         "terminal_mass_kg": float(c1_final["mass_kg"]),
2437         "terminal_mass_shortfall_kg": max(0.0, q3.terminal_mass_min_kg -
float(c1_final["mass_kg"])),
2438         "final_time_s": float(c1_final["time_s"]),
2439         "terminal_height_m": float(c1_final["height_m"]),
2440         "terminal_airspeed_mps": float(c1_final["airspeed_mps"]),
2441         "mass_difference_from_previous_kg": float(layered_final["mass_kg"]) -
float(c1_final["mass_kg"]),
2442     },
2443 ]
2444     return pd.DataFrame(rows)
2445
2446
2447 def _atmosphere_coupling_diagnostics(params: Q2Parameters) -> pd.DataFrame:

```

```

2448 state = {
2449     "height_m": 11_000.0,
2450     "airspeed_mps": 235.0,
2451     "mass_kg": 67_000.0,
2452     "thrust_n": 50_000.0,
2453     "gamma_rad": 0.0,
2454 }
2455 layered = _rates(**state, params=params, atmosphere_model=layered_atmosphere)
2456 c1 = _rates(**state, params=params, atmosphere_model=atmosphere)
2457 desired_dv_dx = 5.0e-4
2458 fuel_penalty = 1.0 + params.beta_s2pm2 * (state["airspeed_mps"] -
params.v_opt_mps) ** 2
2459
2460 def required_thrust_row(rates: dict[str, float]) -> tuple[float, float]:
2461     required_thrust = rates["drag_n"] + state["mass_kg"] * (
2462         params.g_mps2 * math.sin(state["gamma_rad"])) +
rates["groundspeed_mps"] * desired_dv_dx
2463     )
2464     dm_dx = -params.c_t_kg_per_ns * required_thrust * fuel_penalty /
rates["groundspeed_mps"]
2465     return required_thrust, dm_dx
2466
2467 layered_required_thrust, layered_required_dm_dx = required_thrust_row(layered)
2468 c1_required_thrust, c1_required_dm_dx = required_thrust_row(c1)
2469 return pd.DataFrame(
2470     [
2471         {
2472             **state,
2473             "required_dV_dx_per_m": desired_dv_dx,
2474             "layer_density_kgm3": layered["density_kgm3"],
2475             "c1_density_kgm3": c1["density_kgm3"],
2476             "density_delta_c1_minus_layer_kgm3": c1["density_kgm3"] -
layered["density_kgm3"],
2477             "layer_drag_n": layered["drag_n"],
2478             "c1_drag_n": c1["drag_n"],
2479             "drag_delta_c1_minus_layer_n": c1["drag_n"] - layered["drag_n"],
2480             "layer_dV_dx_per_m": layered["dV_dx"],
2481             "c1_dV_dx_per_m": c1["dV_dx"],
2482             "dV_dx_delta_c1_minus_layer_per_m": c1["dV_dx"] -
layered["dV_dx"],
2483             "layer_dm_dx_kgpm": layered["dm_dx"],
2484             "c1_dm_dx_kgpm": c1["dm_dx"],
2485             "dm_dx_delta_c1_minus_layer_kgpm": c1["dm_dx"] - layered["dm_dx"],
2486             "layer_required_thrust_n": layered_required_thrust,

```

```

2487         "c1_required_thrust_n": c1_required_thrust,
2488         "required_thrust_delta_c1_minus_layer_n": c1_required_thrust -
layered_required_thrust,
2489         "layer_required_thrust_dm_dx_kgpm": layered_required_dm_dx,
2490         "c1_required_thrust_dm_dx_kgpm": c1_required_dm_dx,
2491         "required_thrust_dm_dx_delta_c1_minus_layer_kgpm":
c1_required_dm_dx - layered_required_dm_dx,
2492         "interpretation":
"fixed_thrust_mass_rate_is_density_independent_required_thrust_mass_rate_is_not",
2493     }
2494 ]
2495 )
2496
2497
2498 def main() -> int:
2499     args = parse_args()
2500     if args.nodes < 5:
2501         raise ValueError("--nodes must be at least 5")
2502     mesh_nodes = _parse_mesh_nodes(args.mesh_study_nodes)
2503     ode_rtols = _parse_ode_rtols(args.ode_rtols)
2504
2505     root = project_root()
2506     q3 = load_q3_config(root, args.config)
2507     gate_config = _load_gate_config(root, args.config)
2508     params = Q2Parameters(terminal_mass_kg=q3.terminal_mass_min_kg)
2509     gate1 = _gate1_trajectory(root)
2510     qdir = root / "questions" / "q3"
2511
2512     if args.final_fuel:
2513         continuation_nodes = _parse_mesh_nodes(args.continuation_nodes) or
[args.nodes]
2514         multi_initial_guesses =
_parse_initial_guesses(args.multi_initial_guesses) or [args.initial_guess]
2515         if args.final_solver == "slsqp":
2516             trajectory, results, validation, diagnostics =
_solve_final_fuel_workflow(
2517                 gate1,
2518                 q3=q3,
2519                 params=params,
2520                 nodes=args.nodes,
2521                 continuation_nodes=continuation_nodes,
2522                 initial_guess=args.initial_guess,
2523                 multi_initial_guesses=multi_initial_guesses,
2524                 gate_config=gate_config,

```

```

2525         maxiter=args.final_maxiter,
2526     )
2527     elif args.final_solver == "shooting":
2528         trajectory, results, validation, diagnostics =
_solve_final_fuel_shooting_workflow(
2529             gate1,
2530             q3=q3,
2531             params=params,
2532             nodes=args.nodes,
2533             continuation_nodes=continuation_nodes,
2534             initial_guess=args.initial_guess,
2535             multi_initial_guesses=multi_initial_guesses,
2536             gate_config=gate_config,
2537             maxiter=args.final_maxiter,
2538             control_knots=args.shooting_control_knots,
2539         )
2540     else:
2541         trajectory, results, validation, diagnostics =
_candidate_final_fuel_workflow(
2542             gate1,
2543             q3=q3,
2544             params=params,
2545             nodes=args.nodes,
2546             continuation_nodes=continuation_nodes,
2547             gate_config=gate_config,
2548         )
2549     save_table(results, stem="no_wind_final_optimal_results",
question_dir=qdir)
2550     save_table(validation, stem="no_wind_final_optimal_validation",
question_dir=qdir)
2551     save_table(trajectory, stem="no_wind_final_optimal_trajectory",
question_dir=qdir)
2552     save_table(diagnostics, stem="no_wind_final_optimal_diagnostics",
question_dir=qdir)
2553     if args.final_hmax_sensitivity:
2554         if args.final_solver != "shooting":
2555             raise ValueError("--final-hmax-sensitivity currently requires
--final-solver shooting")
2556         final_hmax_values = _parse_float_list(args.final_hmax_values,
option_name="--final-hmax-values") or [
2557             float(x) for x in gate_config["hmax_sensitivity_m"]
2558         ]
2559         save_table(
2560             _final_hmax_sensitivity(

```



```

2561         gate1,
2562         q3=q3,
2563         params=params,
2564         nodes=args.nodes,
2565         continuation_nodes=continuation_nodes,
2566         initial_guess=args.initial_guess,
2567         gate_config=gate_config,
2568         maxiter=max(40, args.final_maxiter // 2),
2569         control_knots=args.shooting_control_knots,
2570         hmax_values=final_hmax_values,
2571     ),
2572     stem="no_wind_final_hmax_sensitivity",
2573     question_dir=qdir,
2574 )
2575 if args.final_idle_thrust_sensitivity:
2576     if args.final_solver != "shooting":
2577         raise ValueError("--final-idle-thrust-sensitivity currently
requires --final-solver shooting")
2578     idle_fractions = (
2579         _parse_float_list(args.final_idle_thrust_fractions,
option_name="--final-idle-thrust-fractions")
2580         or _idle_thrust_sensitivity_fractions(root, args.config)
2581     )
2582     save_table(
2583         _final_idle_thrust_sensitivity(
2584             gate1,
2585             q3=q3,
2586             params=params,
2587             nodes=args.nodes,
2588             continuation_nodes=continuation_nodes,
2589             initial_guess=args.initial_guess,
2590             gate_config=gate_config,
2591             maxiter=max(40, args.final_maxiter // 2),
2592             control_knots=args.shooting_control_knots,
2593             idle_fractions=idle_fractions,
2594             baseline_fuel_used_kg=float(results.iloc[0]["fuel_used_kg"]),
2595         ),
2596         stem="no_wind_final_idle_thrust_sensitivity",
2597         question_dir=qdir,
2598     )
2599 print(results.to_string(index=False))
2600 print(validation.to_string(index=False))
2601 return 0
2602

```

```

2603     if not args.dry_run:
2604         trajectory, summary_row = _formal_gate_case(
2605             gate1,
2606             q3=q3,
2607             params=params,
2608             nodes=args.nodes,
2609             gate_config=gate_config,
2610         )
2611         summary = pd.DataFrame([summary_row])
2612         save_table(summary, stem="no_wind_collocation_formal_gate",
question_dir=qdir)
2613         save_table(trajectory, stem="no_wind_collocation_formal_trajectory",
question_dir=qdir)
2614         if not args.skip_hmax_sensitivity:
2615             save_table(
2616                 _optimized_hmax_sensitivity(
2617                     gate1,
2618                     q3=q3,
2619                     params=params,
2620                     nodes=args.nodes,
2621                     hmax_values=[float(x) for x in
gate_config["hmax_sensitivity_m"]],
2622                     gate_config=gate_config,
2623                 ),
2624                 stem="optimized_hmax_sensitivity",
2625                 question_dir=qdir,
2626             )
2627         if mesh_nodes:
2628             save_table(
2629                 _mesh_convergence_study(
2630                     gate1,
2631                     q3=q3,
2632                     params=params,
2633                     nodes_list=mesh_nodes,
2634                     gate_config=gate_config,
2635                     precomputed_cases={args.nodes: (trajectory, summary_row)},
2636                 ),
2637                 stem="no_wind_collocation_mesh_convergence",
2638                 question_dir=qdir,
2639             )
2640         if ode_rtols:
2641             save_table(
2642                 _reintegration_tolerance_study(trajectory, q3=q3, params=params,
rtols=ode_rtols),

```

```

2643         stem="no_wind_collocation_reintegration_tolerance",
2644         question_dir=qdir,
2645     )
2646     save_table(
2647         _continuous_audit_study(trajjectory, q3=q3, params=params,
rtols=ode_rtols),
2648         stem="no_wind_collocation_continuous_audit",
2649         question_dir=qdir,
2650     )
2651     save_table(
2652         smoothing_diagnostics_table(),
2653         stem="atmosphere_smoothing_diagnostics",
2654         question_dir=qdir,
2655     )
2656     save_table(
2657         _atmosphere_coupling_diagnostics(params),
2658         stem="atmosphere_coupling_diagnostics",
2659         question_dir=qdir,
2660     )
2661     print(summary.to_string(index=False))
2662     return 0
2663
2664 trajectory = _project_gate1_warm_start(gate1, q3=q3, params=params,
nodes=args.nodes)
2665 trajectory["scaled_constraint_violation"] =
trajectory.apply(_scaled_constraint_violation, axis=1, q3=q3)
2666
2667 diagnostics = _collocation_diagnostics(trajectory, q3)
2668 atmosphere_deviation = max_deviation_from_layered_isa()
2669 final = trajectory.iloc[-1]
2670 summary = pd.DataFrame(
2671     [
2672         {
2673             "wind_model": "no_wind",
2674             "method": "range_domain_collocation_readiness_dry_run",
2675             "nodes": args.nodes,
2676             "solver_status": "dry_run_not_optimized",
2677             "atmosphere_model": "C1_temperature_hydrostatic_pressure",
2678             "terminal_mass_kg": float(final["mass_kg"]),
2679             "terminal_mass_shortfall_kg": max(0.0, q3.terminal_mass_min_kg -
float(final["mass_kg"])),
2680             "terminal_height_error_m": float(final["height_m"] -
q3.terminal_height_m),
2681             "terminal_speed_error_mps": float(final["airspeed_mps"] -

```

```

q3.terminal_airspeed_mps),
2682         "final_time_s": float(final["time_s"]),
2683         "max_scaled_constraint_violation":
float(trajjectory["scaled_constraint_violation"].max()),
2684         "constraint_violation_scale": "nondimensional",
2685         **diagnostics,
2686         **atmosphere_deviation,
2687         "slack_smoothing_tolerance_kg":
float(gate_config["slack_smoothing_tolerance_kg"]),
2688     }
2689 ]
2690 )
2691
2692 save_table(summary, stem="no_wind_collocation_gate", question_dir=qdir)
2693 save_table(trajjectory, stem="no_wind_collocation_trajectory",
question_dir=qdir)
2694 hmax_diagnostic = _hmax_sensitivity(trajjectory, [float(x) for x in
gate_config["hmax_sensitivity_m"]], q3)
2695 save_table(hmax_diagnostic, stem="warm_start_hmax_diagnostic",
question_dir=qdir)
2696 save_table(hmax_diagnostic, stem="no_wind_hmax_sensitivity",
question_dir=qdir)
2697 save_table(
2698     smoothing_diagnostics_table(),
2699     stem="atmosphere_smoothing_diagnostics",
2700     question_dir=qdir,
2701 )
2702 save_table(
2703     _projection_audit(gate1=gate1, q3=q3, params=params, nodes=args.nodes,
c1_trajectory=trajjectory),
2704     stem="gate1_to_collocation_projection_audit",
2705     question_dir=qdir,
2706 )
2707 save_table(
2708     _atmosphere_coupling_diagnostics(params),
2709     stem="atmosphere_coupling_diagnostics",
2710     question_dir=qdir,
2711 )
2712 print(summary.to_string(index=False))
2713 return 0
2714
2715
2716 if __name__ == "__main__":
2717     raise SystemExit(main())

```

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import math
6  import sys
7  from dataclasses import dataclass, replace
8  from pathlib import Path
9
10 import numpy as np
11 import pandas as pd
12 from scipy.integrate import solve_ivp
13 from scipy.optimize import minimize
14
15 ROOT = Path(__file__).resolve().parents[3]
16 SRC = ROOT / "src"
17 Q2_SCRIPTS = ROOT / "questions" / "q2" / "scripts"
18 Q3_SCRIPTS = ROOT / "questions" / "q3" / "scripts"
19 for search_path in [SRC, Q2_SCRIPTS, Q3_SCRIPTS]:
20     if str(search_path) not in sys.path:
21         sys.path.insert(0, str(search_path))
22
23 from fuel_path_model import Q2Parameters, atmosphere, wind_speed_mps # noqa: E402
24 from modeling_common.artifacts import save_table # noqa: E402
25 from modeling_common.paths import project_root # noqa: E402
26 from solve_feasibility_no_wind import Q3Config, load_q3_config # noqa: E402
27
28 H_SCALE_M = 1000.0
29 V_SCALE_MPS = 100.0
30 M_SCALE_KG = 10_000.0
31 T_SCALE_N = 90_000.0
32
33
34 @dataclass(frozen=True)
35 class WindEvaluation:
36     trajectory: pd.DataFrame
37     terminal_height_error_m: float
38     terminal_speed_error_mps: float
39     terminal_mass_shortfall_kg: float
40     max_scaled_constraint_violation: float
41     fuel_used_kg: float
42     final_time_s: float
43

```

```

44
45 def parse_args() -> argparse.Namespace:
46     parser = argparse.ArgumentParser(description="q4 wind-aware integrated cruise
47     strategy pipeline")
48     parser.add_argument("--config", default="configs/default.yaml")
49     parser.add_argument("--dry-run", action="store_true")
50     parser.add_argument("--nodes", type=int, default=121)
51     parser.add_argument("--control-knots", type=int, default=9)
52     parser.add_argument("--maxiter", type=int, default=120)
53     parser.add_argument("--beta-factors", default="0.8,0.9,1.0,1.1,1.2")
54     parser.add_argument("--range-factors", default="1.0,1.03,1.06")
55     parser.add_argument("--sensitivity-maxiter", type=int, default=80)
56     parser.add_argument("--range-maxiter", type=int, default=80)
57     parser.add_argument("--skip-beta-sensitivity", action="store_true")
58     parser.add_argument("--skip-range-extension", action="store_true")
59     parser.add_argument("--skip-extension-frameworks", action="store_true")
60     return parser.parse_args()
61
62 def _parse_float_list(raw: str, *, option_name: str) -> list[float]:
63     values: list[float] = []
64     for part in raw.split(","):
65         text = part.strip()
66         if not text:
67             continue
68         try:
69             values.append(float(text))
70         except ValueError as exc:
71             raise ValueError(f"{option_name} contains a non-numeric value:
72             {text}") from exc
73     if not values:
74         raise ValueError(f"{option_name} must contain at least one value")
75     return values
76
77 def _load_no_wind_trajectory(root: Path) -> pd.DataFrame:
78     path = root / "questions" / "q3" / "artifacts" / "tables" /
79     "no_wind_final_optimal_trajectory.csv"
80     if not path.exists():
81         raise FileNotFoundError(
82             "q4-T02 requires
83             questions/q3/artifacts/tables/no_wind_final_optimal_trajectory.csv"
84         )
85     frame =

```

```

84 pd.read_csv(path).sort_values("distance_m").drop_duplicates("distance_m")
    required = {"distance_m", "height_m", "airspeed_mps", "mass_kg", "thrust_n",
"gamma_rad"}
85 missing = required.difference(frame.columns)
86 if missing:
87     raise ValueError(f"q3 no-wind trajectory missing required columns:
{sorted(missing)}")
88 return frame
89
90
91 def _load_q2_standard_baseline(root: Path) -> pd.Series:
92     path = root / "artifacts" / "q2" / "data" / "q2_fuel_summary.csv"
93     if not path.exists():
94         raise FileNotFoundError("q4-T03 requires
artifacts/q2/data/q2_fuel_summary.csv")
95     summary = pd.read_csv(path)
96     row = summary.loc[summary["scenario"] == "standard_isa"]
97     if row.empty:
98         raise ValueError("q2 fuel summary does not contain scenario=standard_isa")
99     return row.iloc[0]
100
101
102 def _load_q1_constant_speed_summary(root: Path) -> pd.Series:
103     path = root / "artifacts" / "q1" / "data" / "strategy_comparison.csv"
104     if not path.exists():
105         raise FileNotFoundError("q4-T03 requires
artifacts/q1/data/strategy_comparison.csv")
106     summary = pd.read_csv(path)
107     row = summary.loc[summary["strategy"] == "constant_speed"]
108     if row.empty:
109         raise ValueError("q1 strategy comparison does not contain constant_speed")
110     return row.iloc[0]
111
112
113 def _load_q1_constant_speed_profile(root: Path) -> pd.DataFrame:
114     path = root / "artifacts" / "q1" / "data" / "constant_speed_profile.csv"
115     if not path.exists():
116         raise FileNotFoundError("q4-T06 requires
artifacts/q1/data/constant_speed_profile.csv")
117     profile =
pd.read_csv(path).sort_values("distance_m").drop_duplicates("distance_m")
118     required = {"distance_m", "height_m", "airspeed_mps"}
119     missing = required.difference(profile.columns)
120     if missing:

```

```

121         raise ValueError(f"q1 constant-speed profile missing required columns:
122         {sorted(missing)}")
123     return profile
124
125 def _interp(frame: pd.DataFrame, column: str, distance_m: np.ndarray | float) ->
126     np.ndarray:
127     return np.interp(distance_m, frame["distance_m"].to_numpy(dtype=float),
128         frame[column].to_numpy(dtype=float))
129
130 def _height_from_q1_profile(profile: pd.DataFrame, distance_m: float) -> float:
131     return float(np.interp(distance_m,
132         profile["distance_m"].to_numpy(dtype=float),
133         profile["height_m"].to_numpy(dtype=float)))
134
135 def _solve_gamma_for_wind_path_slope(airspeed_mps: float, wind_mps: float, dh_dx:
136     float) -> float:
137     gamma = math.atan(dh_dx)
138     for _ in range(12):
139         denominator = airspeed_mps * math.cos(gamma) + wind_mps
140         value = airspeed_mps * math.sin(gamma) / denominator - dh_dx
141         derivative = (
142             airspeed_mps
143             * (airspeed_mps + wind_mps * math.cos(gamma))
144             / max(denominator * denominator, 1.0e-12)
145         )
146         gamma -= value / max(derivative, 1.0e-12)
147     return gamma
148
149 def _initial_controls_from_no_wind(
150     source: pd.DataFrame,
151     *,
152     q3: Q3Config,
153     params: Q2Parameters,
154     control_knots: int,
155 ) -> np.ndarray:
156     distances = np.linspace(0.0, q3.target_distance_m, control_knots)
157     source_distance = source["distance_m"].to_numpy(dtype=float)
158     height = _interp(source, "height_m", distances)
159     airspeed = _interp(source, "airspeed_mps", distances)
160     mass = _interp(source, "mass_kg", distances)

```



```

159 dh_dx = np.gradient(source["height_m"].to_numpy(dtype=float), source_distance)
160 dV_dx = np.gradient(source["airspeed_mps"].to_numpy(dtype=float),
161 source_distance)
162 source_dh_dx = np.interp(distances, source_distance, dh_dx)
163 source_dv_dx = np.interp(distances, source_distance, dV_dx)
164
165 thrust = np.zeros(control_knots)
166 gamma = np.zeros(control_knots)
167 for index in range(control_knots):
168     gamma[index] = _solve_gamma_for_wind_path_slope(
169         float(airspeed[index]),
170         wind_speed_mps(float(height[index])),
171         float(source_dh_dx[index]),
172     )
173     rates = _rates(
174         mass_kg=float(mass[index]),
175         height_m=float(height[index]),
176         airspeed_mps=float(airspeed[index]),
177         thrust_n=float(_interp(source, "thrust_n", distances[index])),
178         gamma_rad=float(gamma[index]),
179         params=params,
180         q3=q3,
181     )
182     thrust[index] = (
183         float(mass[index])
184         * (rates["groundspeed_mps"] * float(source_dv_dx[index]) +
185         params.g_mps2 * math.sin(float(gamma[index])))
186         + rates["drag_n"]
187     )
188     thrust = np.clip(thrust, q3.thrust_min_n + 1.0, q3.thrust_max_n - 1.0)
189     gamma = np.clip(gamma, -q3.gamma_max_rad + 1.0e-5, q3.gamma_max_rad - 1.0e-5)
190     return _pack_controls(thrust, gamma)
191
192 def _pack_controls(thrust_n: np.ndarray, gamma_rad: np.ndarray) -> np.ndarray:
193     return np.concatenate([thrust_n / T_SCALE_N, gamma_rad / max(1.0e-9,
194 0.05236)])
195
196 def _unpack_controls(vector: np.ndarray, control_knots: int) -> tuple[np.ndarray,
197 np.ndarray]:
198     thrust = vector[:control_knots] * T_SCALE_N
199     gamma = vector[control_knots:] * 0.05236
200     return thrust, gamma

```

```

199
200
201 def _control_bounds(control_knots: int, q3: Q3Config) -> list[tuple[float,
202     float]]:
203     thrust_bounds = [(q3.thrust_min_n / T_SCALE_N, q3.thrust_max_n / T_SCALE_N)]
204     * control_knots
205     gamma_bounds = [(-q3.gamma_max_rad / 0.05236, q3.gamma_max_rad / 0.05236)] *
206     control_knots
207     return thrust_bounds + gamma_bounds
208
209 def _rates(
210     *,
211     mass_kg: float,
212     height_m: float,
213     airspeed_mps: float,
214     thrust_n: float,
215     gamma_rad: float,
216     params: Q2Parameters,
217     q3: Q3Config,
218 ) -> dict[str, float]:
219     temperature_k, density_kgm3, sound_speed_mps, pressure_pa =
220     atmosphere(height_m)
221     lift_factor = max(math.cos(gamma_rad), 0.0)
222     cl = (
223         2.0
224         * mass_kg
225         * params.g_mps2
226         * lift_factor
227         / (density_kgm3 * airspeed_mps**2 * params.wing_area_m2)
228     )
229     cd = params.cd0 + params.induced_k * cl**2
230     drag_n = 0.5 * density_kgm3 * airspeed_mps**2 * params.wing_area_m2 * cd
231     wind_mps = wind_speed_mps(height_m)
232     groundspeed_mps = airspeed_mps * math.cos(gamma_rad) + wind_mps
233     if groundspeed_mps <= 0.0:
234         raise ValueError("non-positive configured-wind ground speed")
235     fuel_penalty = 1.0 + params.beta_s2pm2 * (airspeed_mps - params.v_opt_mps) **
236     2
237     mach = airspeed_mps / sound_speed_mps
238     dh_dx = airspeed_mps * math.sin(gamma_rad) / groundspeed_mps
239     dV_dx = ((thrust_n - drag_n) / mass_kg - params.g_mps2 * math.sin(gamma_rad))
240     / groundspeed_mps
241     dm_dx = -params.c_t_kg_per_ns * thrust_n * fuel_penalty / groundspeed_mps

```

```

237 dt_dx = 1.0 / groundspeed_mps
238 violation = max(
239     (q3.h_min_m - height_m) / H_SCALE_M,
240     (height_m - q3.h_max_m) / H_SCALE_M,
241     (q3.v_min_mps - airspeed_mps) / V_SCALE_MPS,
242     (airspeed_mps - q3.v_max_mps) / V_SCALE_MPS,
243     mach - q3.mach_max,
244     (q3.thrust_min_n - thrust_n) / T_SCALE_N,
245     (thrust_n - q3.thrust_max_n) / T_SCALE_N,
246     abs(gamma_rad) - q3.gamma_max_rad,
247     (q3.terminal_mass_min_kg - mass_kg) / M_SCALE_KG,
248     0.0,
249 )
250 return {
251     "temperature_k": temperature_k,
252     "density_kgm3": density_kgm3,
253     "sound_speed_mps": sound_speed_mps,
254     "pressure_pa": pressure_pa,
255     "wind_mps": wind_mps,
256     "groundspeed_mps": groundspeed_mps,
257     "mach": mach,
258     "cl": cl,
259     "cd": cd,
260     "drag_n": drag_n,
261     "dh_dx": dh_dx,
262     "dV_dx": dV_dx,
263     "dm_dx": dm_dx,
264     "dt_dx": dt_dx,
265     "fuel_per_meter_kgpm": -dm_dx,
266     "scaled_constraint_violation": violation,
267 }
268
269
270 def _evaluate_controls(
271     vector: np.ndarray,
272     *,
273     q3: Q3Config,
274     params: Q2Parameters,
275     control_knots: int,
276     nodes: int,
277 ) -> WindEvaluation:
278     thrust_knots, gamma_knots = _unpack_controls(vector, control_knots)
279     control_distance = np.linspace(0.0, q3.target_distance_m, control_knots)
280     output_distance = np.linspace(0.0, q3.target_distance_m, nodes)

```

```

281
282 def controls(distance_m: float) -> tuple[float, float]:
283     return (
284         float(np.interp(distance_m, control_distance, thrust_knots)),
285         float(np.interp(distance_m, control_distance, gamma_knots)),
286     )
287
288 def rhs(distance_m: float, state: np.ndarray) -> list[float]:
289     thrust_n, gamma_rad = controls(distance_m)
290     rates = _rates(
291         mass_kg=float(state[2]),
292         height_m=float(state[0]),
293         airspeed_mps=float(state[1]),
294         thrust_n=thrust_n,
295         gamma_rad=gamma_rad,
296         params=params,
297         q3=q3,
298     )
299     return [rates["dh_dx"], rates["dV_dx"], rates["dm_dx"], rates["dt_dx"]]
300
301 solution = solve_ivp(
302     rhs,
303     (0.0, q3.target_distance_m),
304     np.array([params.h0_m, params.v0_mps, params.m0_kg, 0.0]),
305     t_eval=output_distance,
306     rtol=1.0e-8,
307     atol=np.array([1.0e-5, 1.0e-7, 1.0e-4, 1.0e-6]),
308     max_step=q3.target_distance_m / max(nodes - 1, 1),
309 )
310 if not solution.success:
311     raise RuntimeError(solution.message)
312
313 records: list[dict[str, float]] = []
314 max_violation = 0.0
315 for index, distance_m in enumerate(output_distance):
316     thrust_n, gamma_rad = controls(float(distance_m))
317     height_m = float(solution.y[0, index])
318     airspeed_mps = float(solution.y[1, index])
319     mass_kg = float(solution.y[2, index])
320     rates = _rates(
321         mass_kg=mass_kg,
322         height_m=height_m,
323         airspeed_mps=airspeed_mps,
324         thrust_n=thrust_n,

```

```

325         gamma_rad=gamma_rad,
326         params=params,
327         q3=q3,
328     )
329     max_violation = max(max_violation, rates["scaled_constraint_violation"])
330     records.append(
331         {
332             "distance_m": float(distance_m),
333             "height_m": height_m,
334             "airspeed_mps": airspeed_mps,
335             "mass_kg": mass_kg,
336             "time_s": float(solution.y[3, index]),
337             "thrust_n": thrust_n,
338             "gamma_rad": gamma_rad,
339             **rates,
340         }
341     )
342     trajectory = pd.DataFrame.from_records(records)
343     final = trajectory.iloc[-1]
344     return WindEvaluation(
345         trajectory=trajectory,
346         terminal_height_error_m=float(final["height_m"] - q3.terminal_height_m),
347         terminal_speed_error_mps=float(final["airspeed_mps"] -
348 q3.terminal_airspeed_mps),
349         terminal_mass_shortfall_kg=max(0.0, q3.terminal_mass_min_kg -
350 float(final["mass_kg"])),
351         max_scaled_constraint_violation=max_violation,
352         fuel_used_kg=params.m0_kg - float(final["mass_kg"]),
353         final_time_s=float(final["time_s"]),
354     )
355
356 def _evaluate_or_none(
357     vector: np.ndarray,
358     *,
359     q3: Q3Config,
360     params: Q2Parameters,
361     control_knots: int,
362     nodes: int,
363 ) -> WindEvaluation | None:
364     try:
365         return _evaluate_controls(vector, q3=q3, params=params,
366 control_knots=control_knots, nodes=nodes)
367     except (RuntimeError, ValueError, FloatingPointError):

```

```

366         return None
367
368
369 def _fuel_identity_residual(trajecory: pd.DataFrame, params: Q2Parameters) ->
float:
370     fuel_integral = float(np.trapezoid(trajecory["fuel_per_meter_kgpm"],
trajecory["distance_m"]))
371     mass_loss = params.m0_kg - float(trajecory["mass_kg"].iloc[-1])
372     return abs(mass_loss - fuel_integral)
373
374
375 def _validation_status(
376     evaluation: WindEvaluation,
377     *,
378     fuel_identity_residual_kg: float,
379     optimizer_success: bool,
380 ) -> str:
381     checks = [
382         abs(evaluation.terminal_height_error_m) <= 0.1,
383         abs(evaluation.terminal_speed_error_mps) <= 1.0e-3,
384         evaluation.terminal_mass_shortfall_kg <= 0.05,
385         evaluation.max_scaled_constraint_violation <= 1.0e-6,
386         fuel_identity_residual_kg <= 0.1,
387     ]
388     return "passed" if all(checks) else "failed"
389
390
391 def _solve_wind_optimal(
392     *,
393     q3: Q3Config,
394     params: Q2Parameters,
395     source: pd.DataFrame,
396     nodes: int,
397     control_knots: int,
398     maxiter: int,
399     initial_vector: np.ndarray | None = None,
400 ) -> tuple[WindEvaluation, object, np.ndarray]:
401     initial = (
402         np.asarray(initial_vector, dtype=float)
403         if initial_vector is not None
404         else _initial_controls_from_no_wind(source, q3=q3, params=params,
control_knots=control_knots)
405     )
406     cache: dict[tuple[float, ...], WindEvaluation | None] = {}

```

```

407
408 def evaluation(vector: np.ndarray) -> WindEvaluation | None:
409     key = tuple(np.round(vector, 10))
410     if key not in cache:
411         cache[key] = _evaluate_or_none(vector, q3=q3, params=params,
control_knots=control_knots, nodes=nodes)
412     return cache[key]
413
414 def objective(vector: np.ndarray) -> float:
415     current = evaluation(vector)
416     if current is None:
417         return 1.0e6
418     thrust, gamma = _unpack_controls(vector, control_knots)
419     smooth = 1.0e-7 * float(np.sum(np.diff(thrust) ** 2)) + 1.0e-2 *
float(np.sum(np.diff(gamma) ** 2))
420     return current.fuel_used_kg / 1000.0 + smooth
421
422 def terminal_height_eq(vector: np.ndarray) -> float:
423     current = evaluation(vector)
424     return -1.0 if current is None else current.terminal_height_error_m /
H_SCALE_M
425
426 def terminal_speed_eq(vector: np.ndarray) -> float:
427     current = evaluation(vector)
428     return -1.0 if current is None else current.terminal_speed_error_mps /
V_SCALE_MPS
429
430 def path_ineq(vector: np.ndarray) -> np.ndarray:
431     current = evaluation(vector)
432     if current is None:
433         return np.array([-1.0, -1.0])
434     return np.array(
435         [
436             (params.m0_kg - current.fuel_used_kg - q3.terminal_mass_min_kg) /
M_SCALE_KG,
437             -current.max_scaled_constraint_violation,
438         ]
439     )
440
441 result = minimize(
442     objective,
443     initial,
444     method="SLSQP",
445     bounds=_control_bounds(control_knots, q3),

```

```

446     constraints=[
447         {"type": "eq", "fun": terminal_height_eq},
448         {"type": "eq", "fun": terminal_speed_eq},
449         {"type": "ineq", "fun": path_ineq},
450     ],
451     options={"maxiter": maxiter, "ftol": 1.0e-9, "disp": False},
452 )
453 final_vector = np.asarray(result.x if result.x is not None else initial,
454 dtype=float)
455 final = _evaluate_controls(final_vector, q3=q3, params=params,
456 control_knots=control_knots, nodes=nodes)
457 return final, result, final_vector
458
459 def _params_for_beta(*, q3: Q3Config, beta_s2pm2: float) -> Q2Parameters:
460     return Q2Parameters(terminal_mass_kg=q3.terminal_mass_min_kg,
461 beta_s2pm2=beta_s2pm2)
462
463 def _wind_result_row(
464     evaluation: WindEvaluation,
465     *,
466     q3: Q3Config,
467     params: Q2Parameters,
468     optimizer_result: object,
469     continuation_nodes: str,
470     control_knots: int,
471     fuel_identity_residual_kg: float,
472 ) -> pd.DataFrame:
473     trajectory = evaluation.trajectory
474     optimizer_success = bool(getattr(optimizer_result, "success", False))
475     final_mass = params.m0_kg - evaluation.fuel_used_kg
476     row = {
477         "artifact_id": "q4-T02",
478         "wind_model": "configured_wind",
479         "method": "range_domain_reduced_control_shooting_wind_optimization",
480         "objective": "min_m0_minus_mf",
481         "claim_level": "locally_optimized_solution",
482         "mass_constraint_policy": "m_f_ge_62000",
483         "continuation_nodes": continuation_nodes,
484         "final_nodes": int(len(trajectory)),
485         "shooting_control_knots": int(control_knots),
486         "target_distance_m": q3.target_distance_m,
487         "terminal_mass_kg": final_mass,

```



```

487     "terminal_mass_min_kg": q3.terminal_mass_min_kg,
488     "terminal_mass_shortfall_kg": evaluation.terminal_mass_shortfall_kg,
489     "fuel_used_kg": evaluation.fuel_used_kg,
490     "final_time_s": evaluation.final_time_s,
491     "terminal_height_m": float(trajectory["height_m"].iloc[-1]),
492     "terminal_height_error_signed_m": evaluation.terminal_height_error_m,
493     "terminal_height_error_m": abs(evaluation.terminal_height_error_m),
494     "terminal_airspeed_mps": float(trajectory["airspeed_mps"].iloc[-1]),
495     "terminal_speed_error_signed_mps": evaluation.terminal_speed_error_mps,
496     "terminal_speed_error_mps": abs(evaluation.terminal_speed_error_mps),
497     "max_height_m": float(trajectory["height_m"].max()),
498     "min_height_m": float(trajectory["height_m"].min()),
499     "min_airspeed_mps": float(trajectory["airspeed_mps"].min()),
500     "max_airspeed_mps": float(trajectory["airspeed_mps"].max()),
501     "min_thrust_n": float(trajectory["thrust_n"].min()),
502     "max_thrust_n": float(trajectory["thrust_n"].max()),
503     "max_mach": float(trajectory["mach"].max()),
504     "max_scaled_constraint_violation":
evaluation.max_scaled_constraint_violation,
505     "fuel_identity_residual_kg": fuel_identity_residual_kg,
506     "near_zero_thrust_fraction": float((trajectory["thrust_n"] <= 1.0e-3 *
q3.thrust_max_n).mean()),
507     "optimizer_success": optimizer_success,
508     "optimizer_message": str(getattr(optimizer_result, "message", "")),
509     "optimizer_iterations": int(getattr(optimizer_result, "nit", -1)),
510 }
511 row["validation_status"] = _validation_status(
512     evaluation,
513     fuel_identity_residual_kg=fuel_identity_residual_kg,
514     optimizer_success=optimizer_success,
515 )
516 return pd.DataFrame([row])
517
518
519 def _strategy_comparison(
520     *,
521     q2_baseline: pd.Series,
522     q1_constant_speed: pd.Series,
523     wind_results: pd.DataFrame,
524 ) -> pd.DataFrame:
525     optimal = wind_results.iloc[0]
526     fixed_range = float(q2_baseline["final_distance_m"])
527     q1_full_range = float(q1_constant_speed["final_distance_m"])
528     baseline_fuel = float(q2_baseline["fuel_used_kg"])

```

```

529 optimal_fuel = float(optimal["fuel_used_kg"])
530 fuel_saving = baseline_fuel - optimal_fuel
531 common = {
532     "artifact_id": "q4-T03",
533     "fixed_range_m": fixed_range,
534     "q1_constant_speed_full_range_m": q1_full_range,
535     "range_change_vs_q1_constant_speed_m": fixed_range - q1_full_range,
536 }
537 return pd.DataFrame(
538     [
539         {
540             **common,
541             "strategy": "constant_speed_baseline",
542             "source": "artifacts/q2/data/q2_fuel_summary.csv:standard_isa",
543             "fuel_used_kg": baseline_fuel,
544             "terminal_mass_kg": float(q2_baseline["final_mass_kg"]),
545             "final_time_s": float(q2_baseline["final_time_s"]),
546             "fuel_saving_vs_baseline_kg": 0.0,
547             "fuel_saving_vs_baseline_pct": 0.0,
548             "comparison_status": "baseline",
549         },
550         {
551             **common,
552             "strategy": "configured_wind_optimal",
553             "source":
554                 "questions/q4/artifacts/tables/wind_optimal_results.csv",
555             "fuel_used_kg": optimal_fuel,
556             "terminal_mass_kg": float(optimal["terminal_mass_kg"]),
557             "final_time_s": float(optimal["final_time_s"]),
558             "fuel_saving_vs_baseline_kg": fuel_saving,
559             "fuel_saving_vs_baseline_pct": 100.0 * fuel_saving /
560             baseline_fuel,
561             "comparison_status": str(optimal["validation_status"]),
562         },
563     ]
564 )
565
566 def _beta_sensitivity(
567     *,
568     q3: Q3Config,
569     nominal_params: Q2Parameters,
570     source: pd.DataFrame,
571     nominal_evaluation: WindEvaluation,

```

```

571     nominal_optimizer_result: object,
572     nominal_vector: np.ndarray,
573     nodes: int,
574     control_knots: int,
575     maxiter: int,
576     beta_factors: list[float],
577 ) -> pd.DataFrame:
578     rows: list[dict[str, float | int | str | bool]] = []
579     nominal_beta = nominal_params.beta_s2pm2
580     nominal_fuel = nominal_evaluation.fuel_used_kg
581
582     for factor in beta_factors:
583         beta_value = nominal_beta * factor
584         params = _params_for_beta(q3=q3, beta_s2pm2=beta_value)
585         if abs(factor - 1.0) <= 1.0e-12:
586             evaluation = nominal_evaluation
587             optimizer_result = nominal_optimizer_result
588             final_vector = nominal_vector
589         else:
590             evaluation, optimizer_result, final_vector = _solve_wind_optimal(
591                 q3=q3,
592                 params=params,
593                 source=source,
594                 nodes=nodes,
595                 control_knots=control_knots,
596                 maxiter=maxiter,
597                 initial_vector=nominal_vector,
598             )
599             validation_evaluation = _evaluate_controls(
600                 final_vector,
601                 q3=q3,
602                 params=params,
603                 control_knots=control_knots,
604                 nodes=max(181, nodes * 3),
605             )
606             fuel_identity_residual =
607             _fuel_identity_residual(validation_evaluation.trajectory, params)
608             optimizer_success = bool(getattr(optimizer_result, "success", False))
609             validation_status = _validation_status(
610                 evaluation,
611                 fuel_identity_residual_kg=fuel_identity_residual,
612                 optimizer_success=optimizer_success,
613             )
614             rows.append(

```

```

614         {
615             "artifact_id": "q4-T04",
616             "parameter": "beta_s2pm2",
617             "beta_factor": factor,
618             "beta_s2pm2": beta_value,
619             "nominal_beta_s2pm2": nominal_beta,
620             "perturbation_pct": 100.0 * (factor - 1.0),
621             "reoptimization_performed": True,
622             "post_solution_metric_only": False,
623             "claim_level": "local_reoptimization_sensitivity",
624             "fuel_used_kg": evaluation.fuel_used_kg,
625             "fuel_delta_vs_nominal_kg": evaluation.fuel_used_kg -
nominal_fuel,
626             "fuel_delta_vs_nominal_pct": 100.0 * (evaluation.fuel_used_kg -
nominal_fuel) / nominal_fuel,
627             "terminal_mass_kg": params.m0_kg - evaluation.fuel_used_kg,
628             "final_time_s": evaluation.final_time_s,
629             "terminal_height_error_m":
abs(evaluation.terminal_height_error_m),
630             "terminal_speed_error_mps":
abs(evaluation.terminal_speed_error_mps),
631             "max_scaled_constraint_violation":
evaluation.max_scaled_constraint_violation,
632             "fuel_identity_residual_kg": fuel_identity_residual,
633             "near_zero_thrust_fraction": float(
634                 (evaluation.trajectory["thrust_n"] <= 1.0e-3 *
q3.thrust_max_n).mean()
635             ),
636             "max_height_m": float(evaluation.trajectory["height_m"].max()),
637             "min_airspeed_mps":
float(evaluation.trajectory["airspeed_mps"].min()),
638             "optimizer_success": optimizer_success,
639             "optimizer_iterations": int(getattr(optimizer_result, "nit", -1)),
640             "optimizer_message": str(getattr(optimizer_result, "message",
641 "")),
642             "validation_status": validation_status,
643         }
644     )
645     return pd.DataFrame(rows).sort_values("beta_factor").reset_index(drop=True)
646
647 def _range_extension(
648     *,
649     q3: Q3Config,

```

```

650     params: Q2Parameters,
651     source: pd.DataFrame,
652     q1_profile: pd.DataFrame,
653     nominal_evaluation: WindEvaluation,
654     nominal_optimizer_result: object,
655     nominal_vector: np.ndarray,
656     nodes: int,
657     control_knots: int,
658     maxiter: int,
659     range_factors: list[float],
660 ) -> tuple[pd.DataFrame, pd.DataFrame]:
661     rows: list[dict[str, float | int | str | bool]] = []
662     fixed_fuel_kg = params.m0_kg - q3.terminal_mass_min_kg
663     for factor in range_factors:
664         target_distance = q3.target_distance_m * factor
665         trial_q3 = replace(
666             q3,
667             target_distance_m=target_distance,
668             terminal_height_m=_height_from_q1_profile(q1_profile,
669 target_distance),
670         )
671         if abs(factor - 1.0) <= 1.0e-12:
672             evaluation = nominal_evaluation
673             optimizer_result = nominal_optimizer_result
674             final_vector = nominal_vector
675         else:
676             evaluation, optimizer_result, final_vector = _solve_wind_optimal(
677                 q3=trial_q3,
678                 params=params,
679                 source=source,
680                 nodes=nodes,
681                 control_knots=control_knots,
682                 maxiter=maxiter,
683                 initial_vector=nominal_vector,
684             )
685         validation_evaluation = _evaluate_controls(
686             final_vector,
687             q3=trial_q3,
688             params=params,
689             control_knots=control_knots,
690             nodes=max(181, nodes * 3),
691         )
692     fuel_identity_residual =
        _fuel_identity_residual(validation_evaluation.trajectory, params)

```

```

692     optimizer_success = bool(getattr(optimizer_result, "success", False))
693     validation_status = _validation_status(
694         evaluation,
695         fuel_identity_residual_kg=fuel_identity_residual,
696         optimizer_success=optimizer_success,
697     )
698     rows.append(
699         {
700             "artifact_id": "q4-T06-trial",
701             "range_factor": factor,
702             "target_distance_m": target_distance,
703             "terminal_height_target_m": trial_q3.terminal_height_m,
704             "fuel_budget_kg": fixed_fuel_kg,
705             "fuel_used_kg": evaluation.fuel_used_kg,
706             "fuel_margin_kg": fixed_fuel_kg - evaluation.fuel_used_kg,
707             "terminal_mass_kg": params.m0_kg - evaluation.fuel_used_kg,
708             "final_time_s": evaluation.final_time_s,
709             "terminal_height_error_m":
710             abs(evaluation.terminal_height_error_m),
711             "terminal_speed_error_mps":
712             abs(evaluation.terminal_speed_error_mps),
713             "max_scaled_constraint_violation":
714             evaluation.max_scaled_constraint_violation,
715             "fuel_identity_residual_kg": fuel_identity_residual,
716             "optimizer_success": optimizer_success,
717             "optimizer_iterations": int(getattr(optimizer_result, "nit", -1)),
718             "optimizer_message": str(getattr(optimizer_result, "message",
719             "")),
720             "validation_status": validation_status,
721             "feasible_under_fuel_budget": bool(
722                 validation_status == "passed" and evaluation.fuel_used_kg <=
723                 fixed_fuel_kg + 0.05
724             ),
725         }
726     )
727     trials =
728     pd.DataFrame(rows).sort_values("target_distance_m").reset_index(drop=True)
729
730     fuel = trials["fuel_used_kg"].to_numpy(dtype=float)
731     distance = trials["target_distance_m"].to_numpy(dtype=float)
732     feasible = trials.loc[trials["feasible_under_fuel_budget"]]
733     if np.any(fuel >= fixed_fuel_kg) and np.any(fuel <= fixed_fuel_kg):
734         order = np.argsort(fuel)
735         estimated_range = float(np.interp(fixed_fuel_kg, fuel[order],

```

```

distance[order]))
    status = "interpolated_between_reoptimized_trials"
elif not feasible.empty:
    estimated_range = float(feasible["target_distance_m"].max())
    status = "lower_bound_no_infeasible_bracket"
else:
    estimated_range = float(q3.target_distance_m)
    status = "no_feasible_extension_found"

summary = pd.DataFrame(
    [
        {
            "artifact_id": "q4-T06",
            "method":
"local_reoptimized_range_grid_with_linear_interpolation",
            "claim_level": "local_fixed_fuel_range_estimate",
            "fixed_range_m": q3.target_distance_m,
            "fuel_budget_kg": fixed_fuel_kg,
            "estimated_fixed_fuel_range_m": estimated_range,
            "range_gain_vs_fixed_range_m": estimated_range -
q3.target_distance_m,
            "range_gain_vs_fixed_range_pct": 100.0 * (estimated_range -
q3.target_distance_m) / q3.target_distance_m,
            "trial_count": int(len(trials)),
            "passed_trial_count": int((trials["validation_status"] ==
"passed").sum()),
            "range_status": status,
            "evidence_file":
"questions/q4/artifacts/tables/fixed_fuel_range_trials.csv",
        }
    ]
)
return summary, trials

def _extension_frameworks() -> pd.DataFrame:
    return pd.DataFrame(
        [
            {
                "artifact_id": "q4-T05",
                "framework_id": "temperature_realtime_correction",
                "framework_name": "Real-time temperature and density correction",
                "model_change": "Replace ISA atmosphere calls with a
temperature-offset or measured-profile atmosphere T(h,t), p(h,t), rho(h,t),

```

```

a(h,t); re-integrate pressure hydrostatically before evaluating drag, Mach,
and fuel penalty.",
    "required_data": "Along-route temperature deviation profile or
onboard/forecast temperature field; pressure reference or hydrostatic
boundary; timestamped altitude samples.",
    "validation_plan": "Compare corrected atmosphere against q2
hydrostatic temperature scenarios, verify dp/dh=-rho g residual, and rerun
q4-T02/T04 under matched temperature offsets.",
    "implementation_interface": "Extend Q2Parameters with temperature
profile provider and pass temperature_offset/profile into q4_rates.",
    "evidence_boundary": "Framework only; no new numerical
fuel-saving claim until temperature scenarios are re-optimized.",
    "status": "framework_only",
},
{
    "artifact_id": "q4-T05",
    "framework_id": "engine_installation_loss",
    "framework_name": "Engine installation and altitude-dependent
thrust loss",
    "model_change": "Replace ideal commanded thrust T with effective
thrust eta_install(h,M,T) T and add installation fuel-flow multiplier
phi_install(h,M); keep thrust bounds on commanded thrust.",
    "required_data": "Engine deck or empirical installation
efficiency table over altitude, Mach, and thrust setting; uncertainty bounds
for efficiency multipliers.",
    "validation_plan": "Run zero-loss recovery test, monotonic loss
sanity checks, and sensitivity over eta_install={0.95,0.98,1.00}; compare
terminal constraints and fuel impact.",
    "implementation_interface": "Add installation_loss_model callable
to q4_rates before drag balance and fuel-flow evaluation.",
    "evidence_boundary": "Framework only; current q4-T02/T04 use
ideal installed thrust and must not claim installed-engine accuracy.",
    "status": "framework_only",
},
]
)

def run_pipeline(
    *,
    config_path: str,
    nodes: int,
    control_knots: int,
    maxiter: int,

```



```

794     beta_factors: list[float],
795     range_factors: list[float],
796     sensitivity_maxiter: int,
797     range_maxiter: int,
798     skip_beta_sensitivity: bool = False,
799     skip_range_extension: bool = False,
800     skip_extension_frameworks: bool = False,
801 ) -> dict[str, Path]:
802     if nodes < 11:
803         raise ValueError("--nodes must be at least 11")
804     if control_knots < 3:
805         raise ValueError("--control-knots must be at least 3")
806     root = project_root()
807     qdir = root / "questions" / "q4"
808     q3 = load_q3_config(root, config_path)
809     params = Q2Parameters(terminal_mass_kg=q3.terminal_mass_min_kg)
810     source = _load_no_wind_trajectory(root)
811     q2_baseline = _load_q2_standard_baseline(root)
812     q1_constant_speed = _load_q1_constant_speed_summary(root)
813     q1_profile = _load_q1_constant_speed_profile(root)
814
815     evaluation, optimizer_result, final_vector = _solve_wind_optimal(
816         q3=q3,
817         params=params,
818         source=source,
819         nodes=nodes,
820         control_knots=control_knots,
821         maxiter=maxiter,
822     )
823     validation_evaluation = _evaluate_controls(
824         final_vector,
825         q3=q3,
826         params=params,
827         control_knots=control_knots,
828         nodes=max(241, nodes * 3),
829     )
830     fuel_identity_residual =
831     _fuel_identity_residual(validation_evaluation.trajectory, params)
832     wind_results = _wind_result_row(
833         evaluation,
834         q3=q3,
835         params=params,
836         optimizer_result=optimizer_result,
837         continuation_nodes=str(nodes),

```

```

837     control_knots=control_knots,
838     fuel_identity_residual_kg=fuel_identity_residual,
839 )
840 comparison = _strategy_comparison(
841     q2_baseline=q2_baseline,
842     q1_constant_speed=q1_constant_speed,
843     wind_results=wind_results,
844 )
845
846 outputs: dict[str, Path] = {}
847 outputs.update({f"wind_optimal_results_{key}": value for key, value in
save_table(wind_results, stem="wind_optimal_results",
question_dir=qdir).items()})
848 outputs.update({f"wind_optimal_trajectory_{key}": value for key, value in
save_table(evaluation.trajectory, stem="wind_optimal_trajectory",
question_dir=qdir).items()})
849 outputs.update({f"strategy_comparison_{key}": value for key, value in
save_table(comparison, stem="strategy_comparison",
question_dir=qdir).items()})
850 if not skip_beta_sensitivity:
851     beta = _beta_sensitivity(
852         q3=q3,
853         nominal_params=params,
854         source=source,
855         nominal_evaluation=evaluation,
856         nominal_optimizer_result=optimizer_result,
857         nominal_vector=final_vector,
858         nodes=nodes,
859         control_knots=control_knots,
860         maxiter=sensitivity_maxiter,
861         beta_factors=beta_factors,
862     )
863     outputs.update({f"beta_sensitivity_{key}": value for key, value in
save_table(beta, stem="beta_sensitivity", question_dir=qdir).items()})
864 if not skip_range_extension:
865     range_summary, range_trials = _range_extension(
866         q3=q3,
867         params=params,
868         source=source,
869         q1_profile=q1_profile,
870         nominal_evaluation=evaluation,
871         nominal_optimizer_result=optimizer_result,
872         nominal_vector=final_vector,
873         nodes=nodes,

```

```

874         control_knots=control_knots,
875         maxiter=range_maxiter,
876         range_factors=range_factors,
877     )
878     outputs.update({f"fixed_fuel_range_{key}": value for key, value in
save_table(range_summary, stem="fixed_fuel_range",
question_dir=qdir).items()})
879     outputs.update({f"fixed_fuel_range_trials_{key}": value for key, value in
save_table(range_trials, stem="fixed_fuel_range_trials",
question_dir=qdir).items()})
880     if not skip_extension_frameworks:
881         extensions = _extension_frameworks()
882         outputs.update({f"extension_frameworks_{key}": value for key, value in
save_table(extensions, stem="extension_frameworks",
question_dir=qdir).items()})
883     return outputs
884
885
886 def main() -> int:
887     args = parse_args()
888     root = project_root()
889     question_dir = root / "questions" / "q4"
890
891     steps = [
892         "load q1/q2/q3 upstream artifacts",
893         "initialize configured-wind reduced-control shooting from q3 no-wind
trajectory",
894         "solve q4-T02 wind-aware fixed-range fuel objective",
895         "save wind_optimal_results and wind_optimal_trajectory",
896         "build q4-T03 fixed-range strategy comparison",
897         "validate terminal state, constraints, mass floor, and fuel identity",
898         "rerun beta sensitivity by scenario and save q4-T04",
899         "estimate fixed-fuel local range extension and save q4-T06",
900         "save q4-T05 extension frameworks",
901     ]
902     if args.dry_run:
903         print("q4 planned pipeline:")
904         for index, step in enumerate(steps, start=1):
905             print(f" {index}. {step}")
906         print(f"question_dir={question_dir}")
907         print(f"config={root / args.config}")
908         print(f"nodes={args.nodes}")
909         print(f"control_knots={args.control_knots}")
910     return 0

```

```

911
912     outputs = run_pipeline(
913         config_path=args.config,
914         nodes=args.nodes,
915         control_knots=args.control_knots,
916         maxiter=args.maxiter,
917         beta_factors=_parse_float_list(args.beta_factors,
option_name="--beta-factors"),
918         range_factors=_parse_float_list(args.range_factors,
option_name="--range-factors"),
919         sensitivity_maxiter=args.sensitivity_maxiter,
920         range_maxiter=args.range_maxiter,
921         skip_beta_sensitivity=args.skip_beta_sensitivity,
922         skip_range_extension=args.skip_range_extension,
923         skip_extension_frameworks=args.skip_extension_frameworks,
924     )
925     for name, path in sorted(outputs.items()):
926         print(f"{name}={path}")
927     return 0
928
929
930 if __name__ == "__main__":
931     raise SystemExit(main())

```